COSC2637 - Big Data Processing

Lecture 6

MapReduce Processing Large Graphs

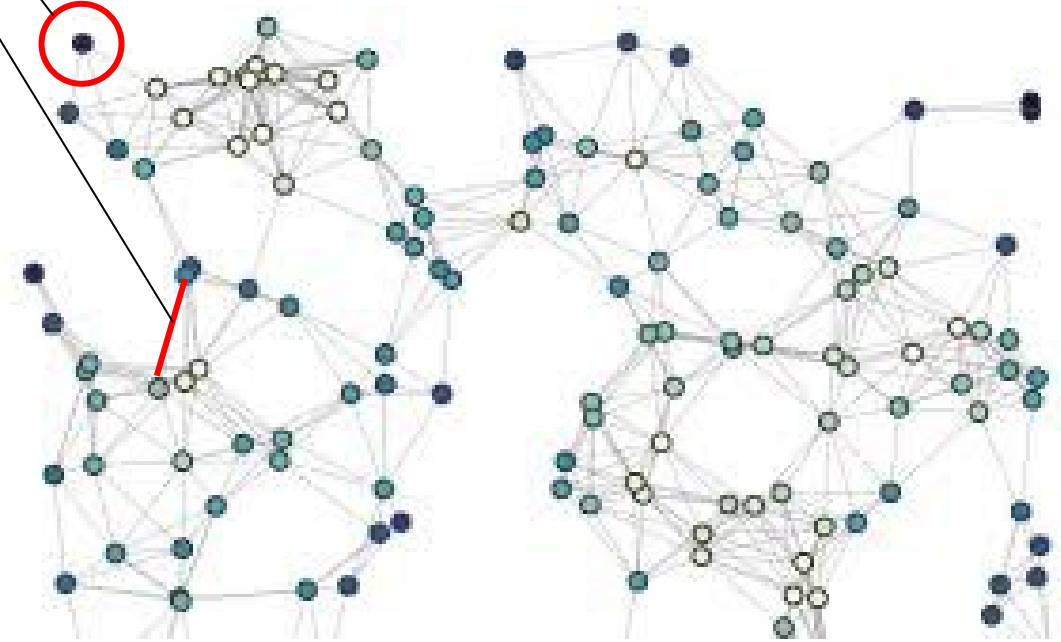
RMIT UNIVERSITY

Part I

Graph problems and representations

What's a graph?

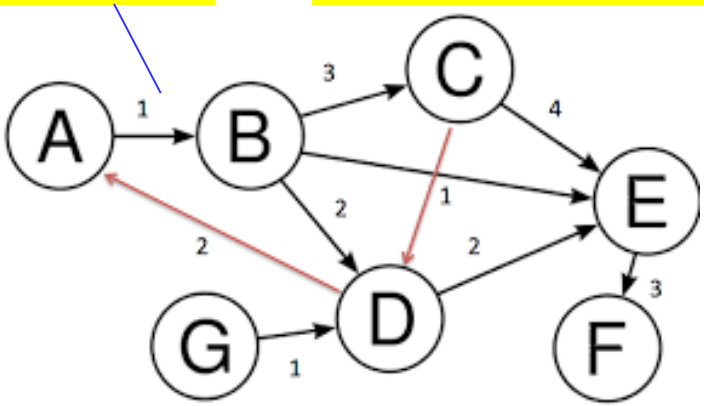
- $G = (V, E)$ where
  - V represents the set of vertices (nodes)
  - E represents the set of edges (links)



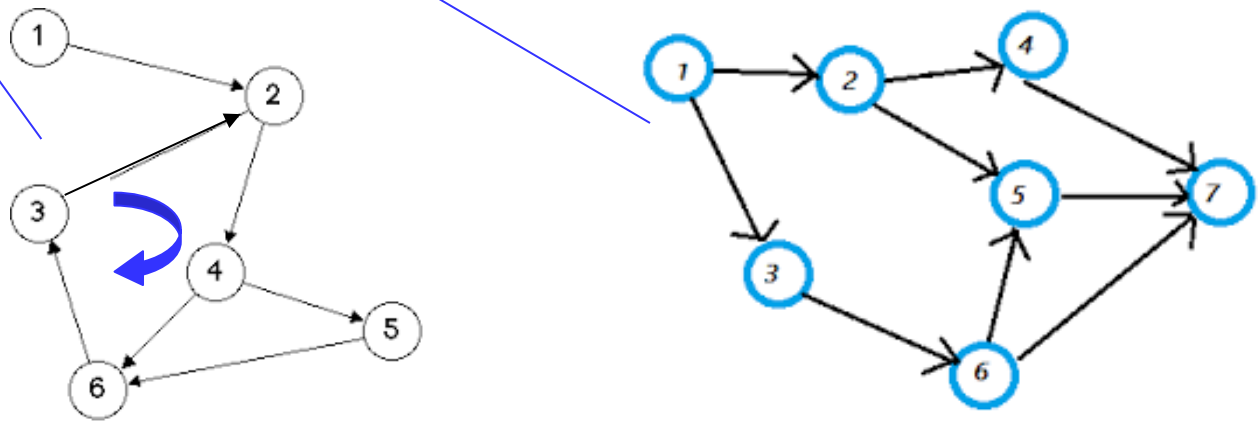
Both vertices and edges may contain additional information

What's a graph?

- Different types of graphs:
  - Directed vs. undirected edges

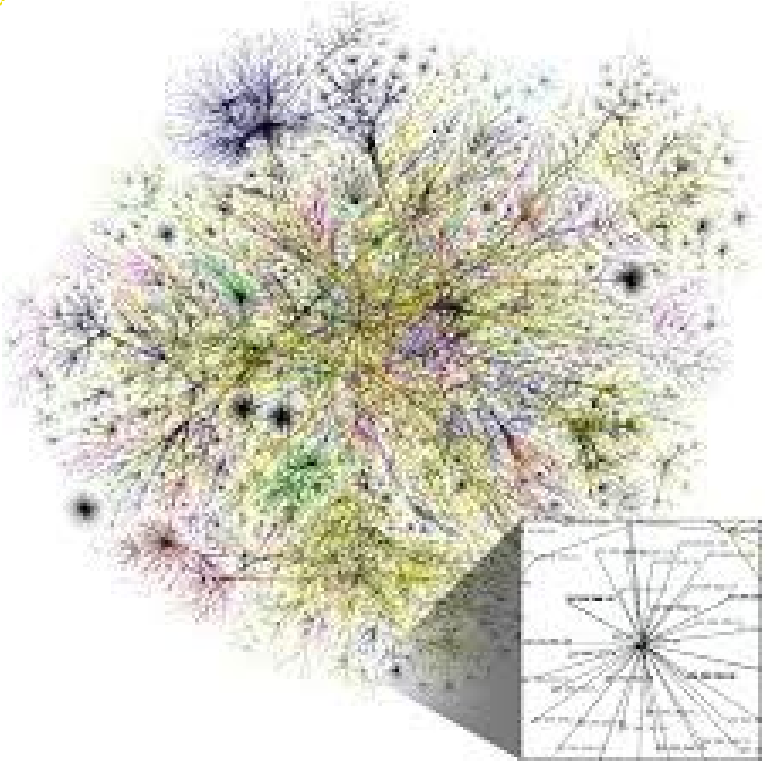


  - Presence or absence of cycles (directed graph)

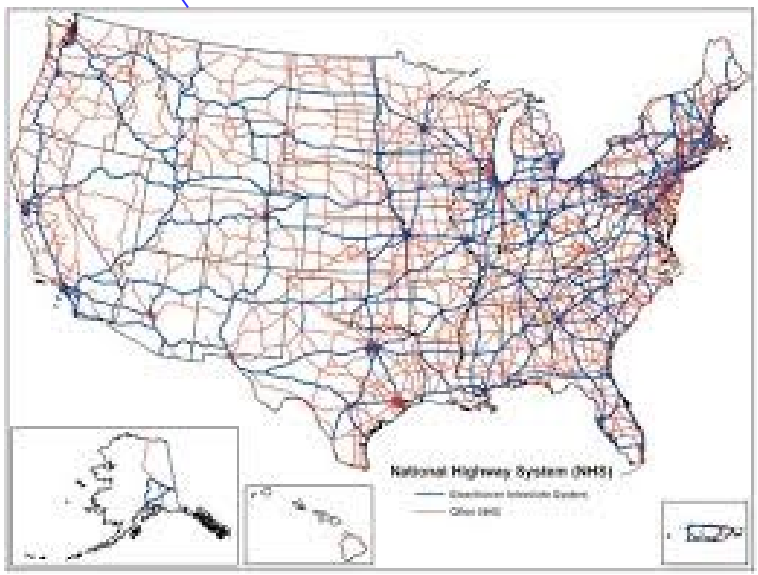


What's a graph?

- Graphs are everywhere:
  - Hyperlink structure of the web
  - Interstate highway system
  - Social networks
  - …





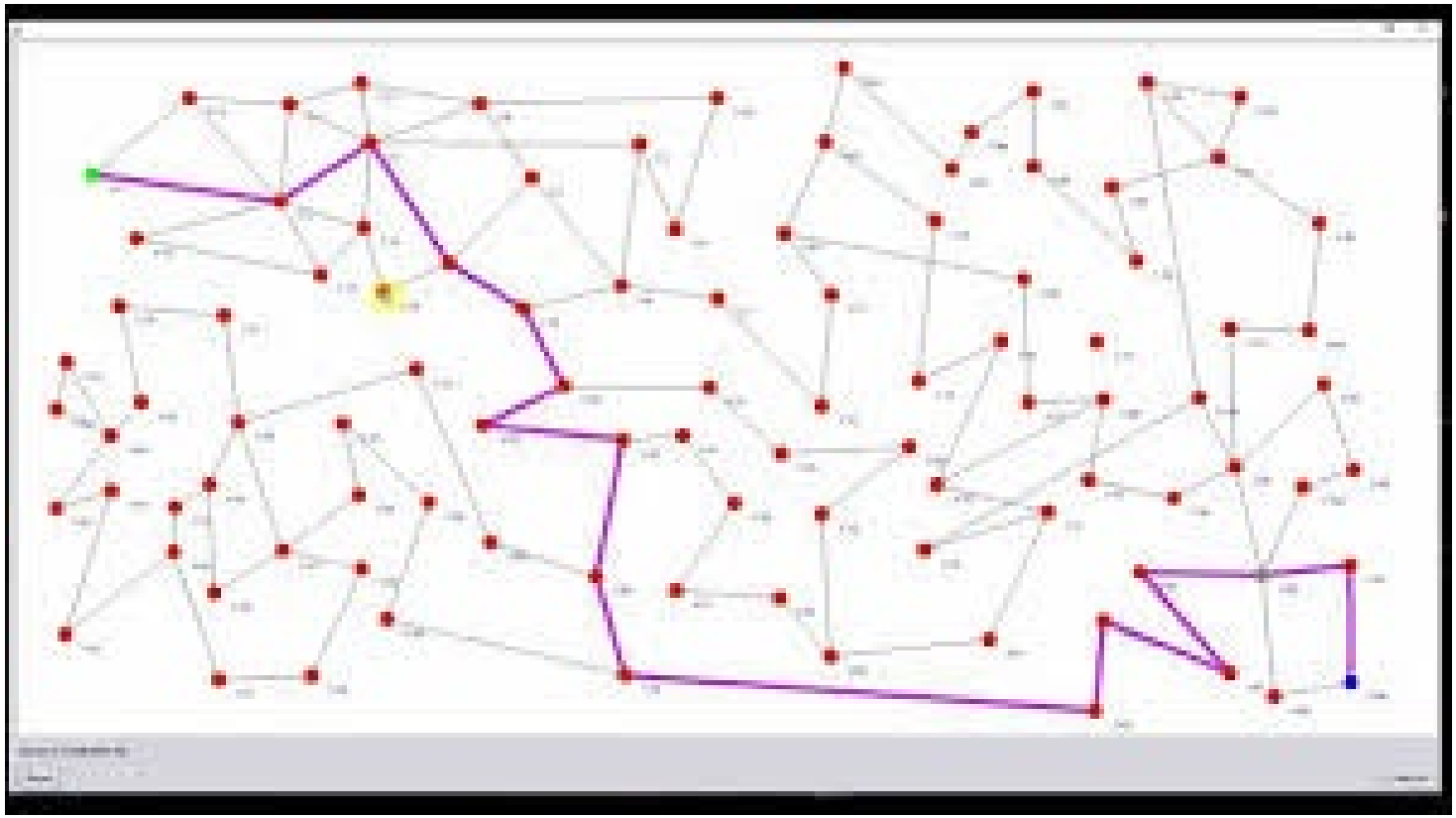


Some Graph Problems

- Finding shortest paths
  - Routing Internet traffic and UPS trucks
- Finding minimum spanning trees
  - Telco laying down fiber
- Finding Max Flow
  - Airline scheduling
- Identify “special” nodes and communities
  - Breaking up terrorist cells, spread of avian flu
- Bipartite matching
  - Monster.com, Match.com
- And of course... PageRank

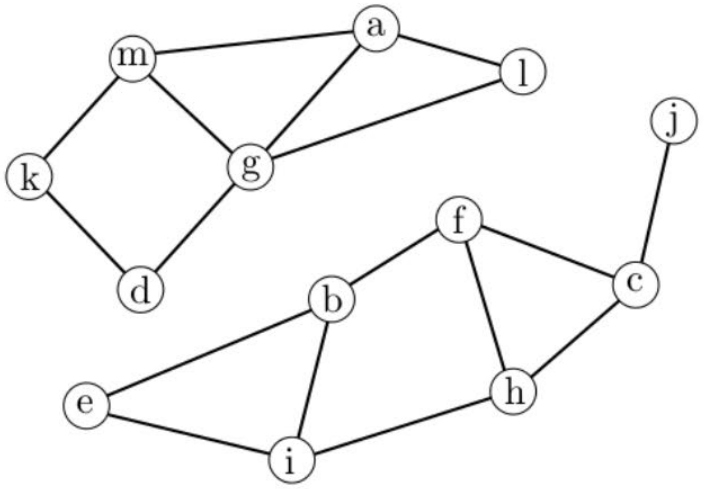
Some Graph Problems

- Finding shortest paths
  - Routing Internet traffic and UPS trucks



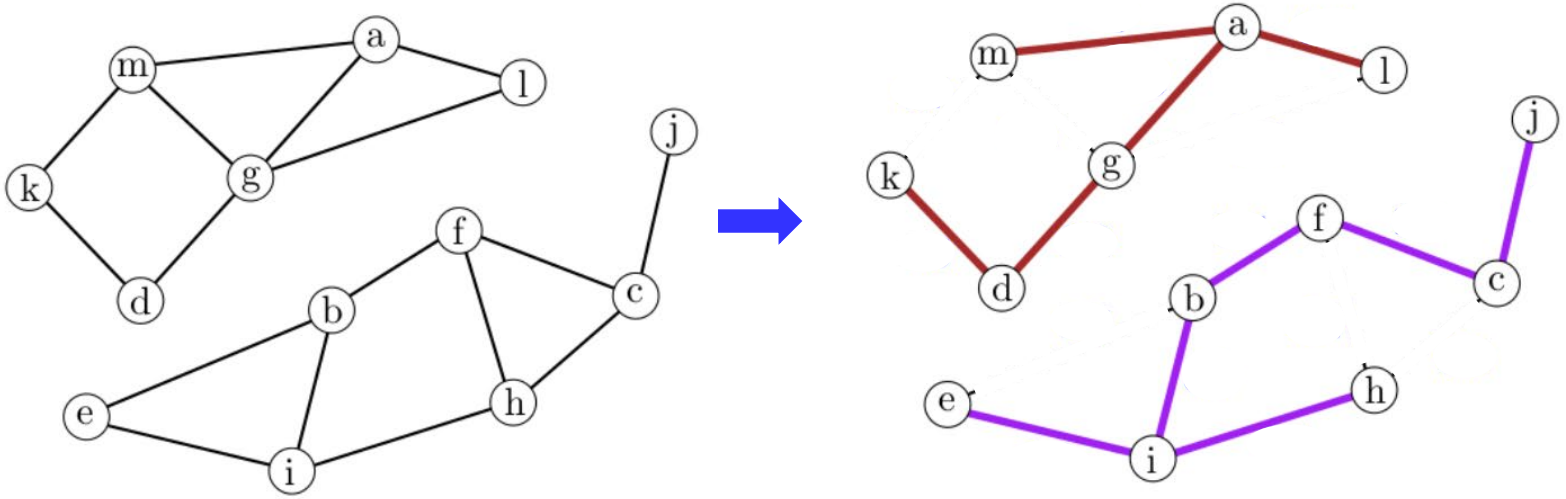
Some Graph Problems

- Finding minimum spanning trees



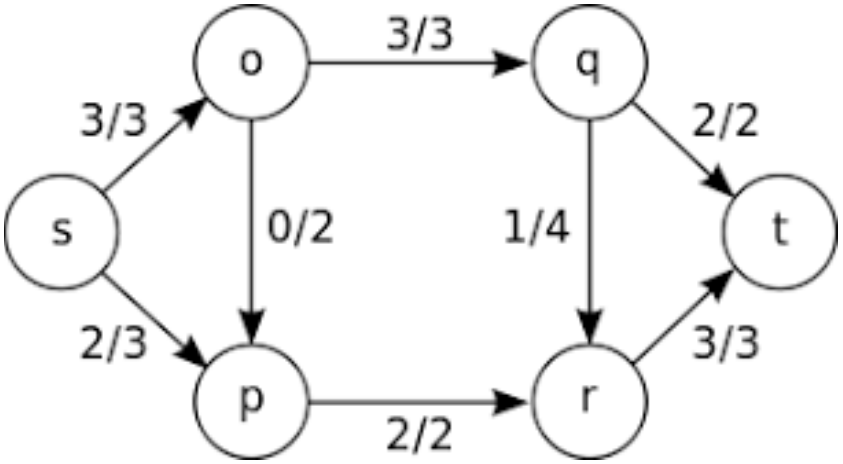
Some Graph Problems

- Finding minimum spanning trees



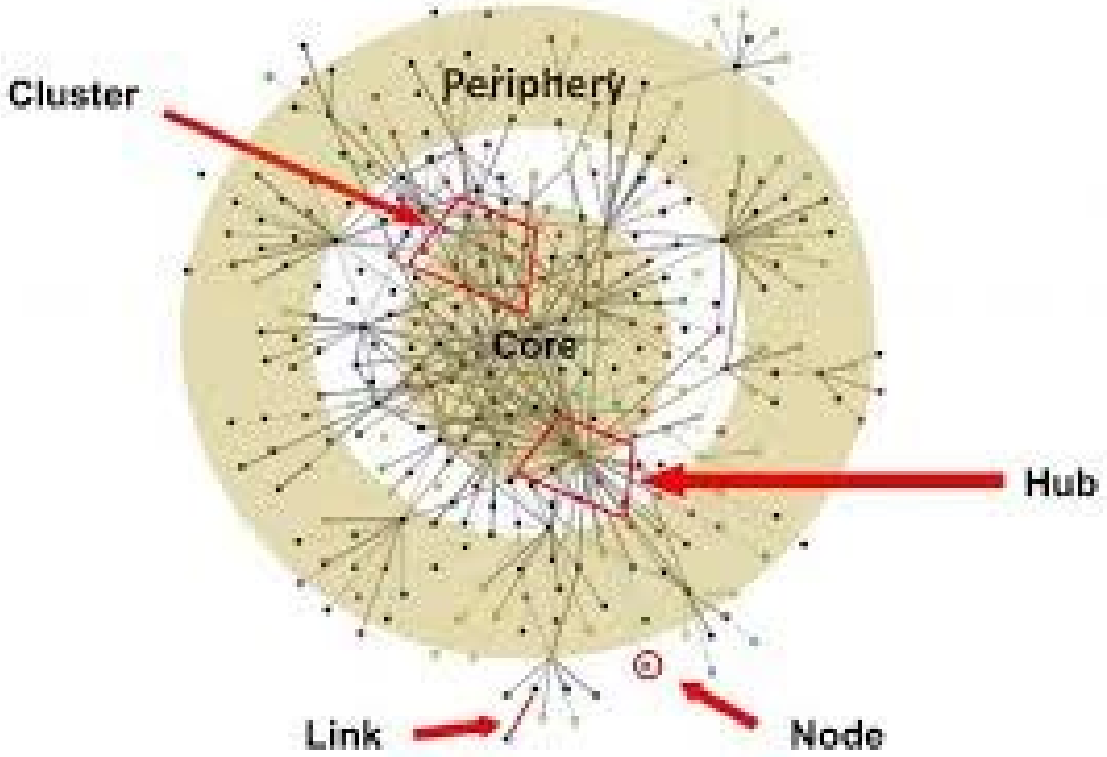
Some Graph Problems

- Finding Max Flow
  - It is defined as the maximum amount of flow that the network would allow to flow from source to sink.



Some Graph Problems

- Identify “special” nodes and communities
  - Breaking up terrorist cells, spread of avian flu



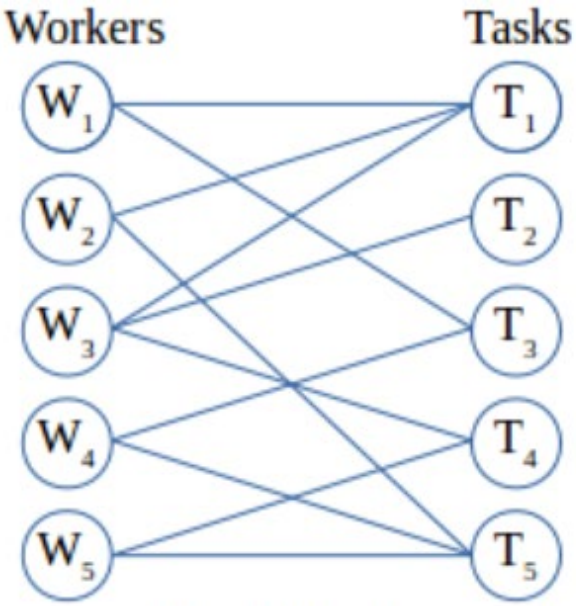
Some Graph Problems

- Bipartite matching
  - Monster.com, Match.com

W1 : T1, T3
W2 : T1, T5
W3 : T1, T2, T4
W4 : T3, T5
W5 : T4, T5

T1 : W1, W2, W3
T2 : W3
T3 : W1, W4
T4 : W3, W5
T5 : W2, W4, W5

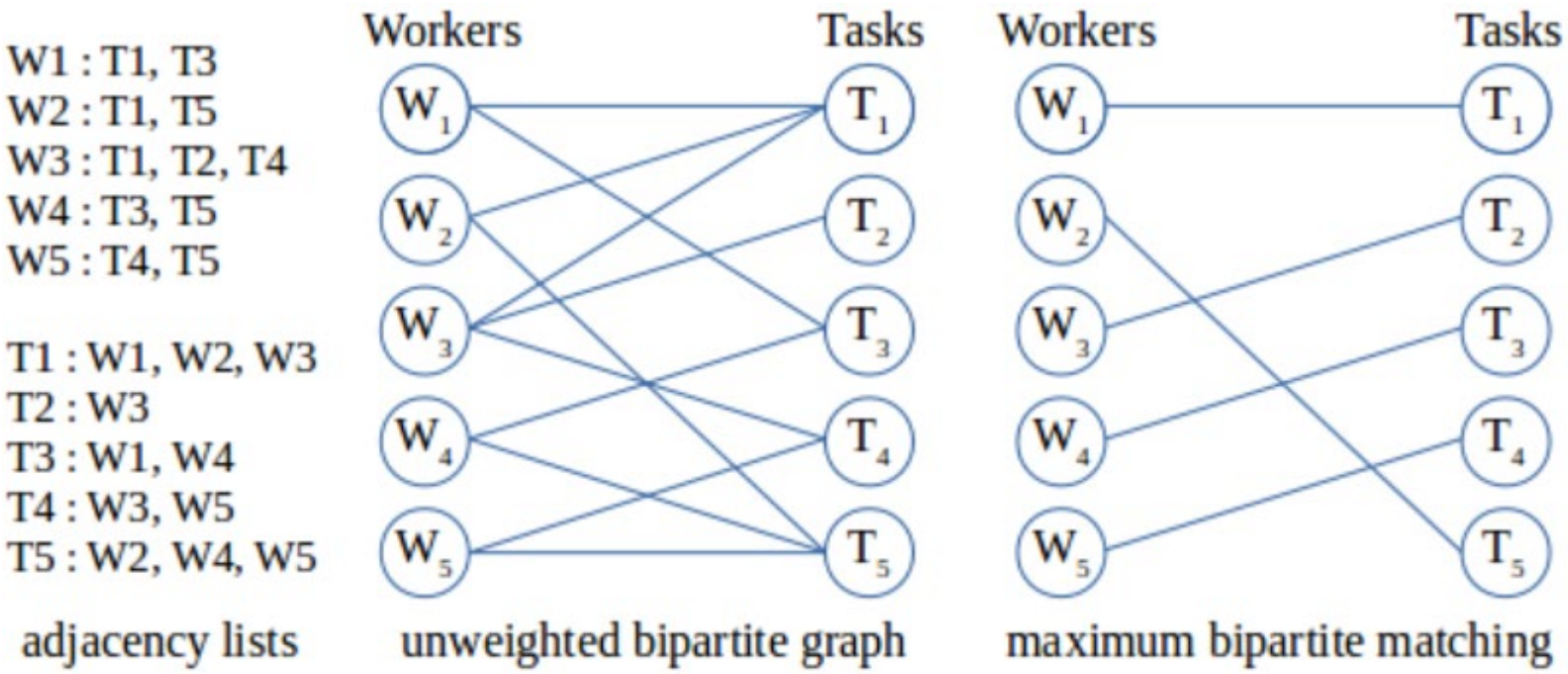
adjacency lists



unweighted bipartite graph

Some Graph Problems

- Bipartite matching
  - Monster.com, Match.com



Graphs and MapReduce

- A large class of graph algorithms involve:
  - Performing computations at each node
    - based on node features, edge features, and local link structure
  - Propagating computations
    - "traversing" the graph
- Key questions:
  - How do you represent graph data in MapReduce?
  - How do you traverse a graph in MapReduce?

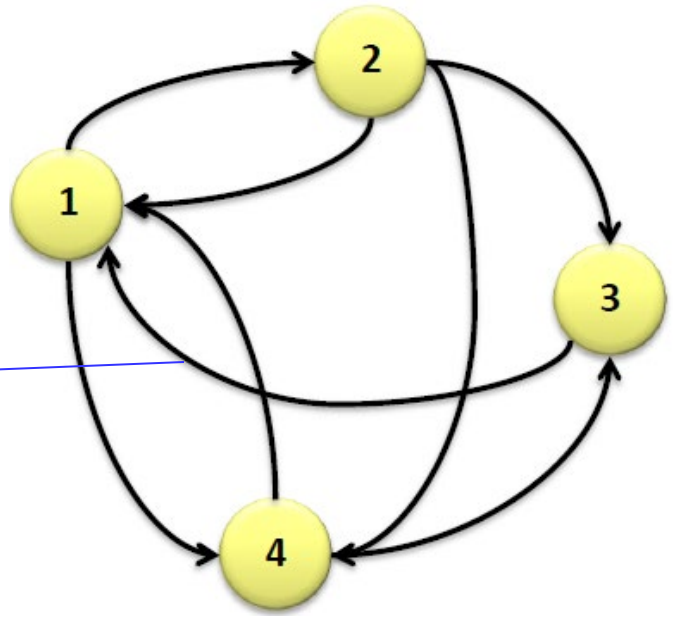
Representing Graphs

- Two common representations of G = (V, E)
  - Adjacency matrix
  - Adjacency list

Representing Graphs - Adjacency Matrices

- Represent a graph as an $n \times n$ square matrix M
  - $n = |V|$
  - $M_{ij} = 1$ means a link from node i to j

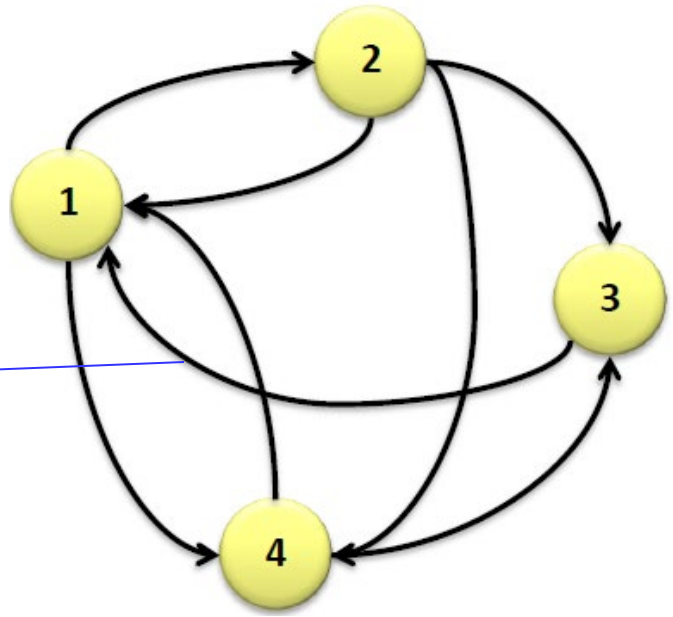
	1	2	3	4
1	0	1	0	1
2	1	0	1	1
3	1	0	0	0
4	1	0	1	0



Representing Graphs - Adjacency Matrices

- Represent a graph as an $n \times n$ square matrix M
  - $n = |V|$
  - $M_{ij} = 1$ means a link from node i to j

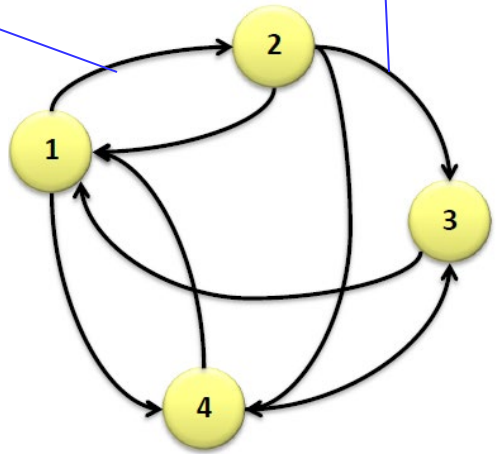
	1	2	3	4
1	0	1	0	1
2	1	0	1	1
3	1	0	0	0
4	1	0	1	0



Representing Graphs - Adjacency Matrices

- Advantages:
  - Amenable to mathematical manipulation
  - Iteration over rows and columns corresponds to computations on *outlinks* and *inlinks*
- Disadvantages:
  - Lots of zeros for sparse matrices
  - Lots of wasted space

	1	2	3	4
1	0	1	0	1
2	1	0	1	1
3	1	0	0	0
4	1	0	1	0



Representing Graphs

- Two common representations of G = (V, E)
  - Adjacency matrix
  - Adjacency list

Representing Graphs - Adjacency Lists

Take adjacency matrices… and throw away all the zeros

	1	2	3	4
1	~~0~~	1	~~0~~	1
2	1	0	1	1
3	1	0	0	0
4	1	0	1	0



1: 2, 4

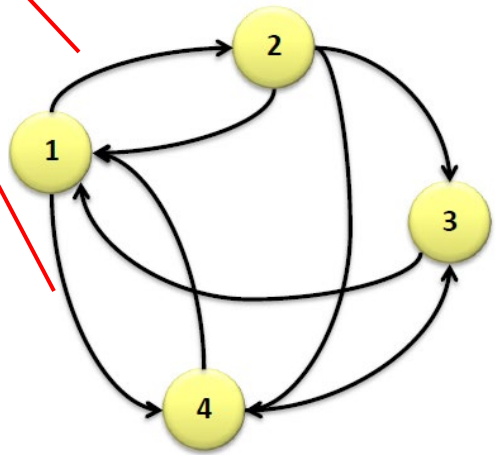
Representing Graphs - Adjacency Lists

- Advantages:
  - Much more compact representation
  - Easy to compute over *outlinks*
- Disadvantages:
  - Much more difficult to compute over *inlinks*

	1	2	3	4
1	0	1	0	1
2	1	0	1	1
3	1	0	0	0
4	1	0	1	0



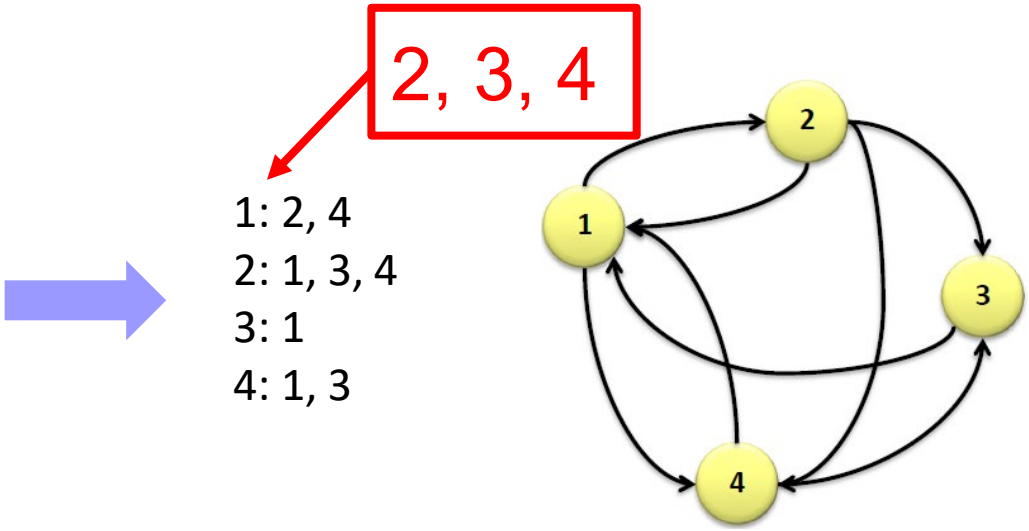
1: 2, 4
2: 1, 3, 4
3: 1
4: 1, 3



Representing Graphs - Adjacency Lists

- Advantages:
  - Much more compact representation
  - Easy to compute over *outlinks*
- Disadvantages:
  - Much more difficult to compute over *inlinks*

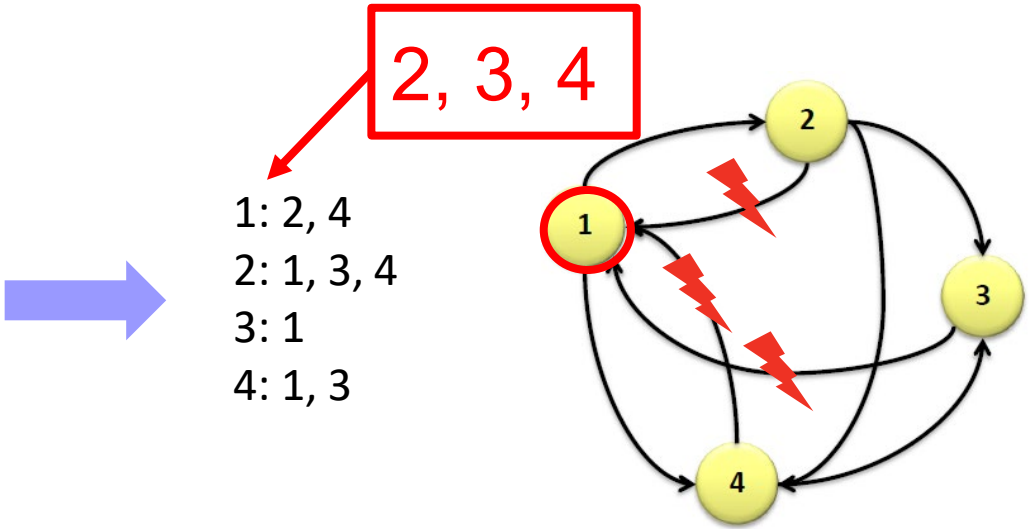
	1	2	3	4
1	0	1	0	1
2	1	0	1	1
3	1	0	0	0
4	1	0	1	0



Representing Graphs - Adjacency Lists

- Advantages:
  - Much more compact representation
  - Easy to compute over *outlinks*
- Disadvantages:
  - Much more difficult to compute over *inlinks*

	1	2	3	4
1	0	1	0	1
2	1	0	1	1
3	1	0	0	0
4	1	0	1	0



End of Part I

Part II

Single-Source Shortest Path on Graph with MapReduce

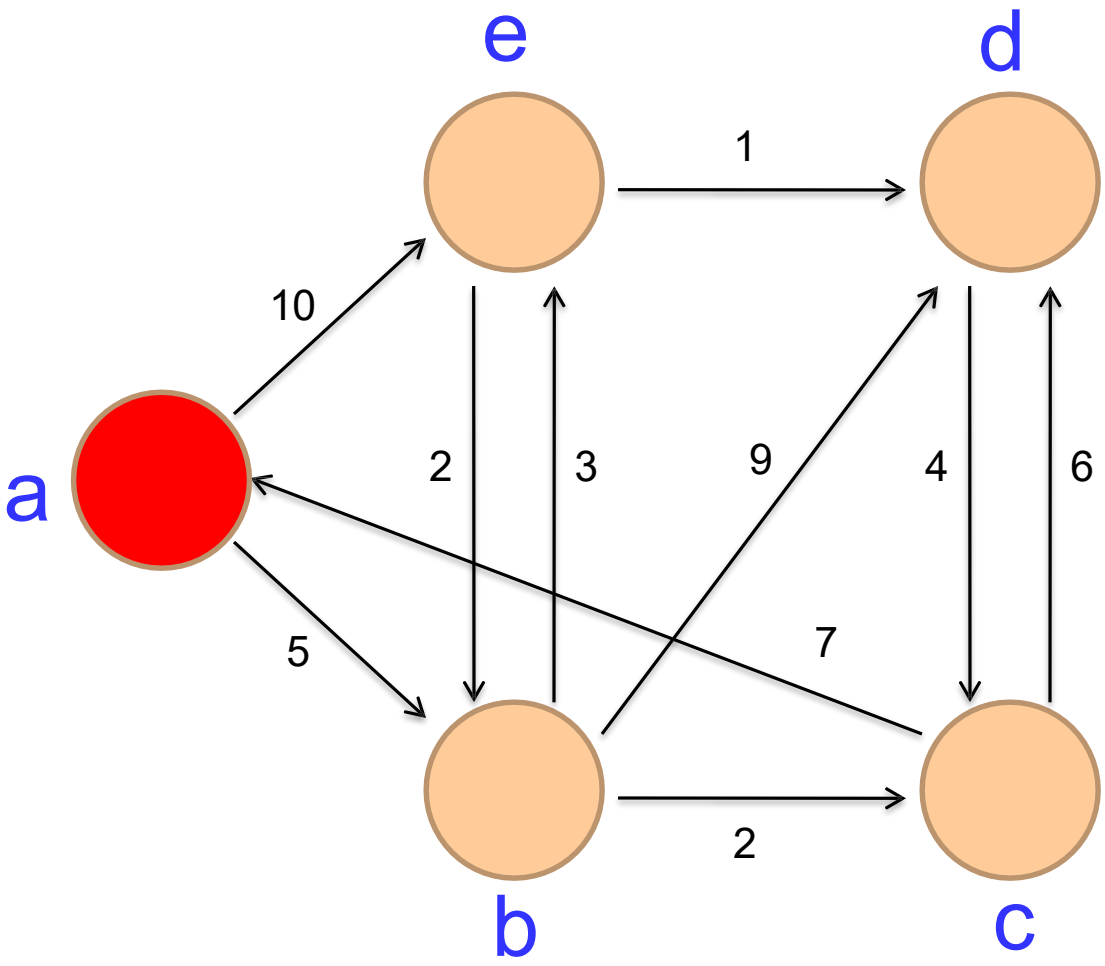
Single-Source Shortest Path

Problem: find shortest path from a source node to one or more target nodes

- Shortest might also mean lowest weight or cost
- Single processor machine: Dijkstra's Algorithm
- MapReduce: Parallel Breadth-First search (BFS)

Dijkstra's Algorithm Example

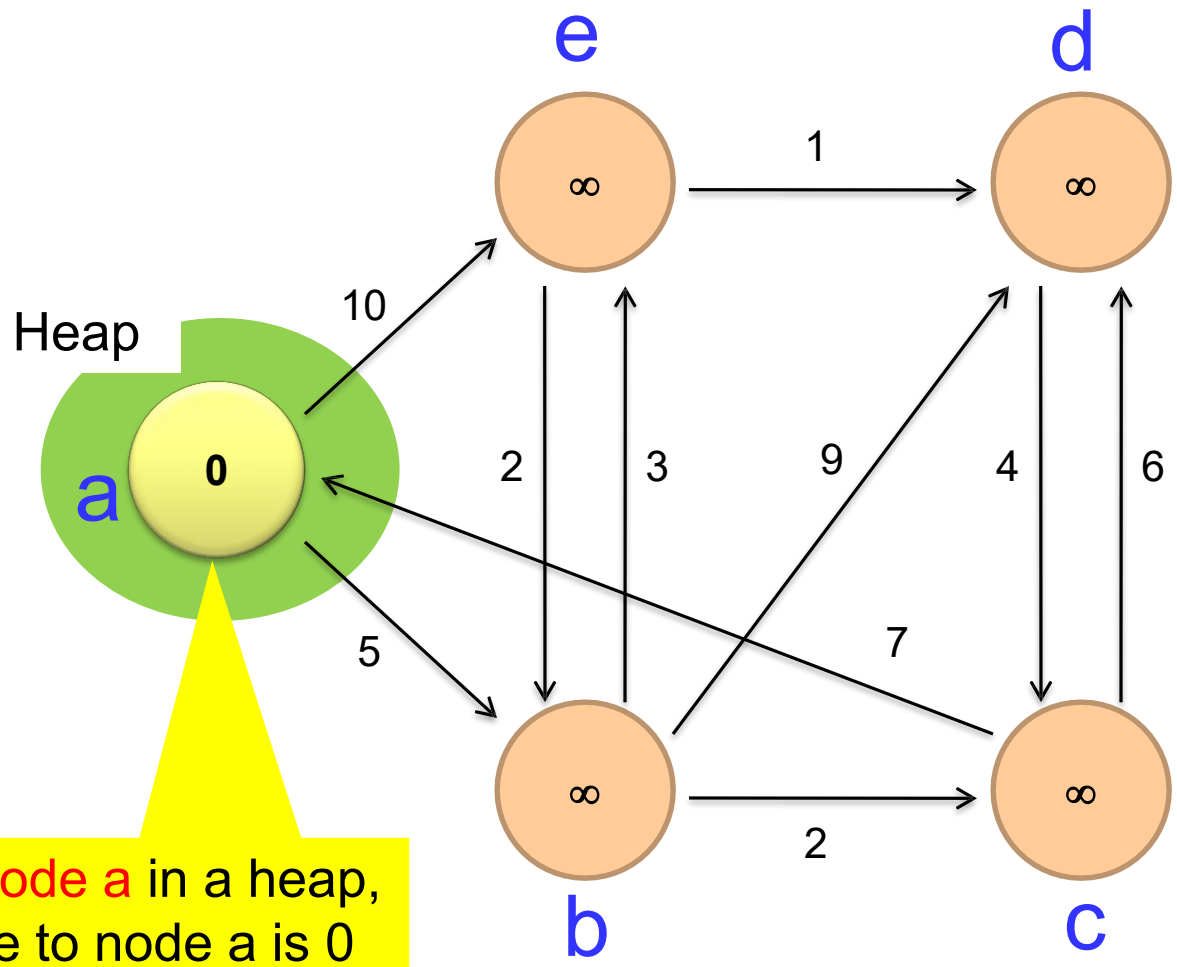
find shortest path from a to other nodes



Example from CLR

Dijkstra's Algorithm Example

find shortest path from a to other nodes

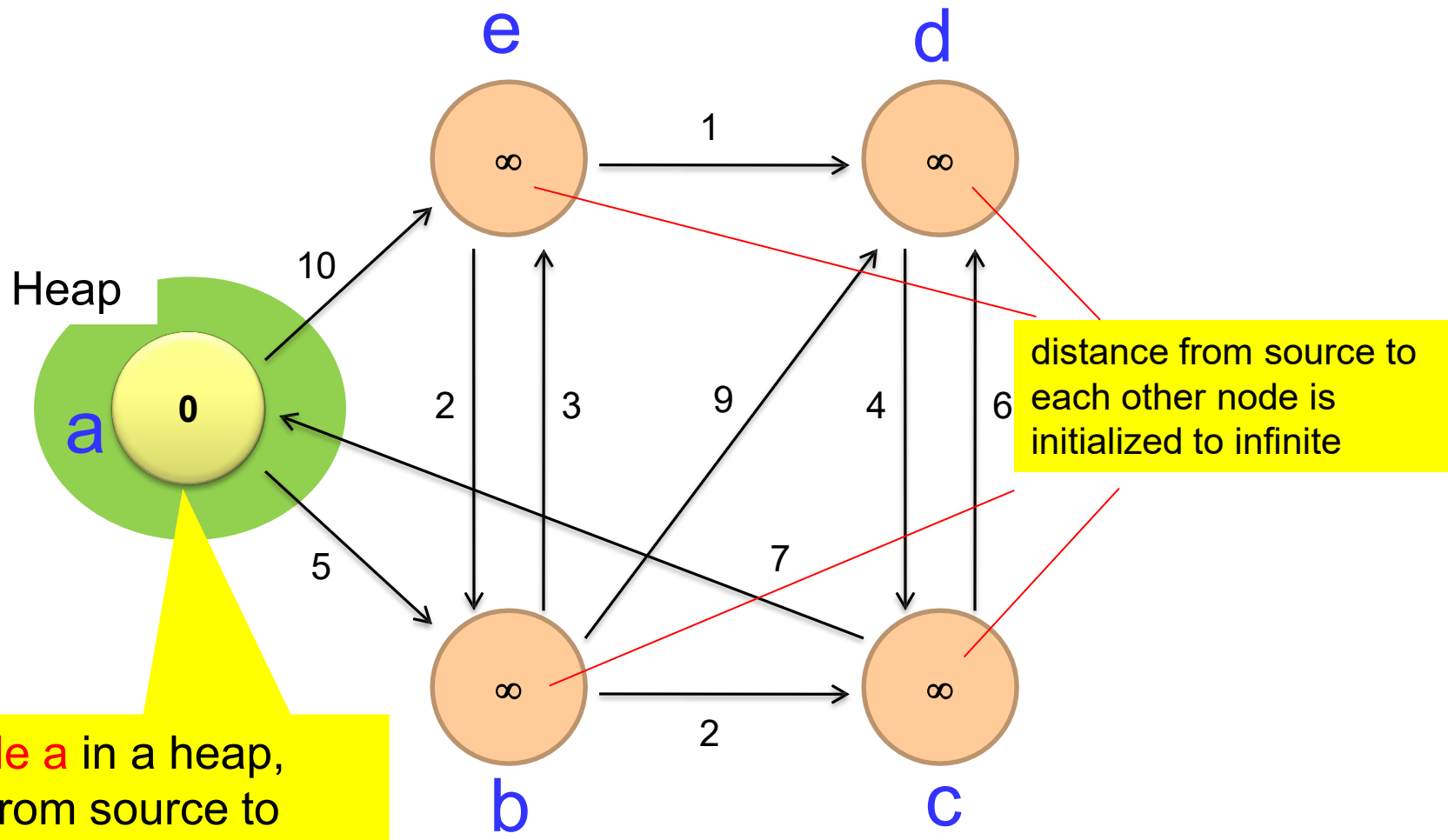


- Insert node a in a heap, distance to node a is 0

Example from CLR

Dijkstra's Algorithm Example

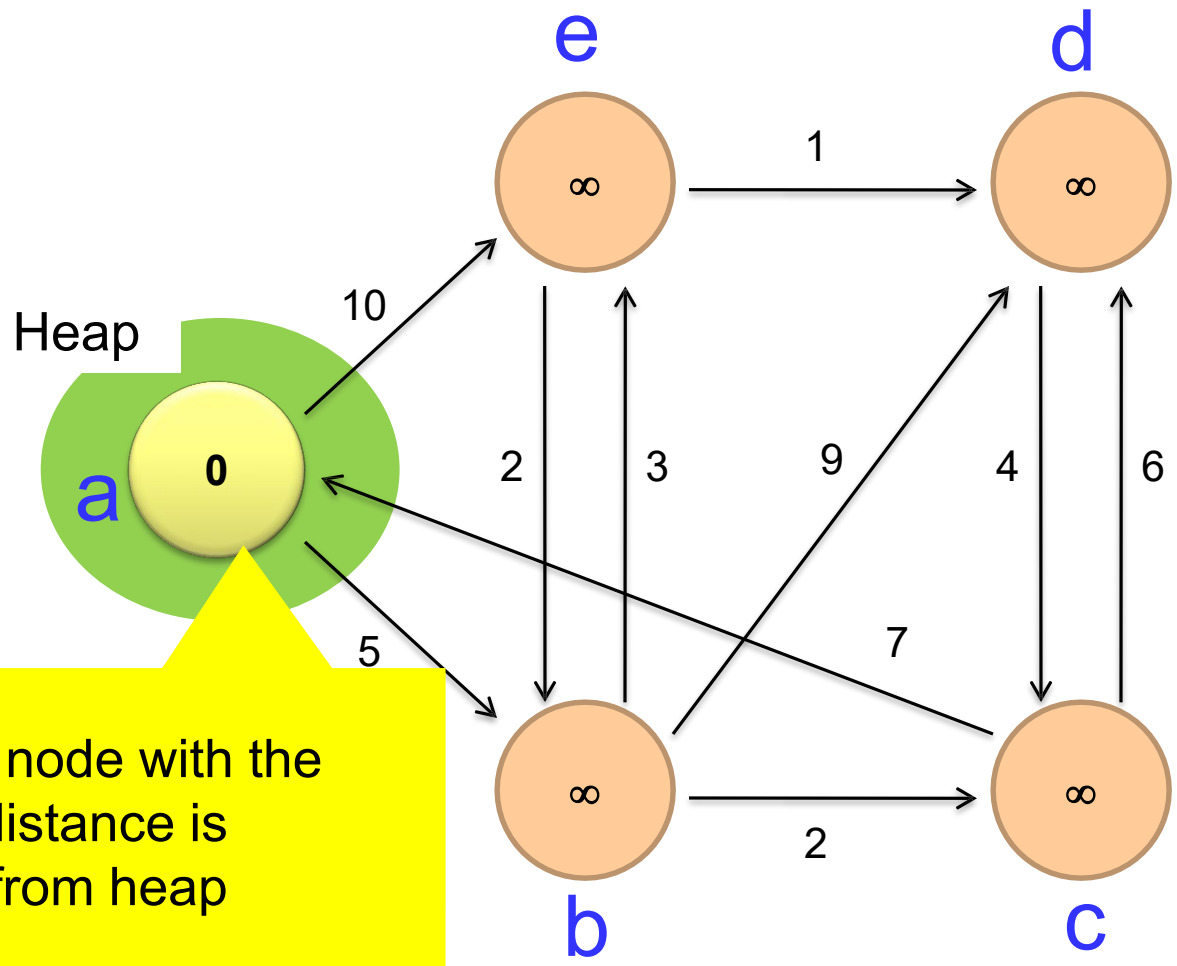
find shortest path from a to other nodes



Example from CLR

Dijkstra's Algorithm Example

find shortest path from a to other nodes



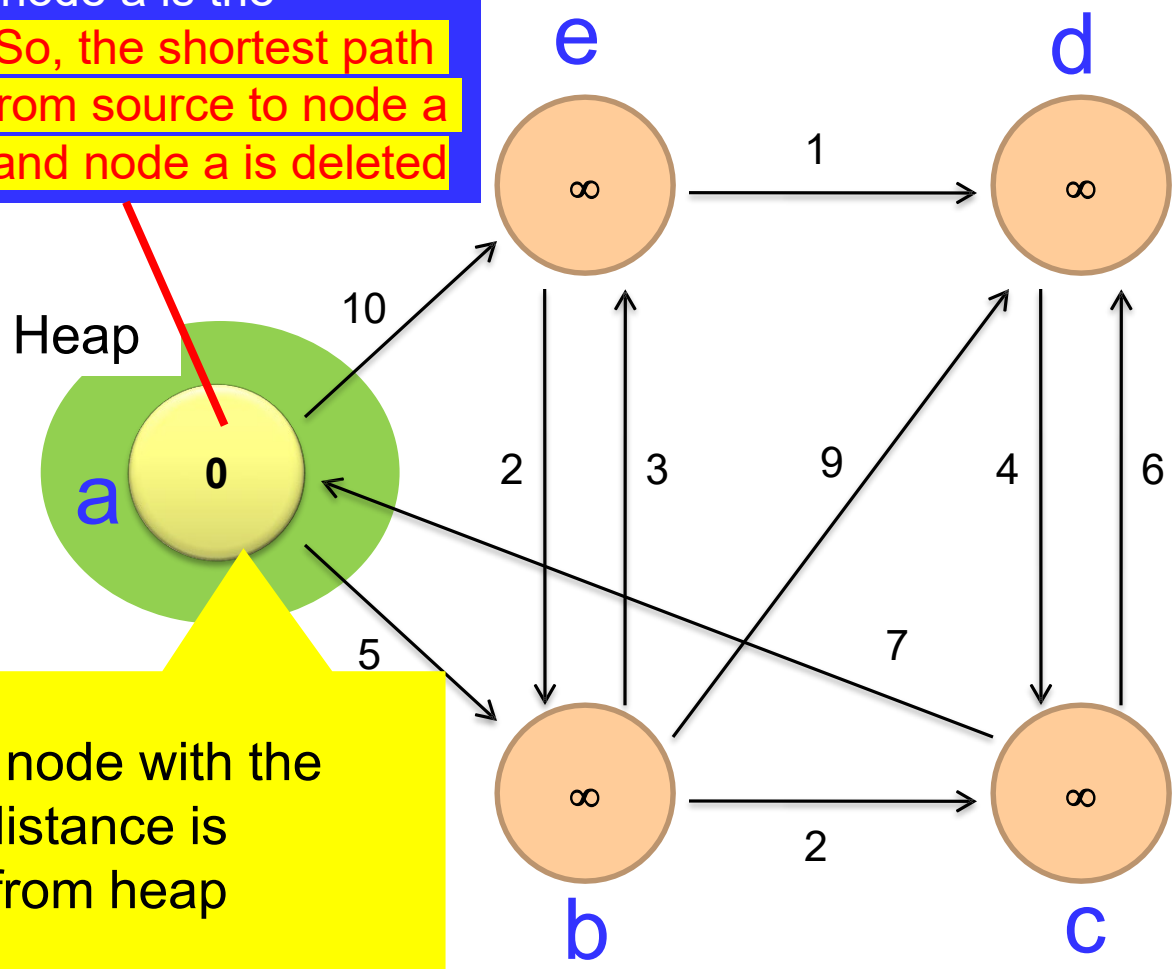
Then, the node with the smallest distance is removed from heap

Example from CLR

Dijkstra's Algorithm Example

t path from a to other nodes

In the Heap, the distance from source to node a is the smallest. So, the shortest path distance from source to node a is known and node a is deleted

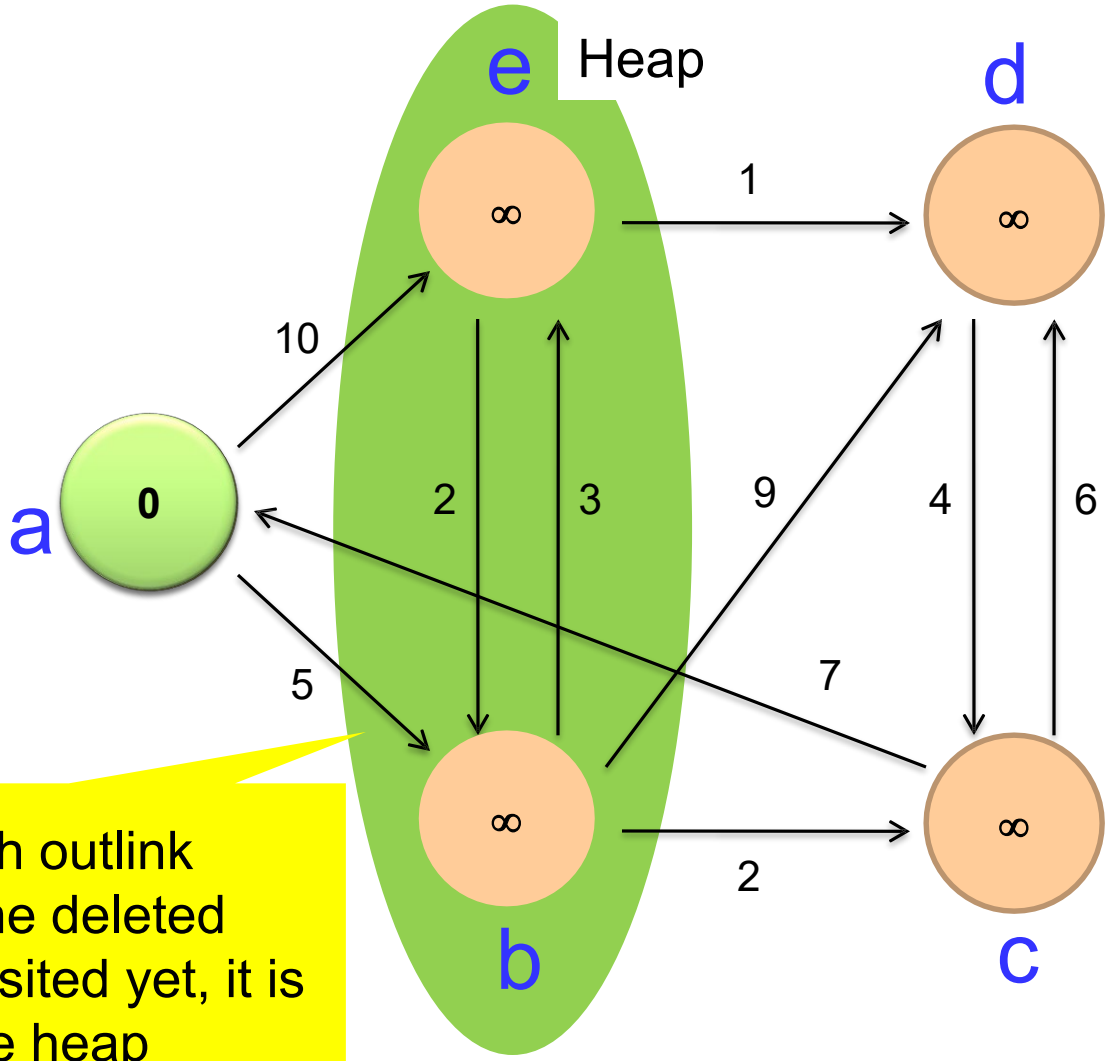


Then, the node with the smallest distance is removed from heap

Example from CLR

Dijkstra's Algorithm Example

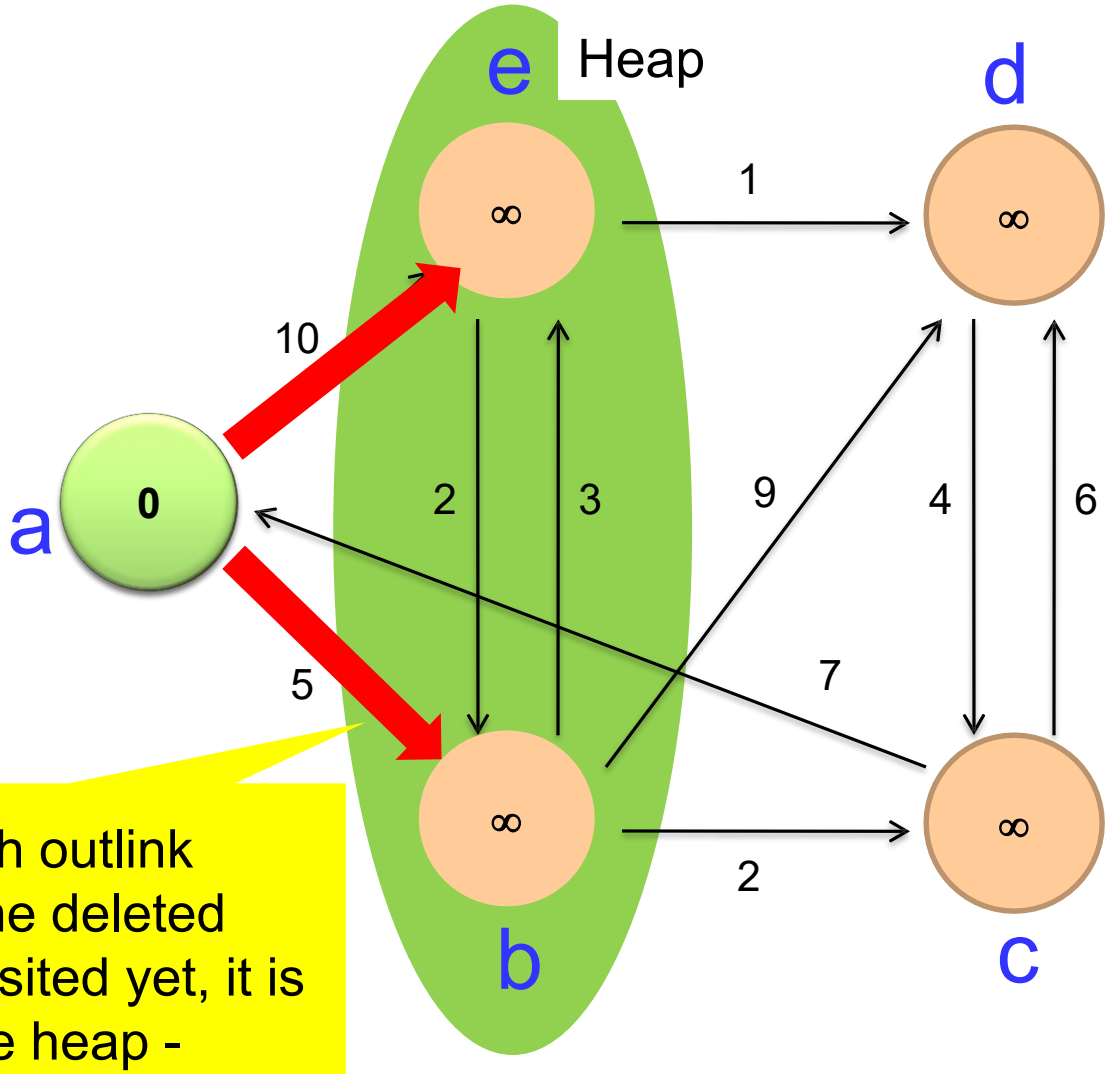
find shortest path from a to other nodes



Then, for each outlink neighbor of the deleted node, if not visited yet, it is inserted in the heap

Dijkstra's Algorithm Example

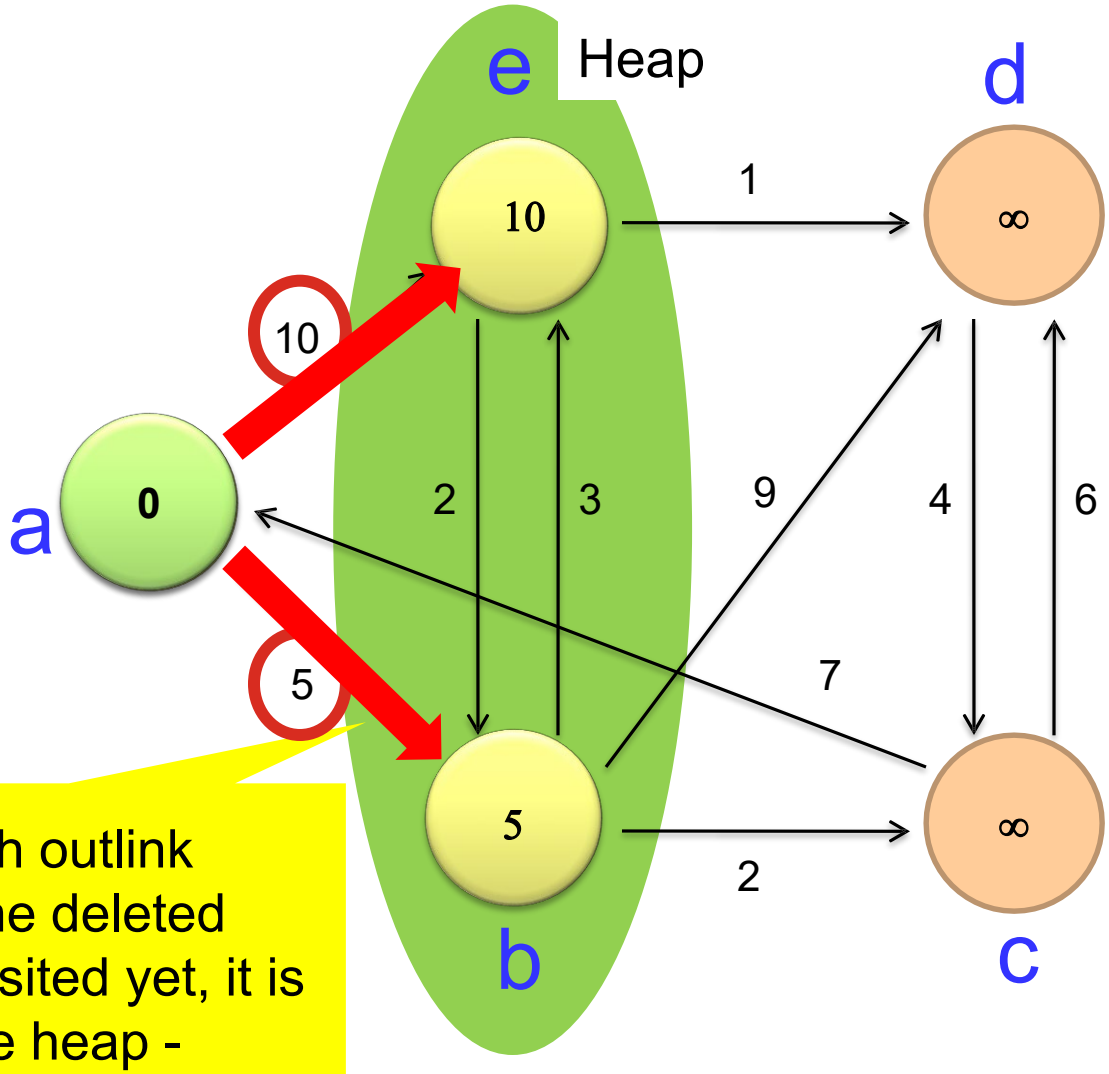
find shortest path from a to other nodes



Then, for each outlink neighbor of the deleted node, if not visited yet, it is inserted in the heap - update distance

Dijkstra's Algorithm Example

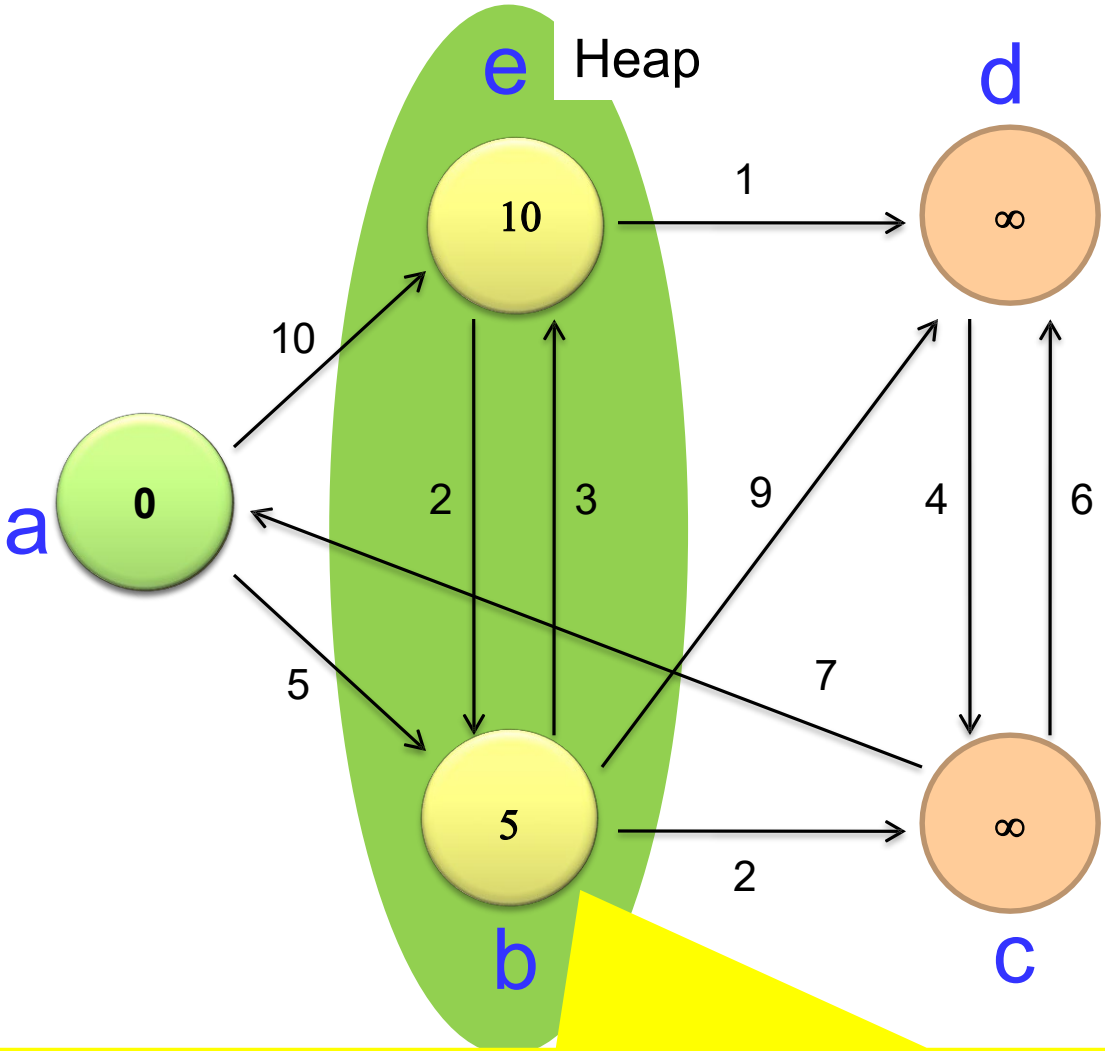
find shortest path from a to other nodes



Then, for each outlink neighbor of the deleted node, if not visited yet, it is inserted in the heap - update distance

Dijkstra's Algorithm Example

find shortest path from a to other nodes

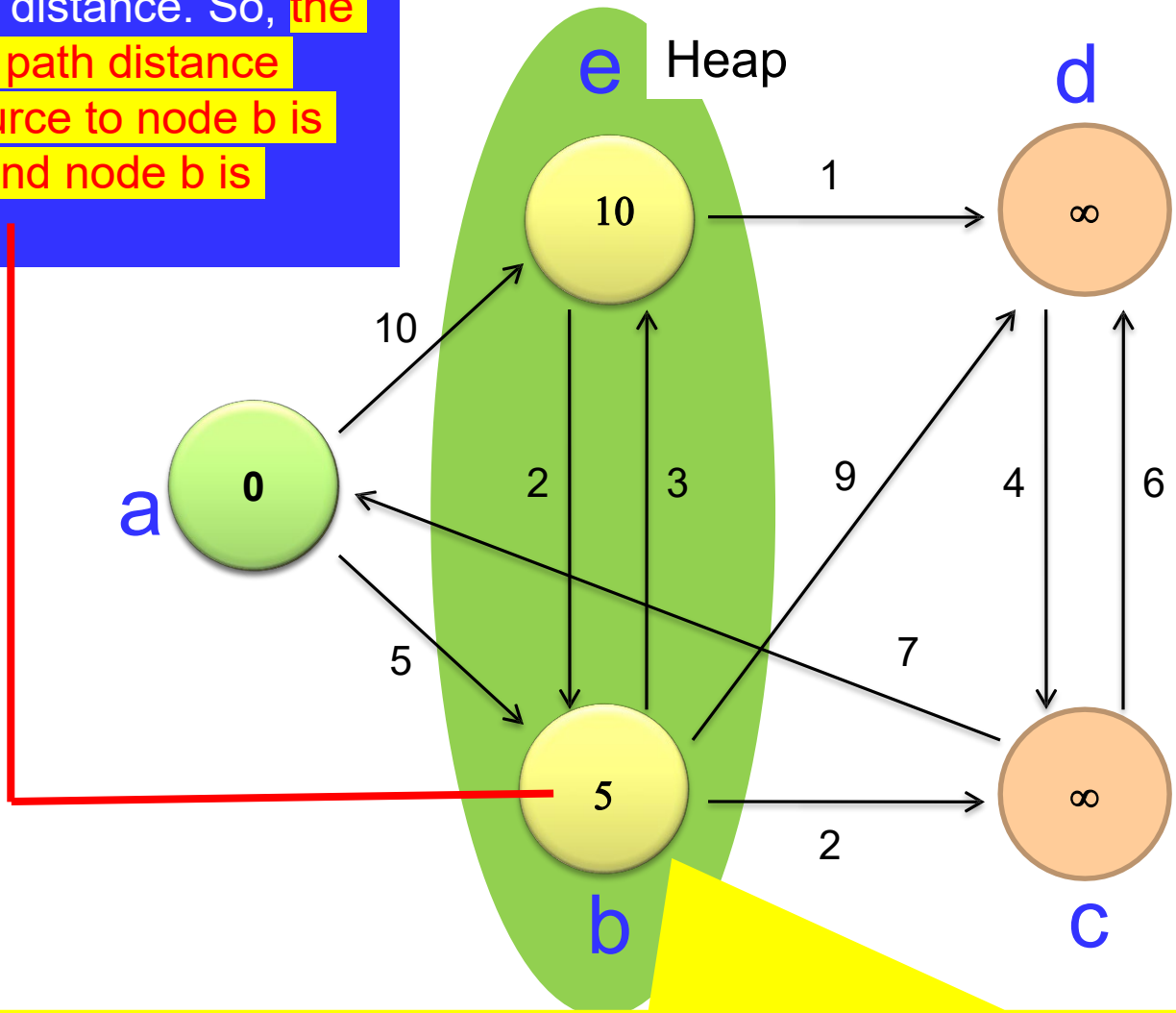


Then, the node with the smallest distance from source is removed from heap

Dijkstra's Algorithm Example

In Heap, node b has the smallest distance. So, the shortest path distance from source to node b is known and node b is deleted.

ortest path from a to other nodes

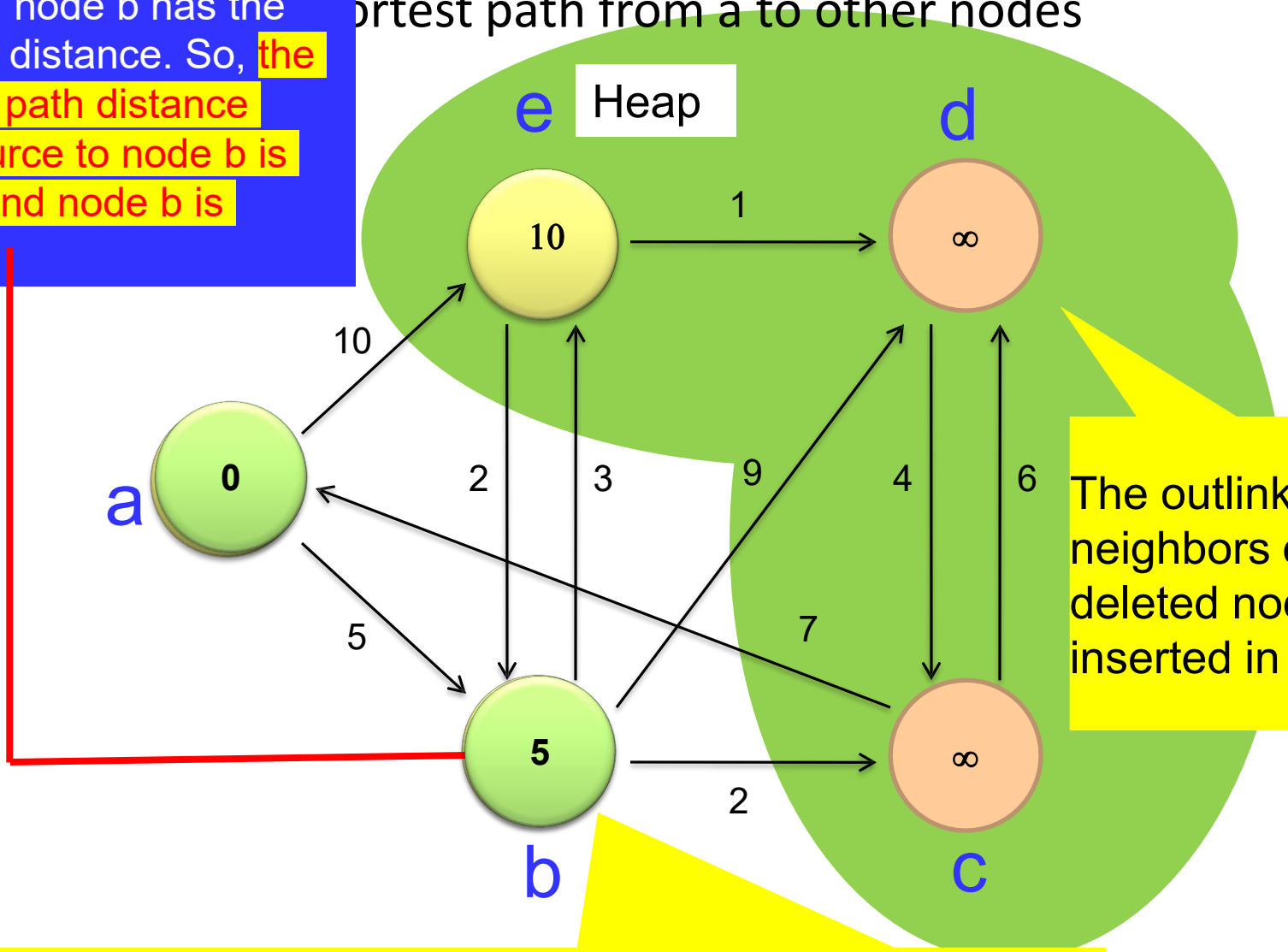


Then, the node with the smallest distance from source is removed from heap

Dijkstra's Algorithm Example

ortest path from a to other nodes

In Heap, node b has the smallest distance. So, the shortest path distance from source to node b is known and node b is deleted.

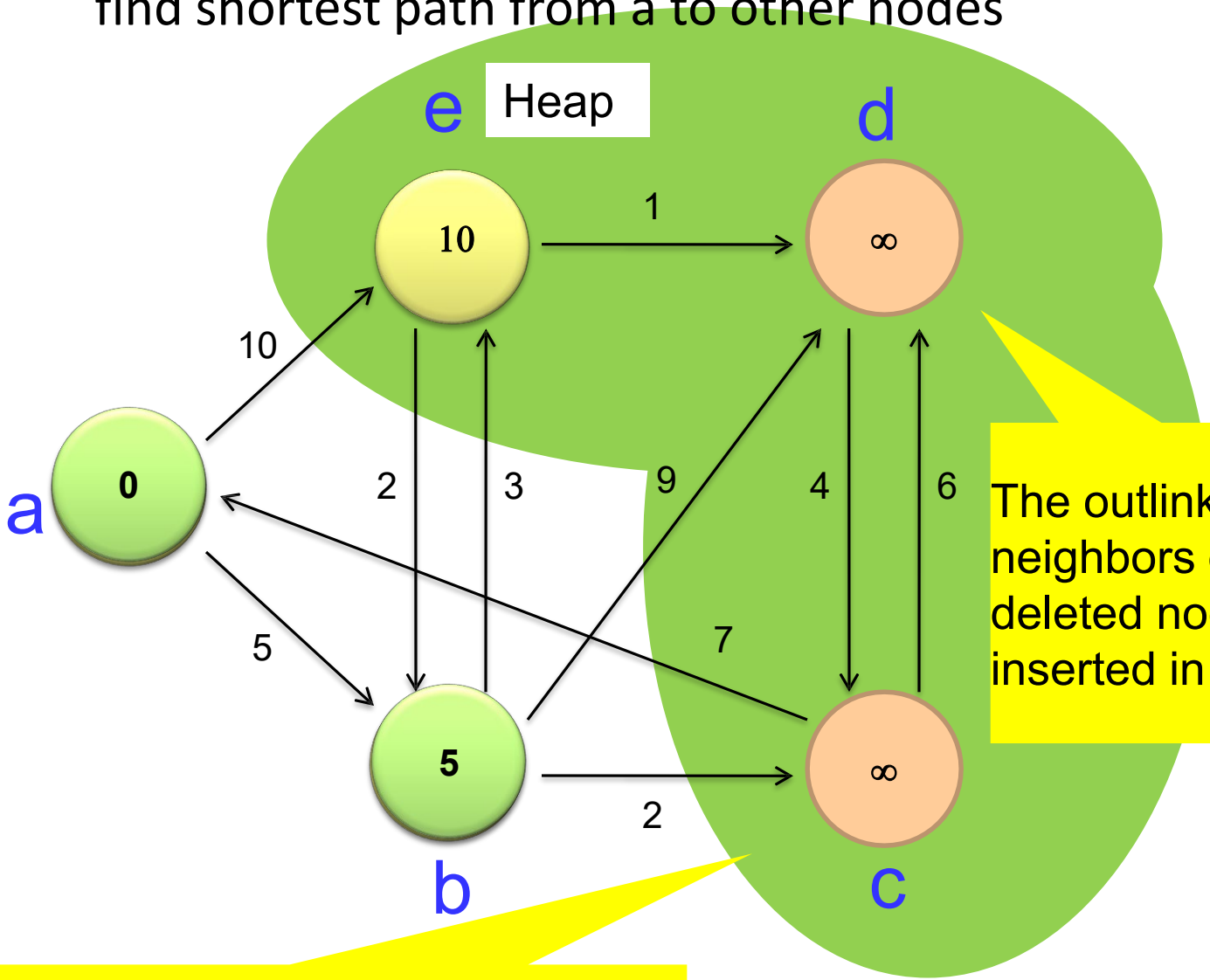


The outlink neighbors of the deleted node b are inserted in Heap

Then, the node with the smallest distance from source is removed from heap

Dijkstra's Algorithm Example

find shortest path from a to other nodes

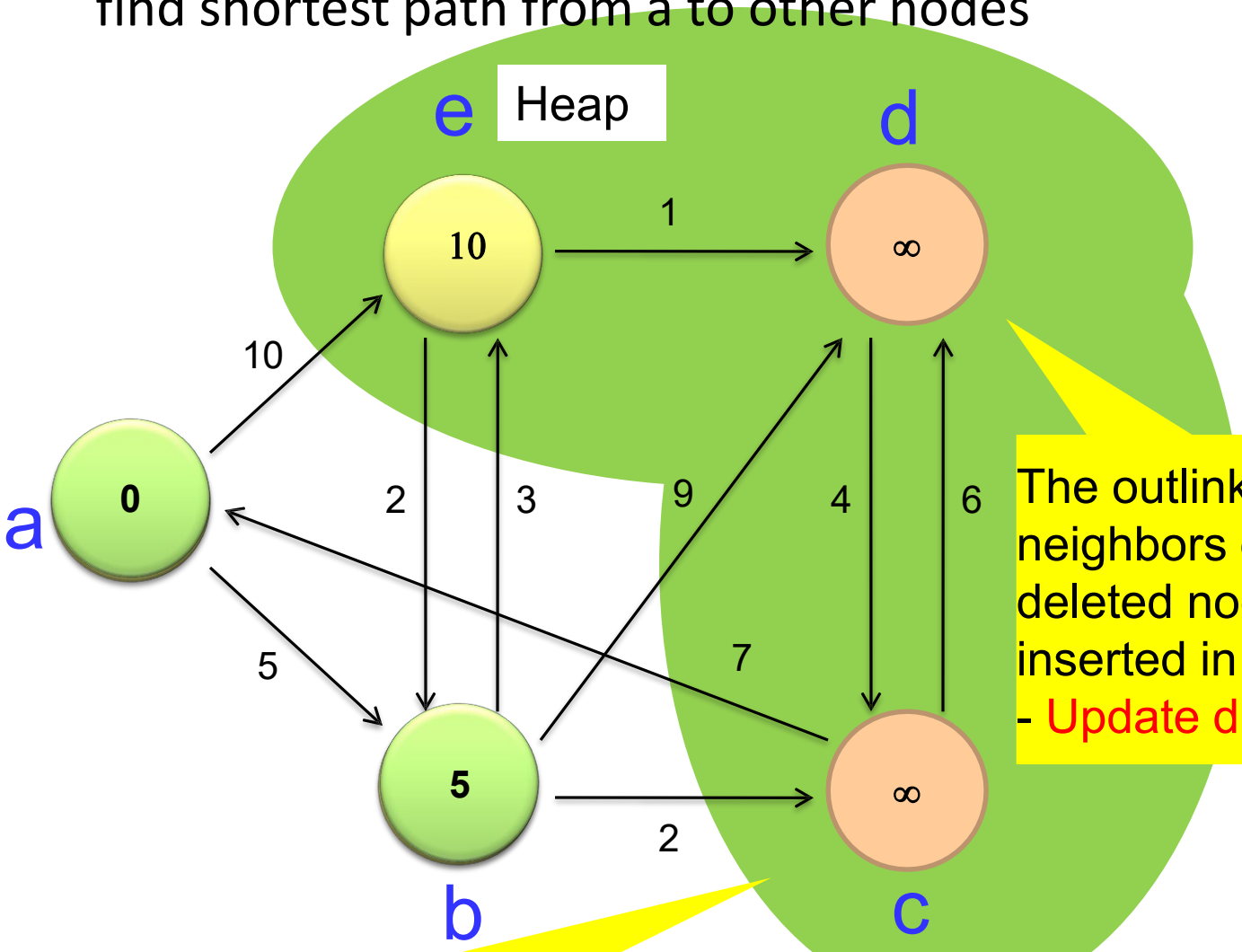


The outlink neighbors of the deleted node b are inserted in Heap

The heap includes node e, c, d now.

Dijkstra's Algorithm Example

find shortest path from a to other nodes

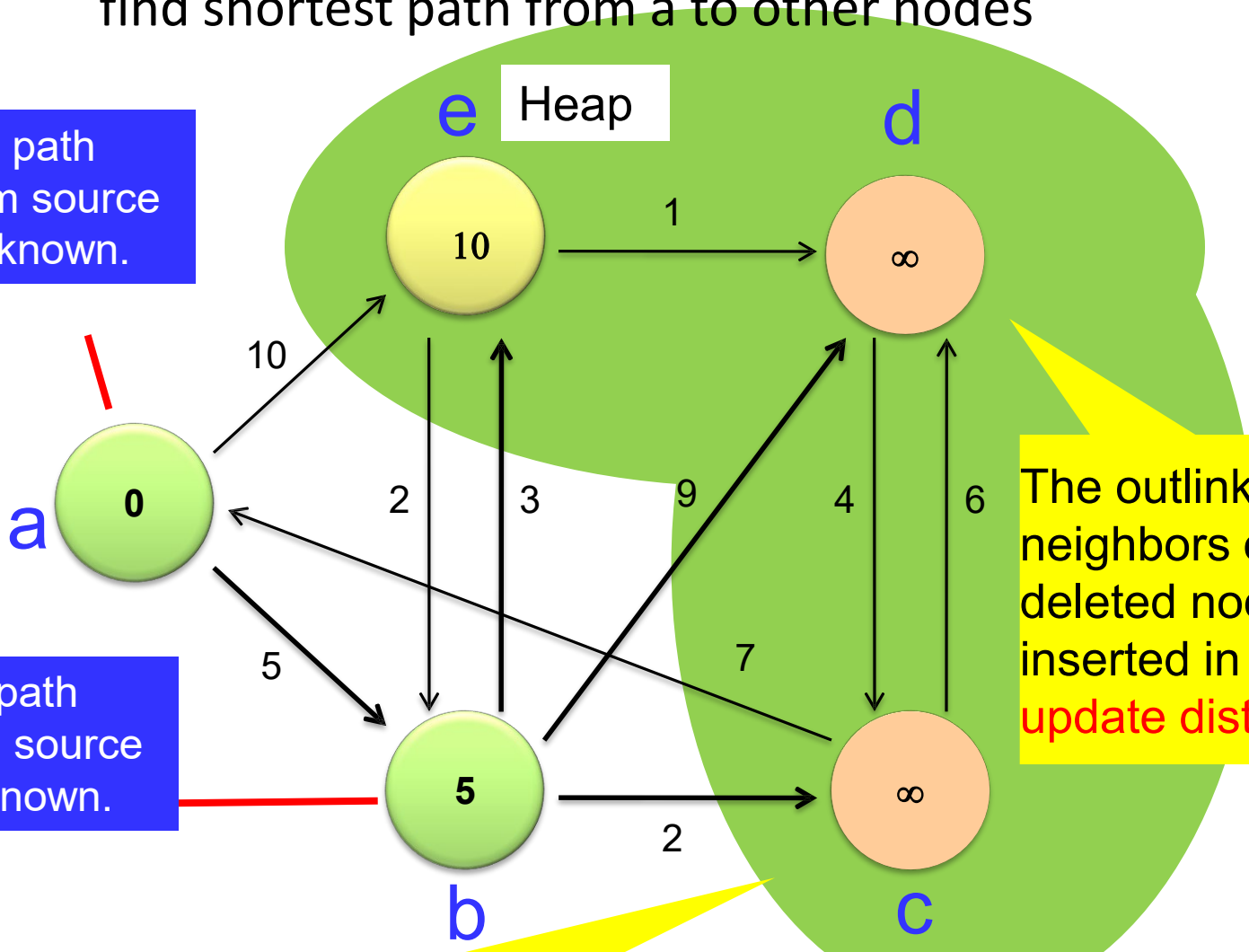


The outlink neighbors of the deleted node b are inserted in Heap - Update distance

The heap includes node e, c, d now.

Dijkstra's Algorithm Example

find shortest path from a to other nodes



The shortest path distance from source to node a is known.

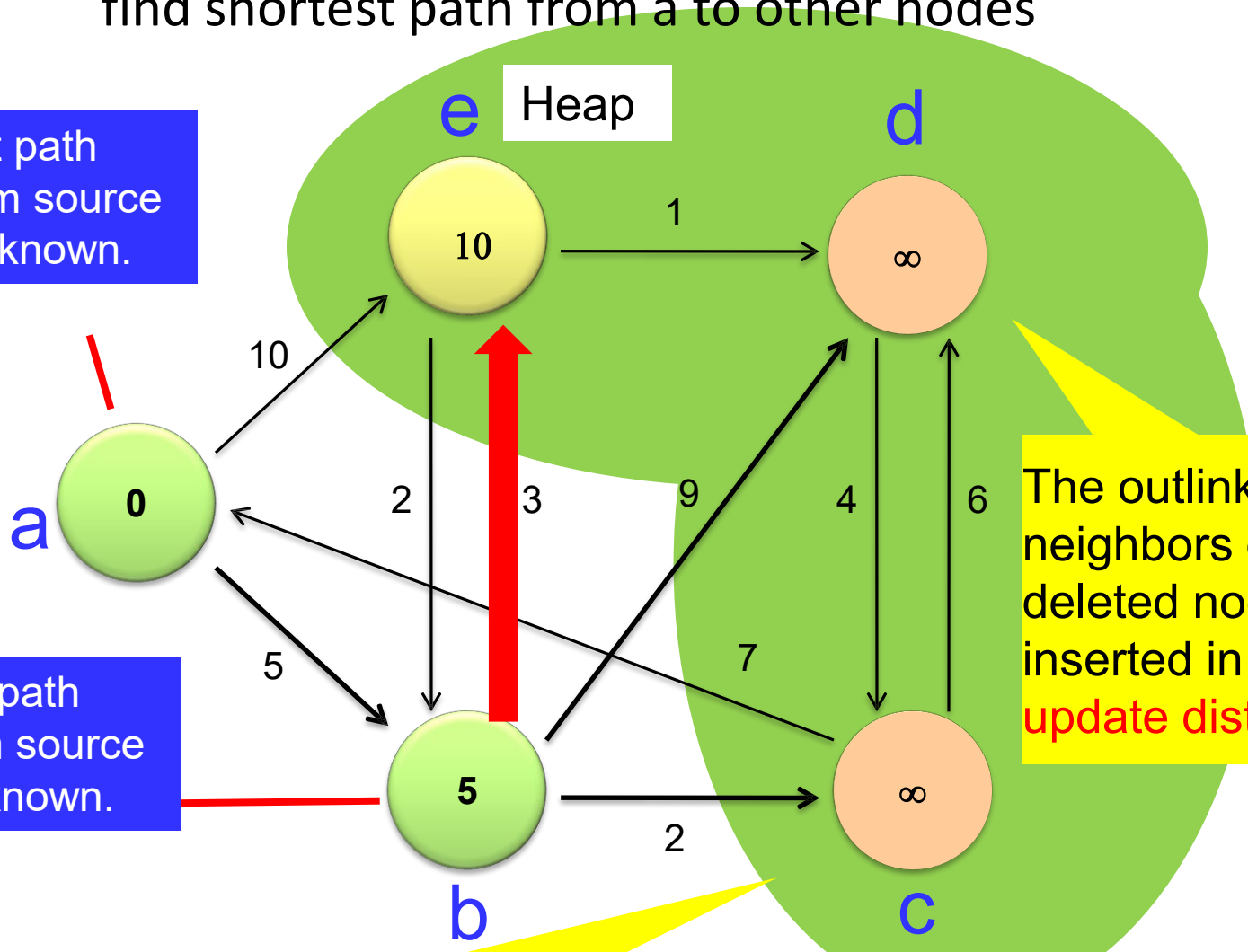
The shortest path distance from source to node b is known.

The outlink neighbors of the deleted node b are inserted in Heap – update distance

The heap includes node e, c, d now.

Dijkstra's Algorithm Example

find shortest path from a to other nodes



The shortest path distance from source to node a is known.

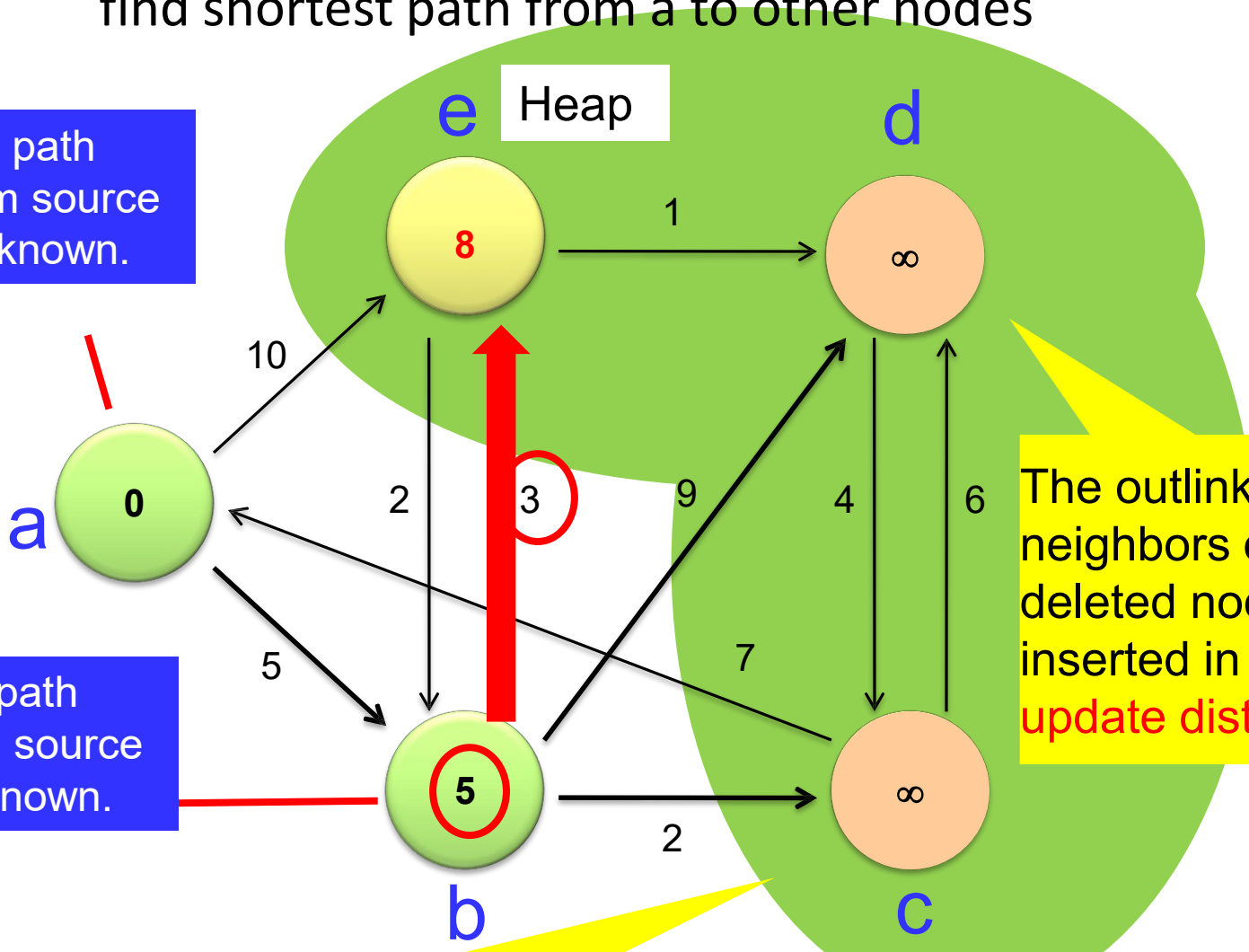
The shortest path distance from source to node b is known.

The outlink neighbors of the deleted node b are inserted in Heap – update distance

The heap includes node e, c, d now.

Dijkstra's Algorithm Example

find shortest path from a to other nodes



The shortest path distance from source to node a is known.

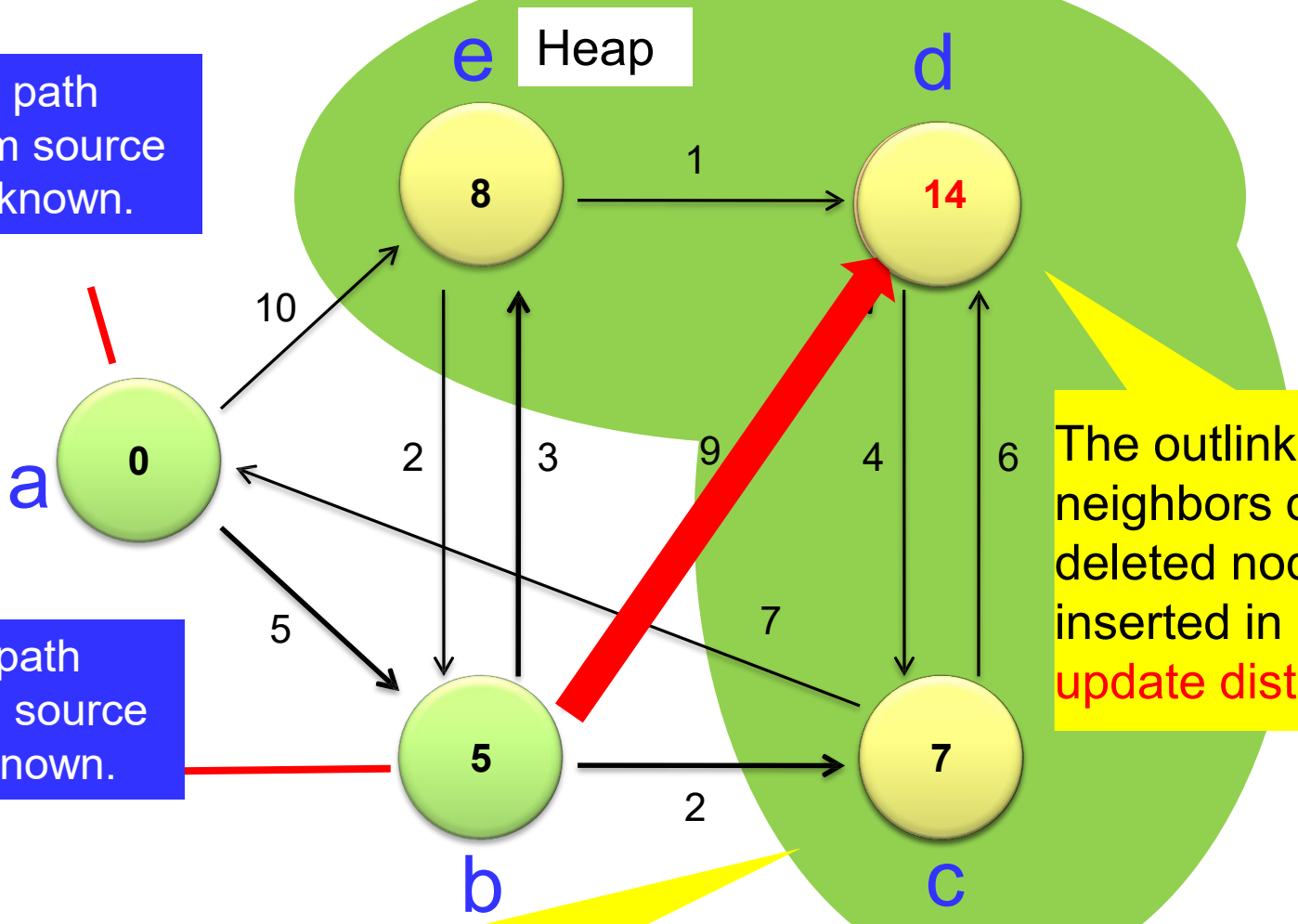
The shortest path distance from source to node b is known.

The outlink neighbors of the deleted node b are inserted in Heap – update distance

The heap includes node e, c, d now.

Dijkstra's Algorithm Example

find shortest path from a to other nodes



The shortest path distance from source to node a is known.

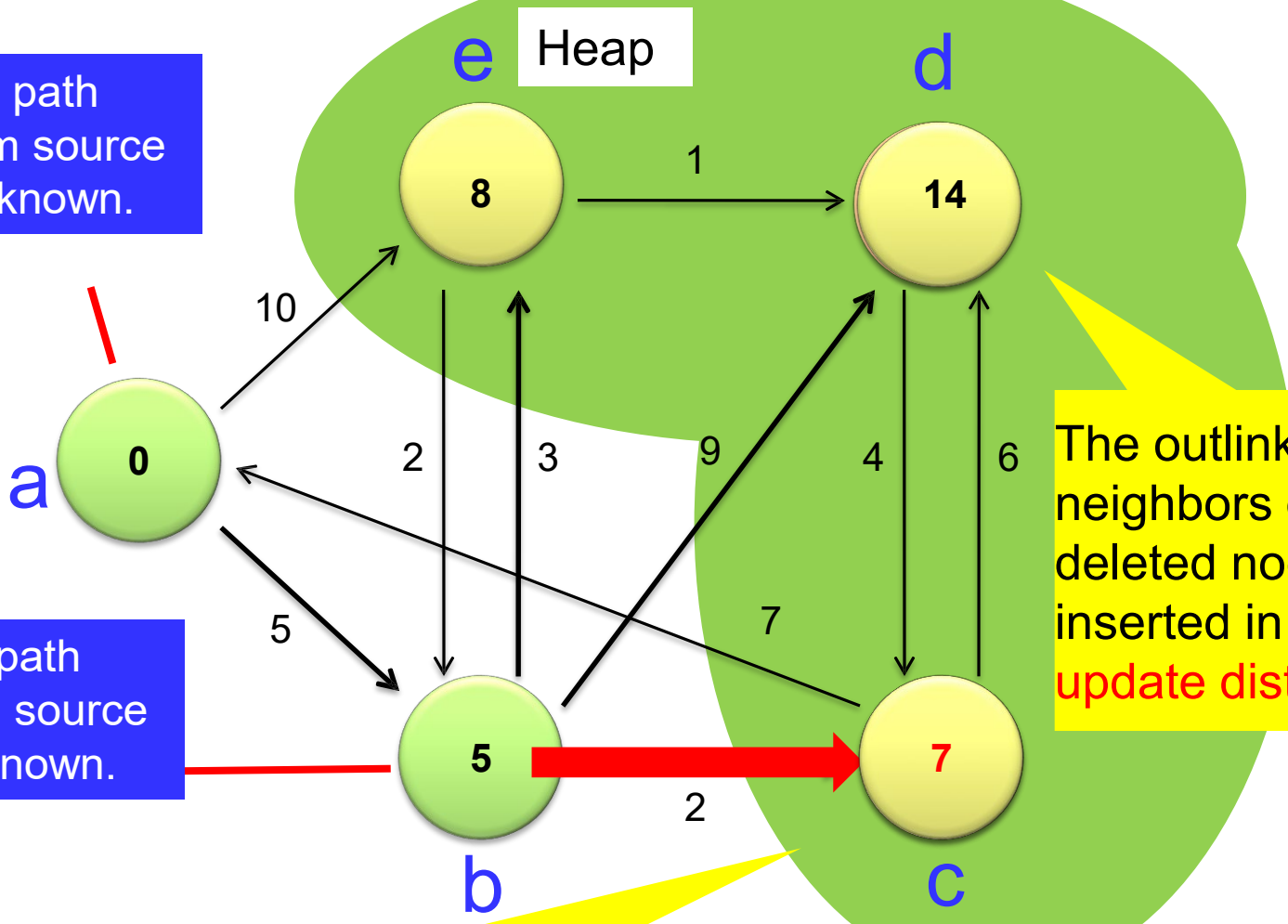
The shortest path distance from source to node b is known.

The outlink neighbors of the deleted node b are inserted in Heap – update distance

The heap includes node e, c, d now.

Dijkstra's Algorithm Example

find shortest path from a to other nodes



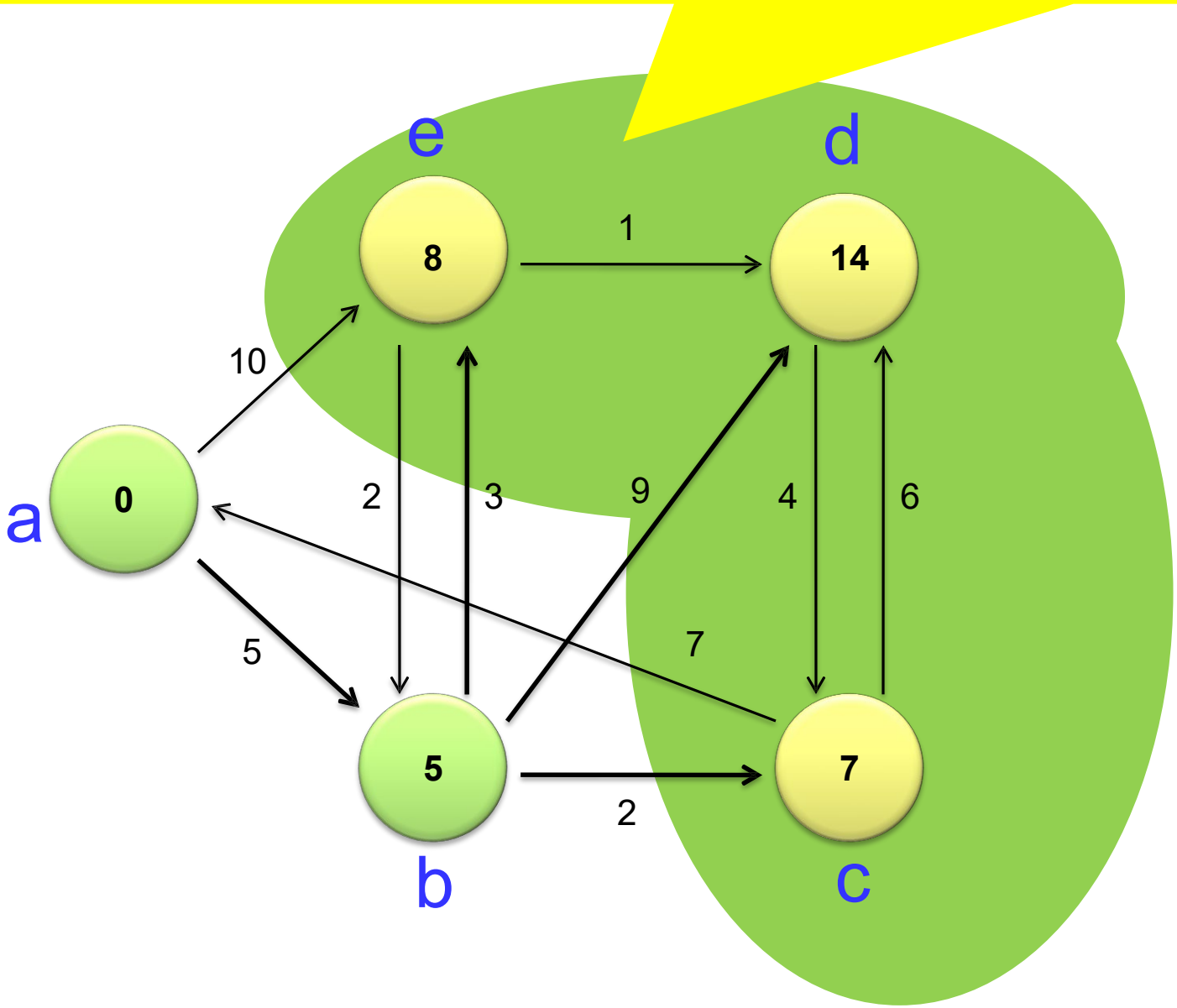
The shortest path distance from source to node a is known.

The shortest path distance from source to node b is known.

The outlink neighbors of the deleted node b are inserted in Heap – update distance

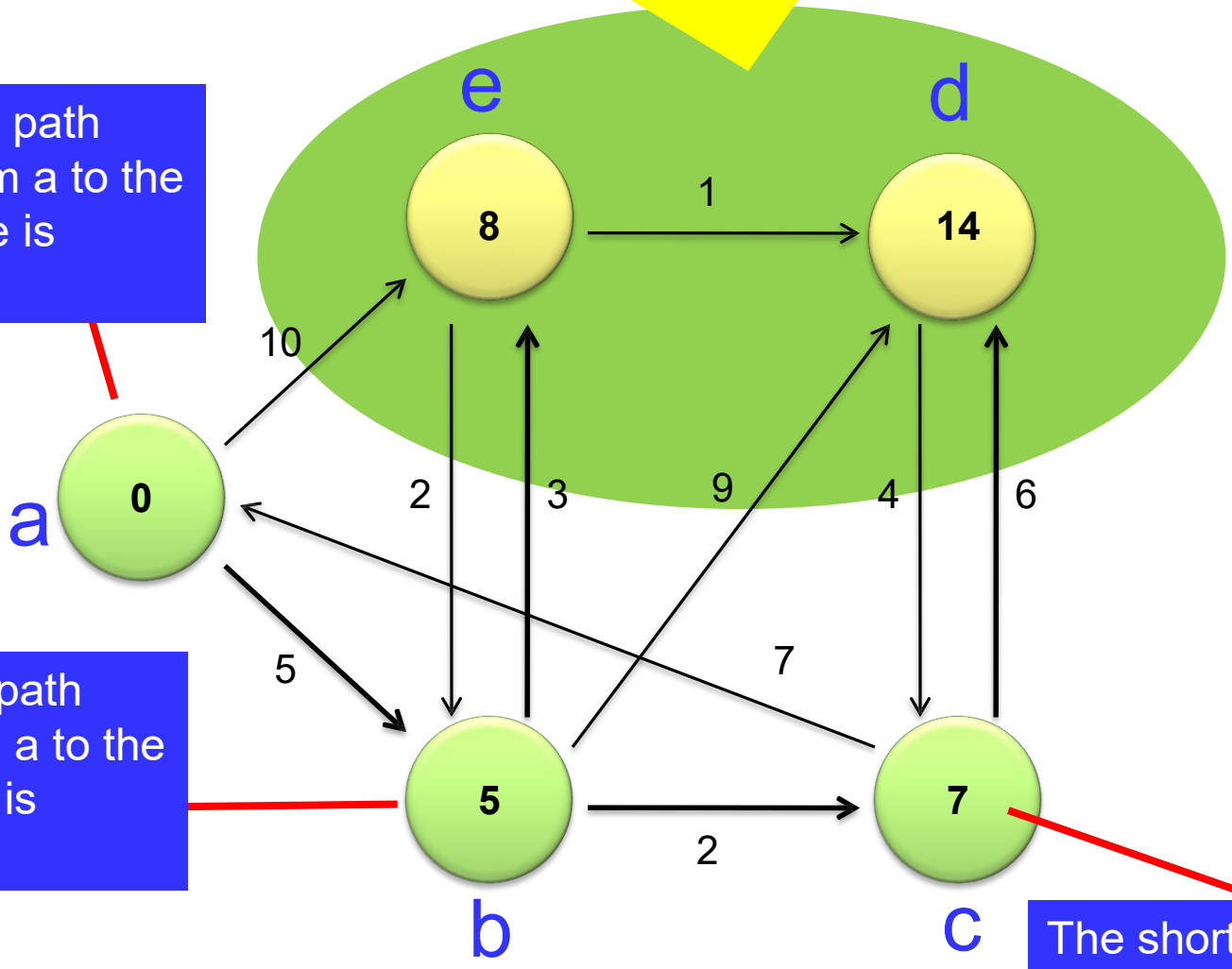
The heap includes node e, c, d now.

- The node with the smallest distance value is removed from heap
- not visited outlink neighbors are inserted in the heap



Example from CLR

The heap includes e, d two nodes now.



The shortest path distance from a to the deleted node is known.

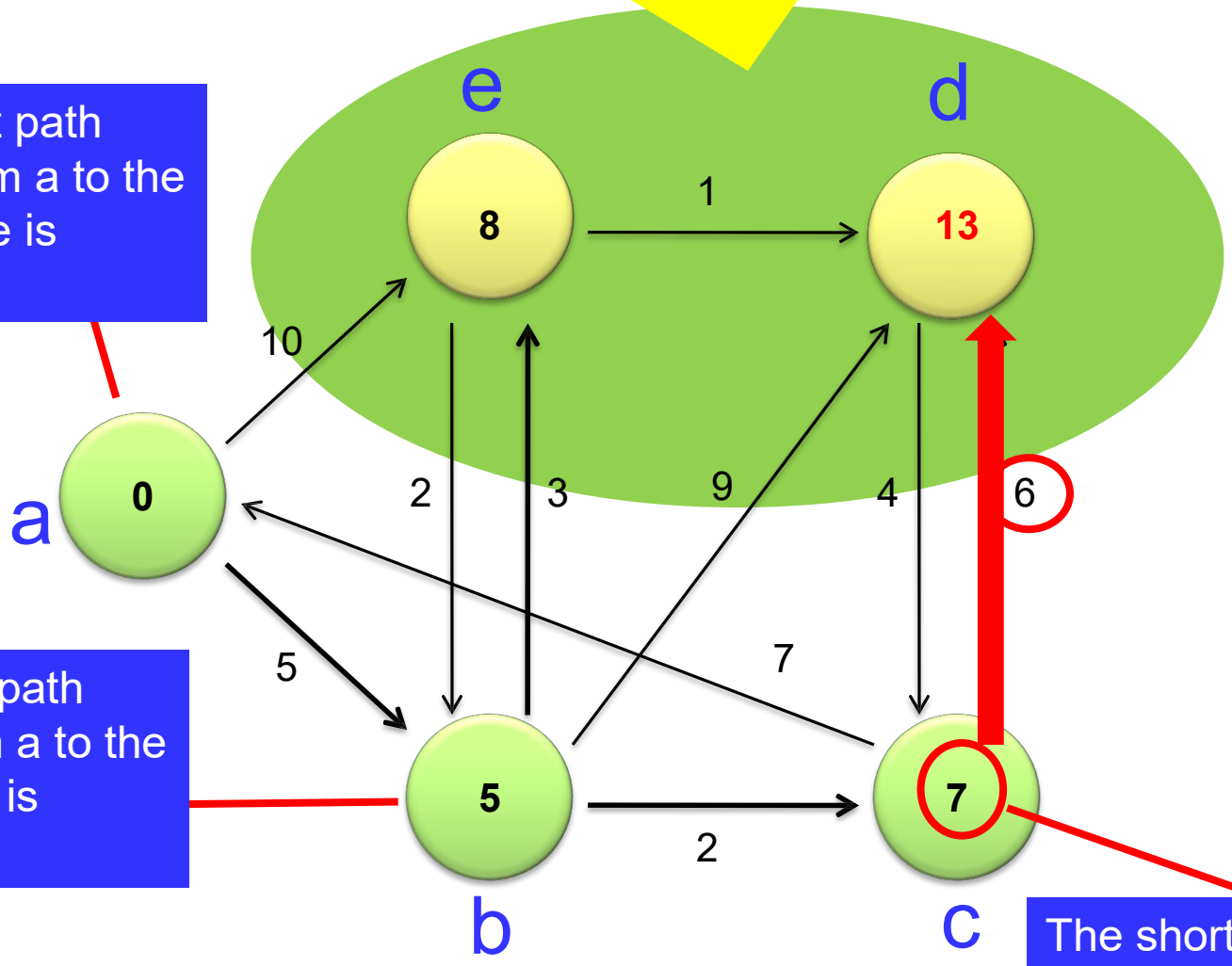
The shortest path distance from a to the deleted node is known.

The shortest path distance from a to the deleted node is known.

Example from CLR

The heap includes e, d two nodes now.

The shortest path distance from a to the deleted node is known.



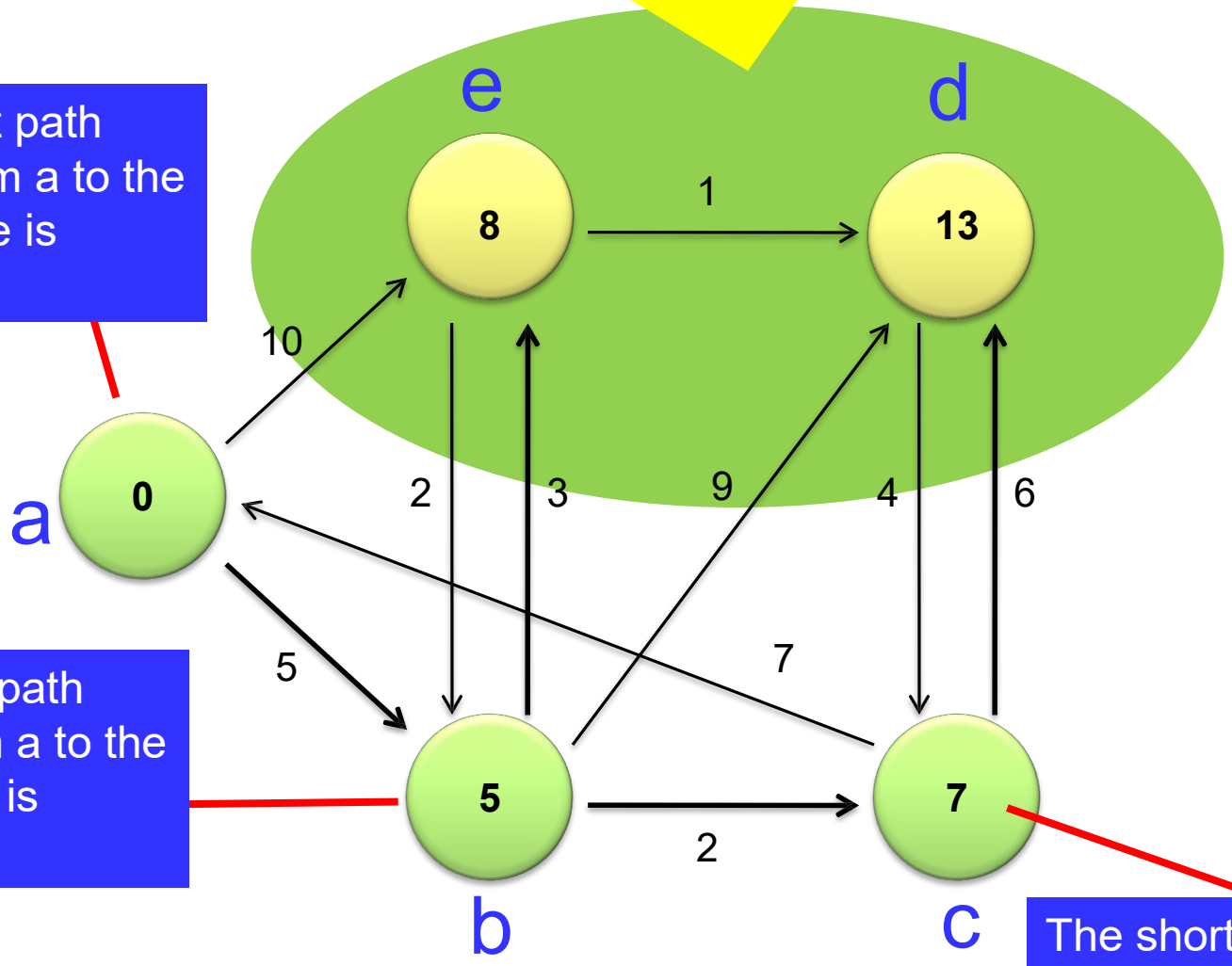
The shortest path distance from a to the deleted node is known.

The shortest path distance from a to the deleted node is known.

Example from CLR

The heap includes e, d two nodes now.

The shortest path distance from a to the deleted node is known.



The shortest path distance from a to the deleted node is known.

The shortest path distance from a to the deleted node is known.

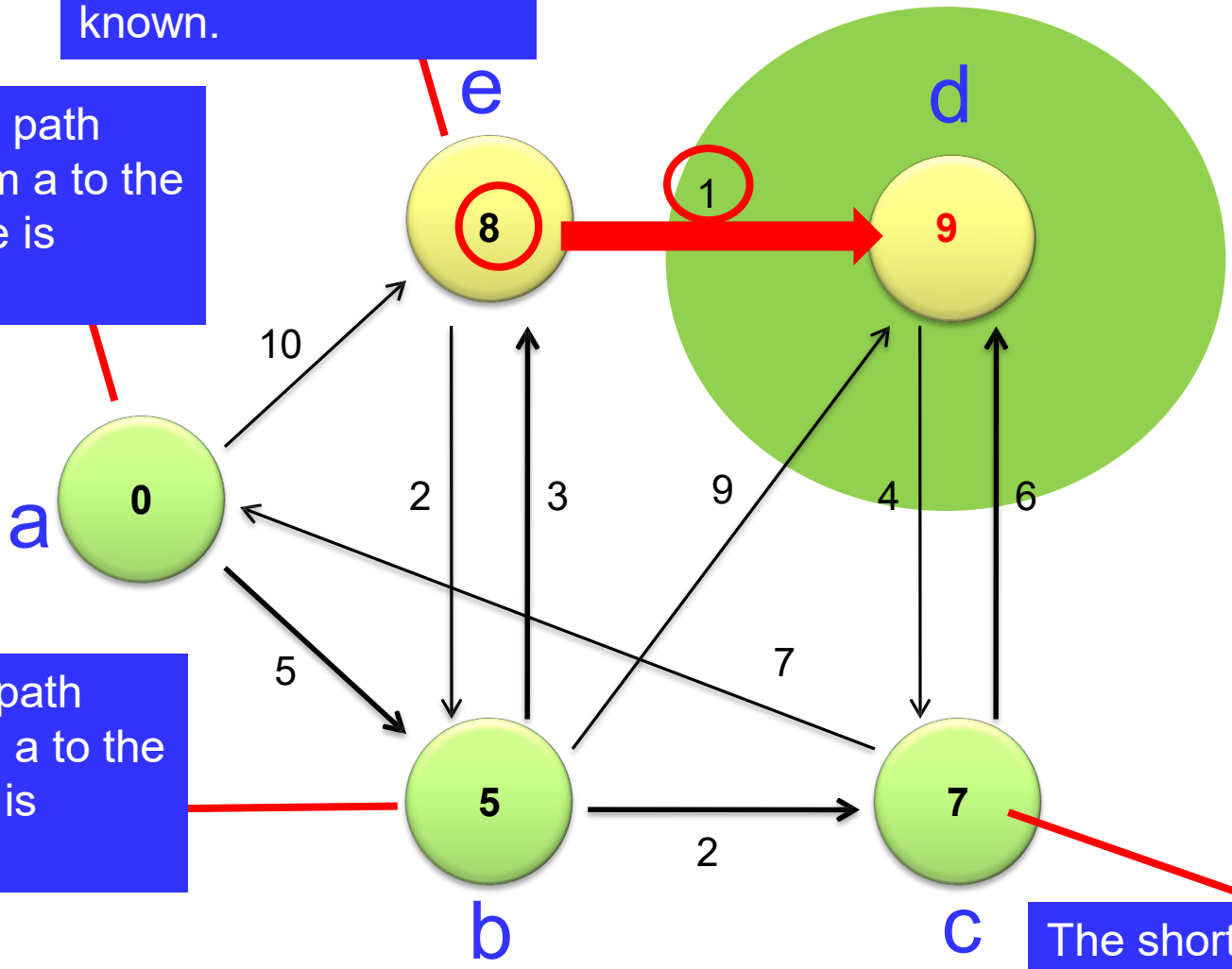
Example from CLR

The shortest path distance from a to the deleted node is known.

The shortest path distance from a to the deleted node is known.

The shortest path distance from a to the deleted node is known.

The shortest path distance from a to the deleted node is known.



Example from CLR

RMIT Classification: Trusted

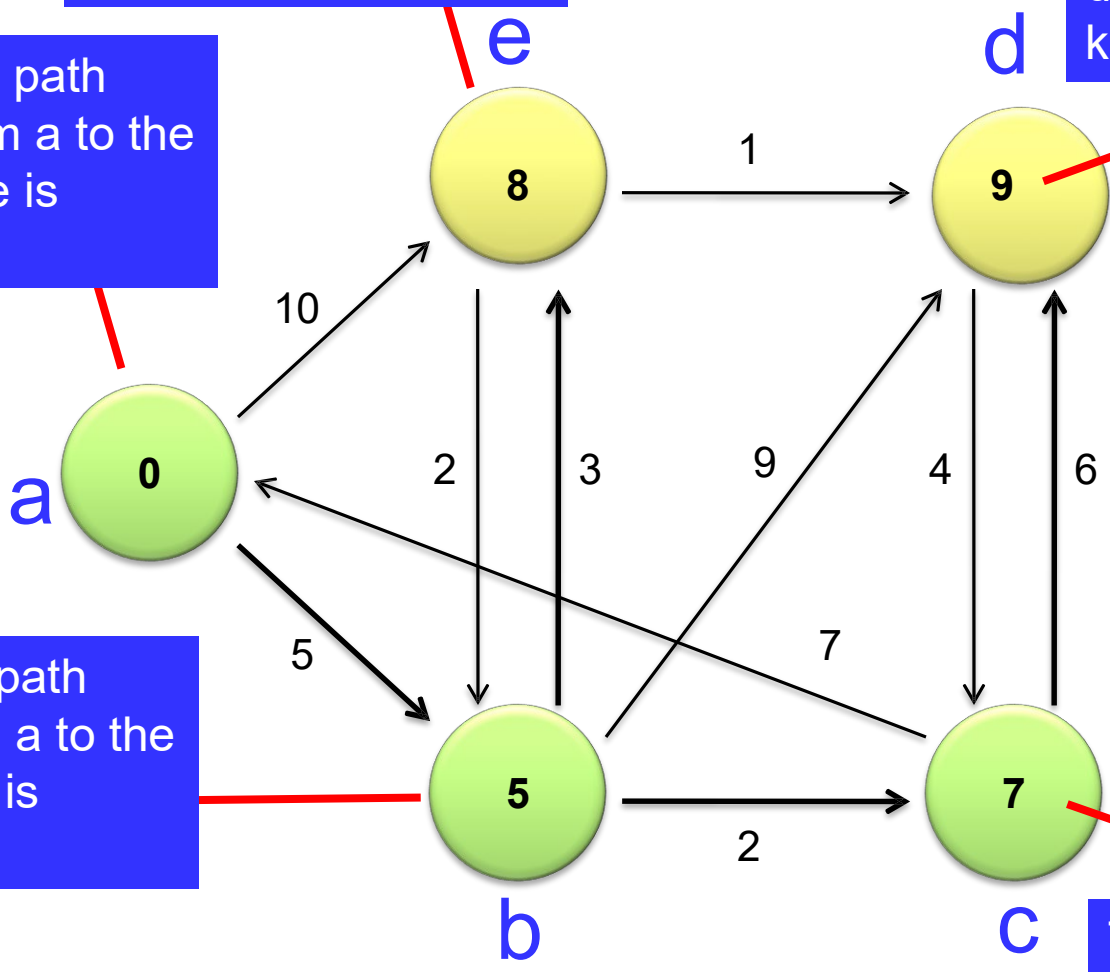
The shortest path distance from a to the deleted node is known.

The shortest path distance from a to the deleted node is known.

The shortest path distance from a to the deleted node is known.

The shortest path distance from a to the deleted node is known.

The shortest path distance from a to the deleted node is known.



Example from CLR

Can we use Dijkstra's Algorithm in MapReduce Framework?

Can we use Dijkstra's Algorithm in MapReduce Framework?

No

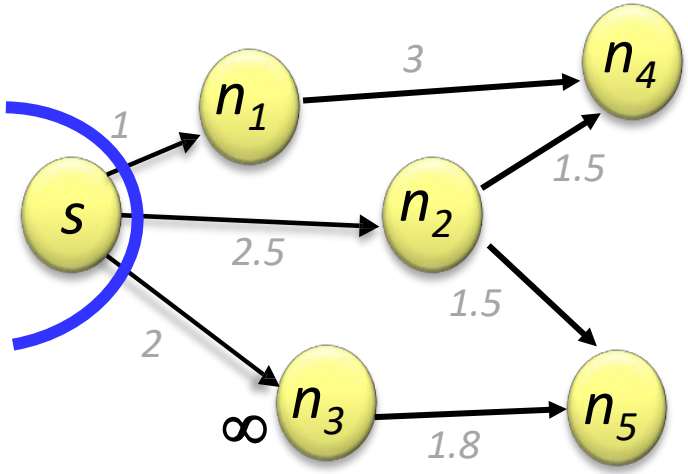
Single-Source Shortest Path

Problem: find shortest path from a source node to one or more target nodes

- Shortest might also mean lowest weight or cost
- Single processor machine: Dijkstra's Algorithm
- MapReduce: Parallel Breadth-First search (BFS)

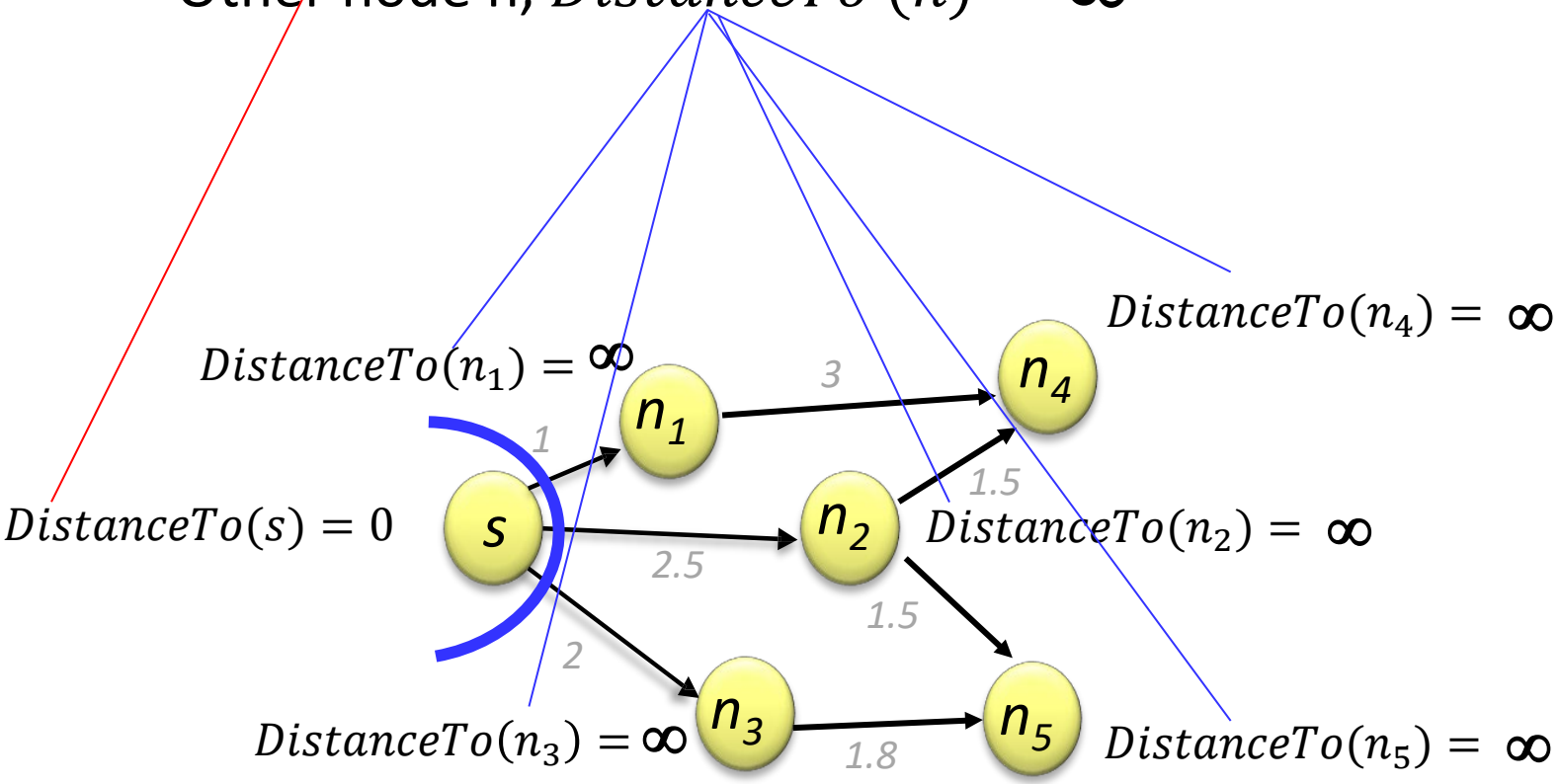
Finding the Shortest Path

- Here's the intuition:
  - source *s*



Finding the Shortest Path

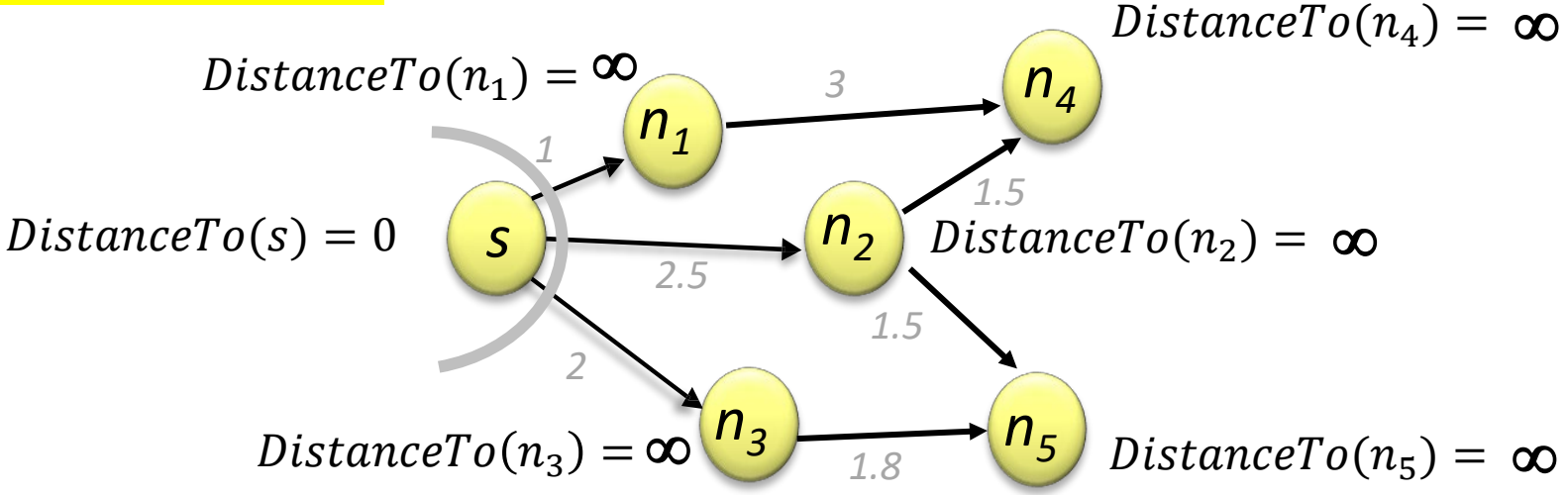
- Here's the intuition:
  - source s, initially
    - $DistanceTo\ (s) = 0$,
    - Other node n, $DistanceTo\ (n) = \infty$



Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M, $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

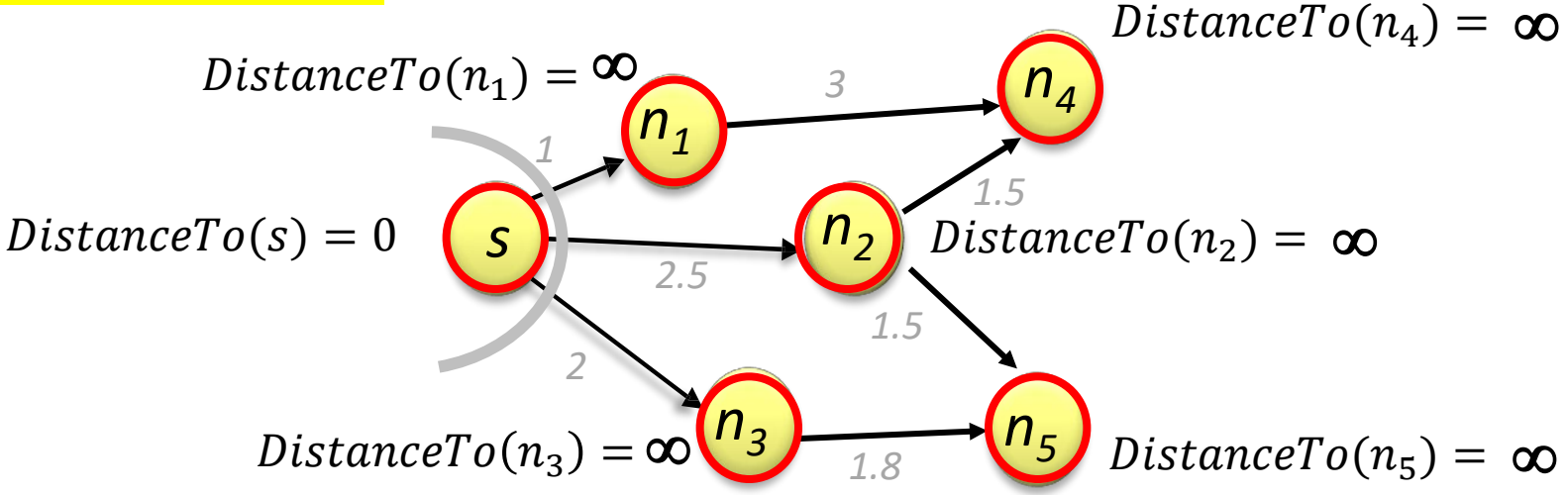
First iteration



Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M, $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

First iteration

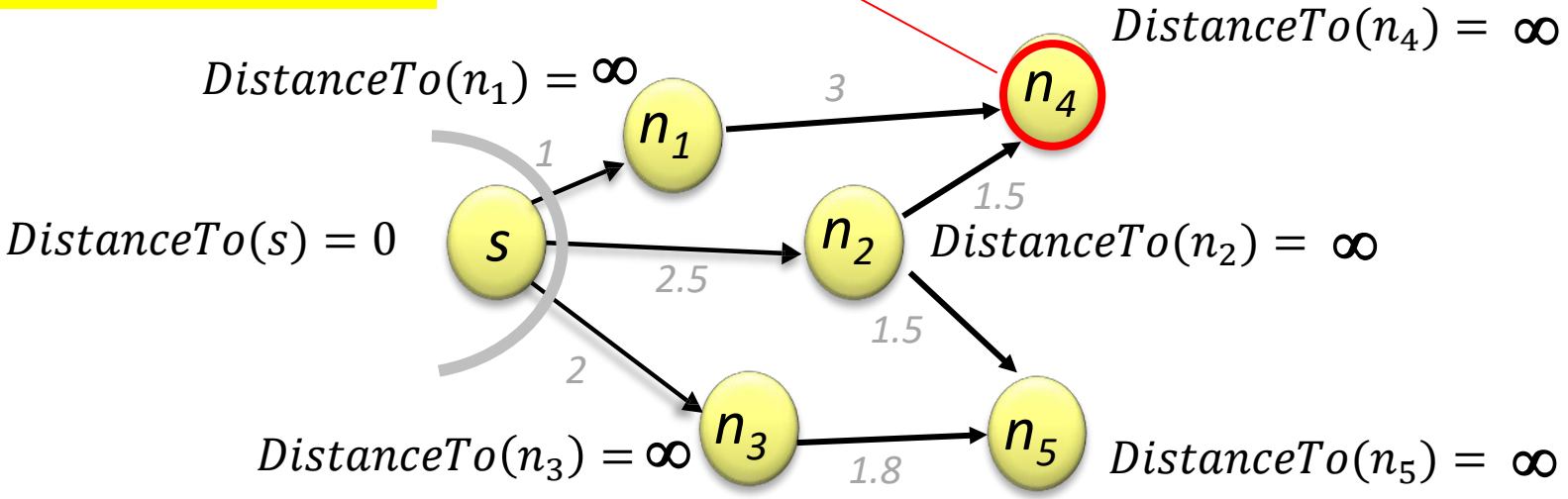


Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M, $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

First iteration

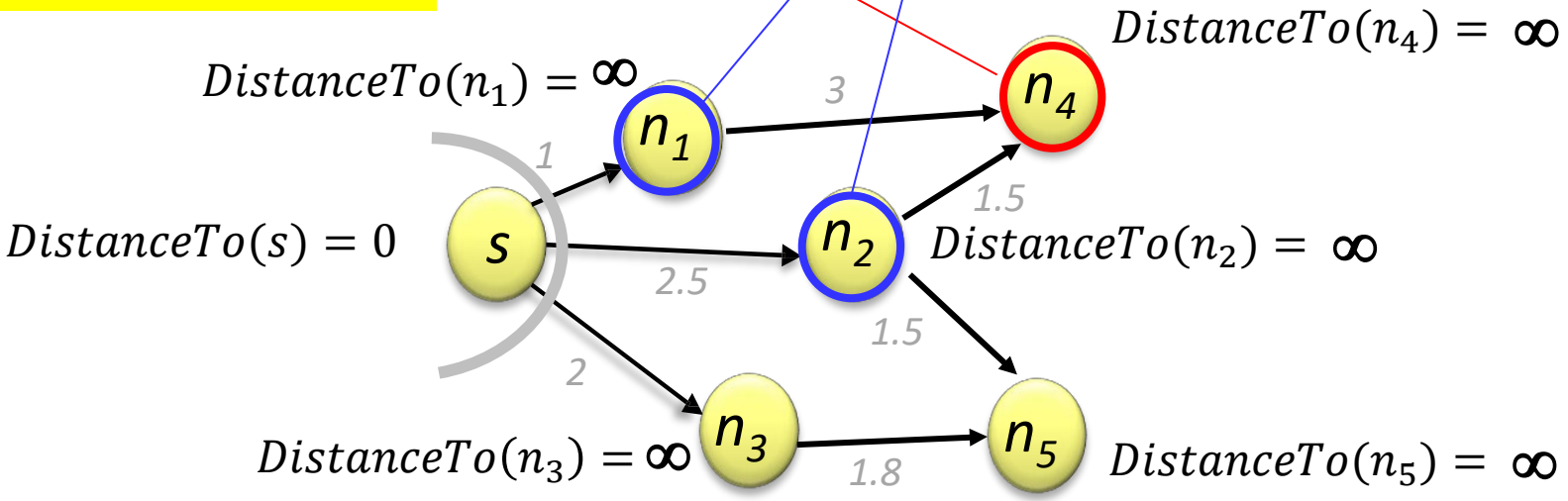
use n4 as example



Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M,
    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

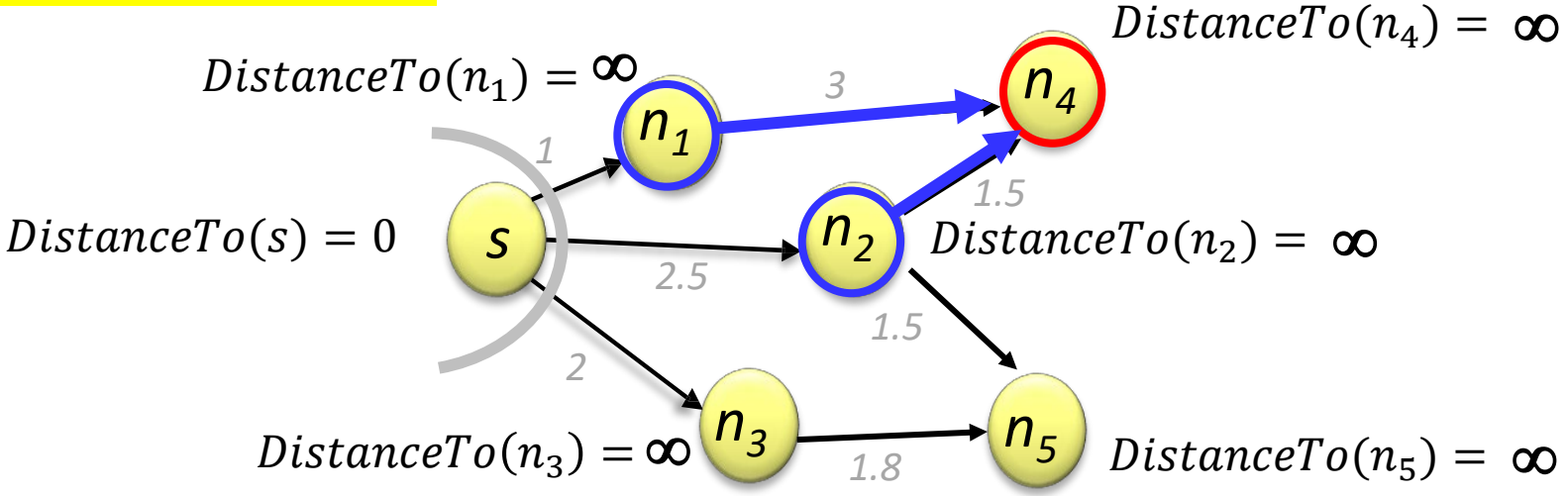
First iteration, n4



Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M,
    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

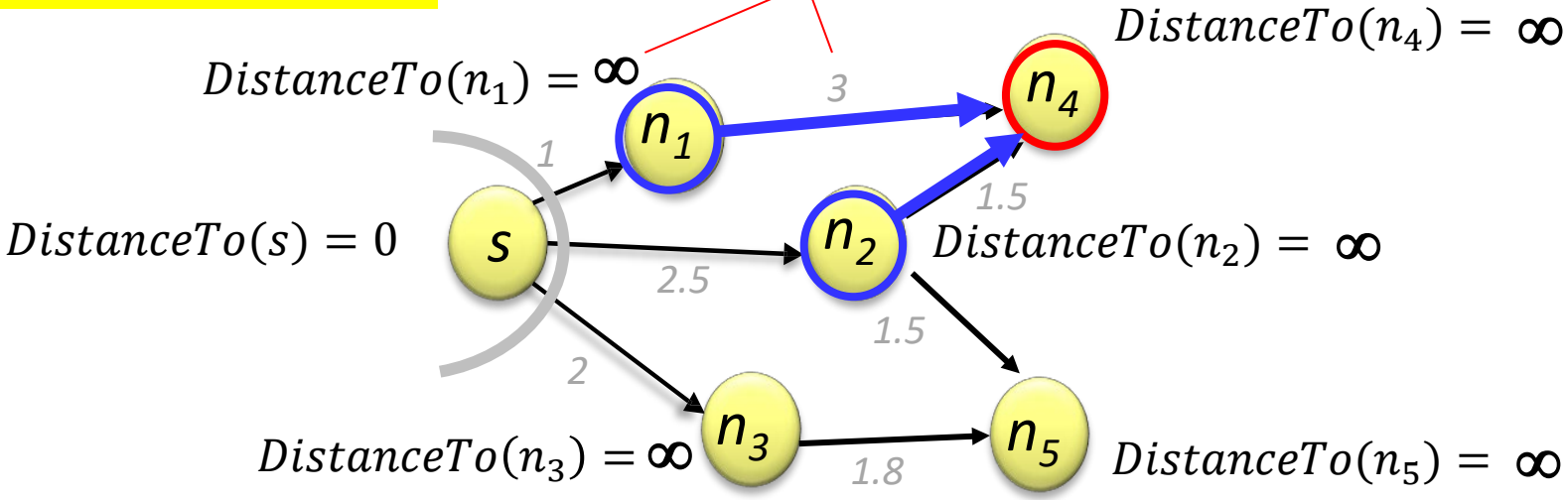
First iteration, n4



Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M,
    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

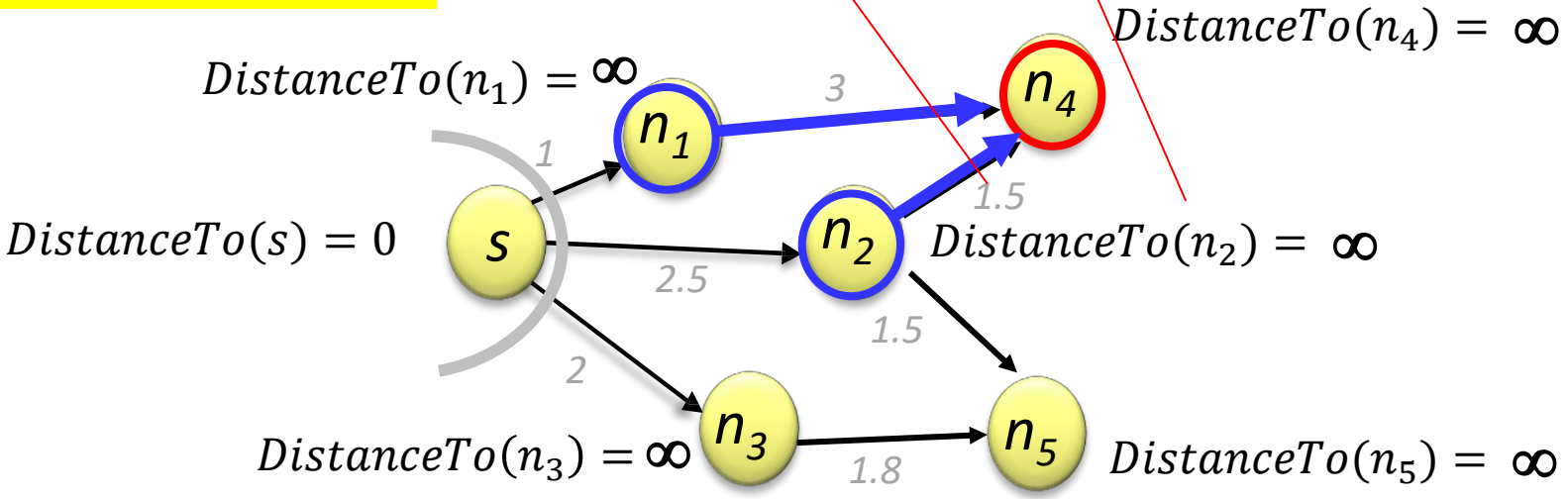
First iteration, n4



Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M,
    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

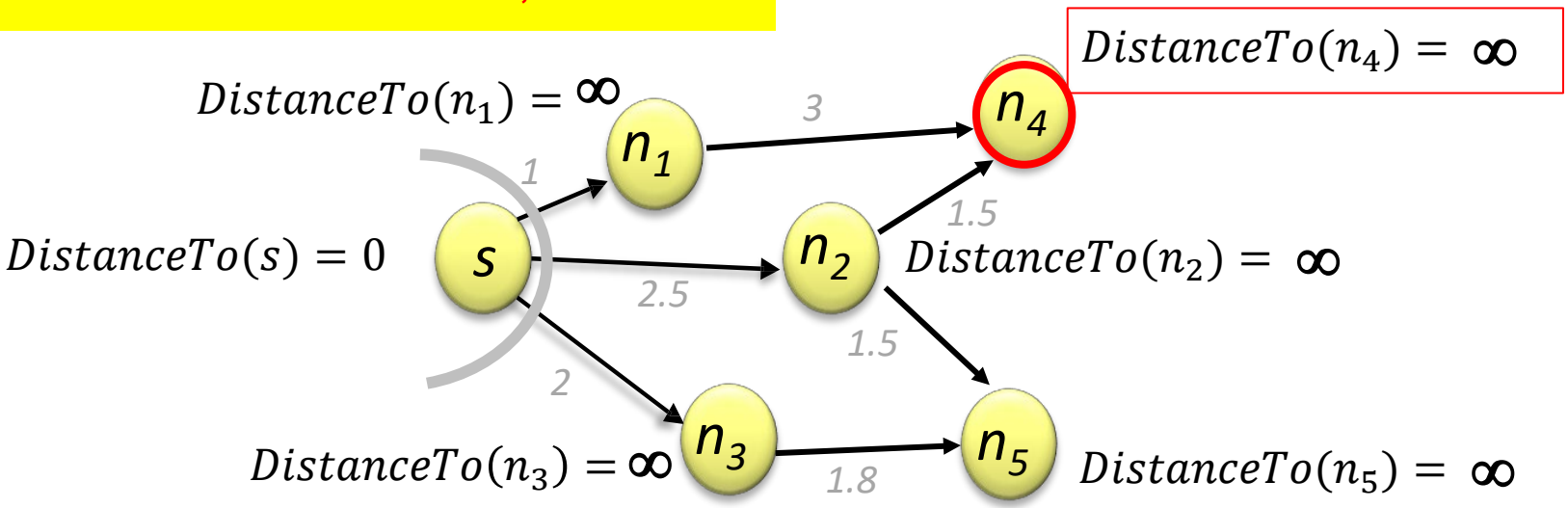
First iteration, n4



Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M,
    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

At the end of first iteration, n4

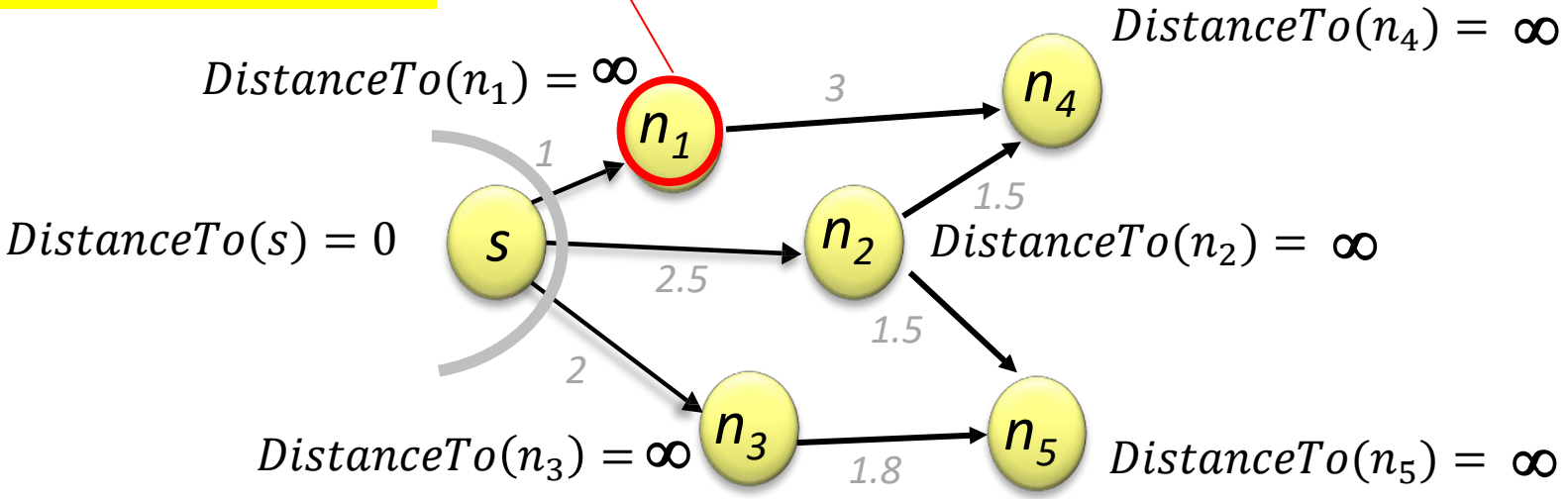


Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M, $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

First iteration

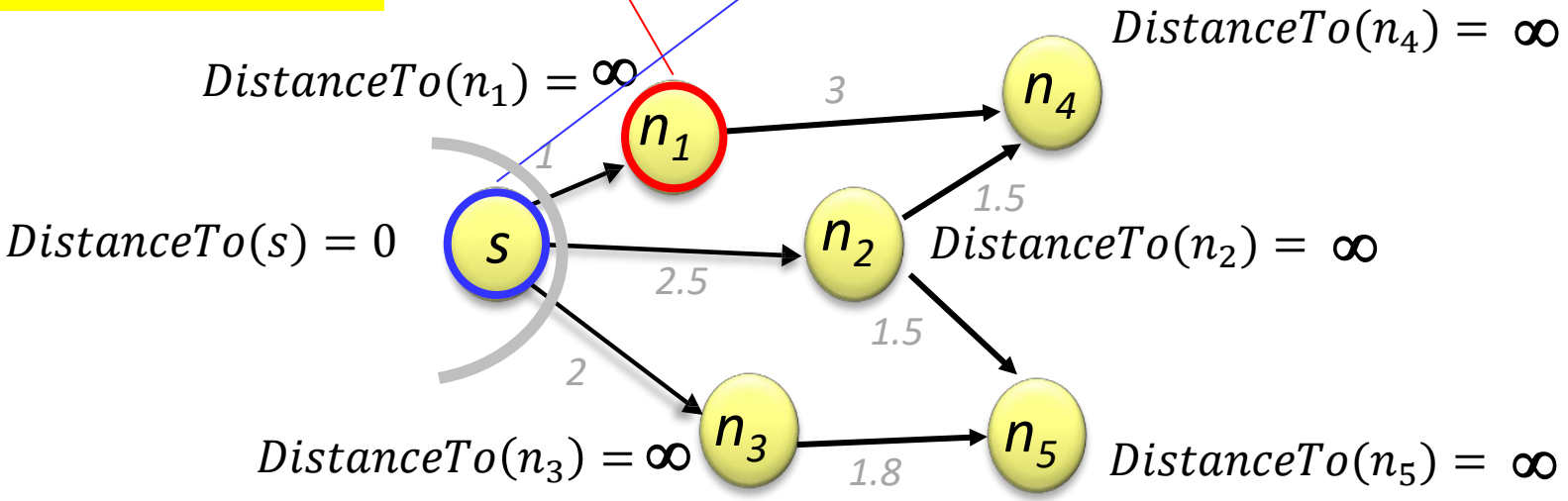
use n1 as example



Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M,
    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

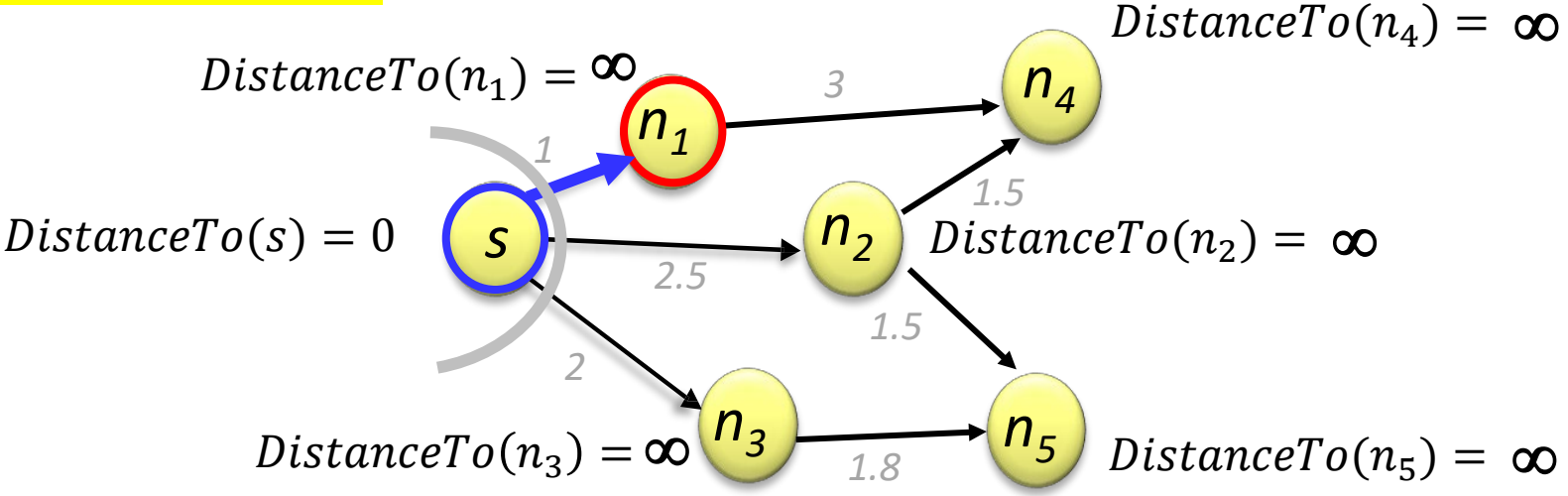
First iteration, n1



Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M,
    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

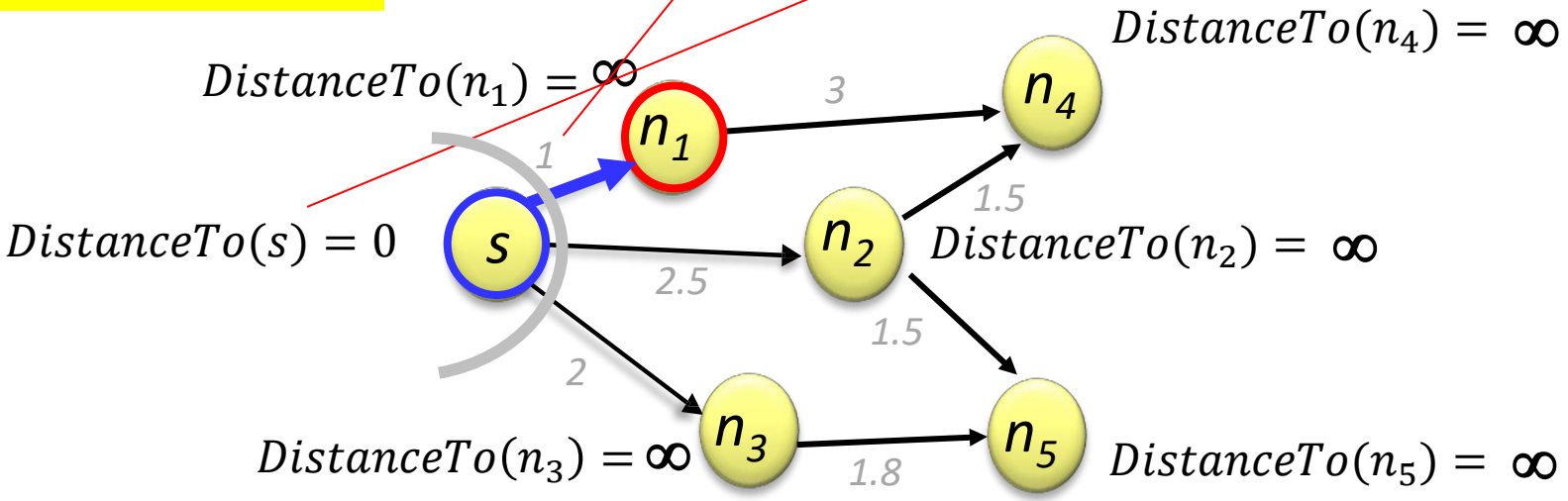
First iteration, n1



Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M,
    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

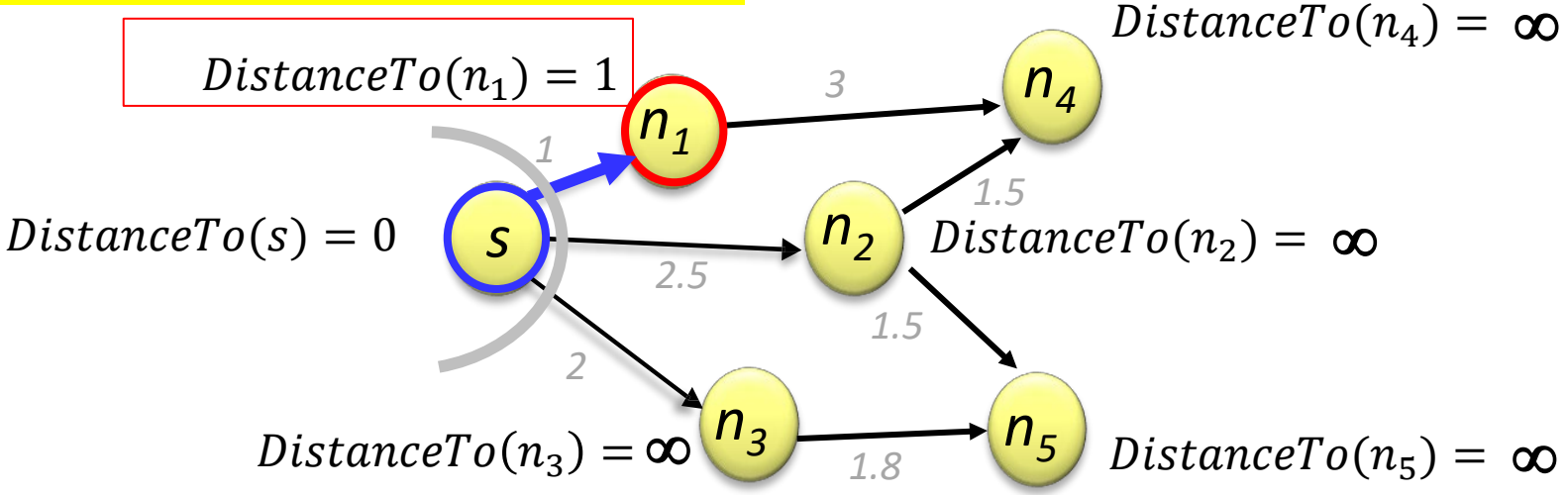
First iteration, n1



Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M,
    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

At the end of first iteration, n1

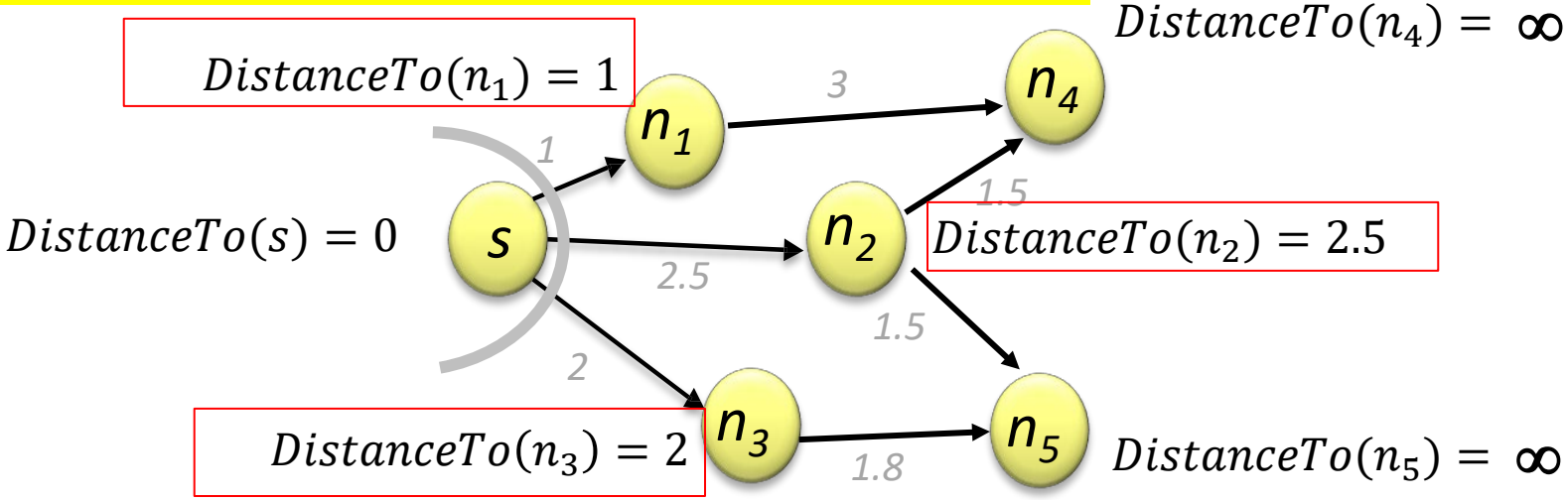


Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

$$DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$$

At the end of first iteration, s,n1,n2,n3,n4,n5

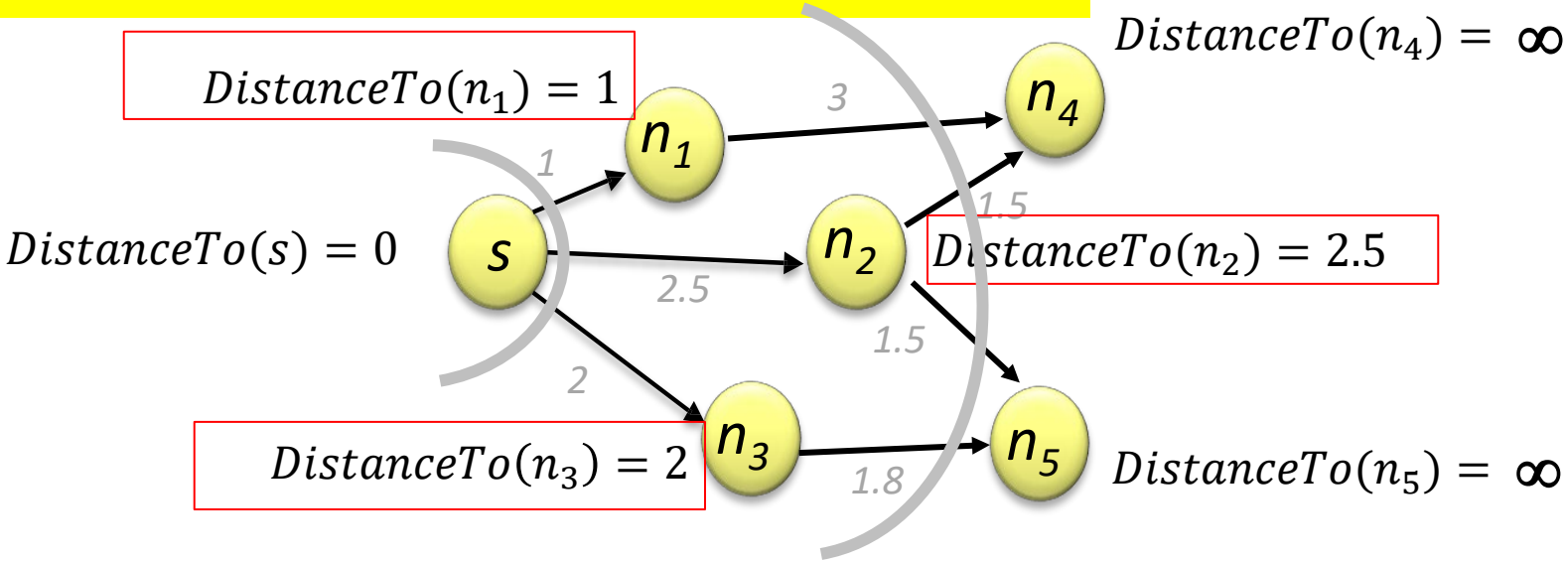


Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

$$DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$$

At the end of first iteration, s,n1,n2,n3,n4,n5

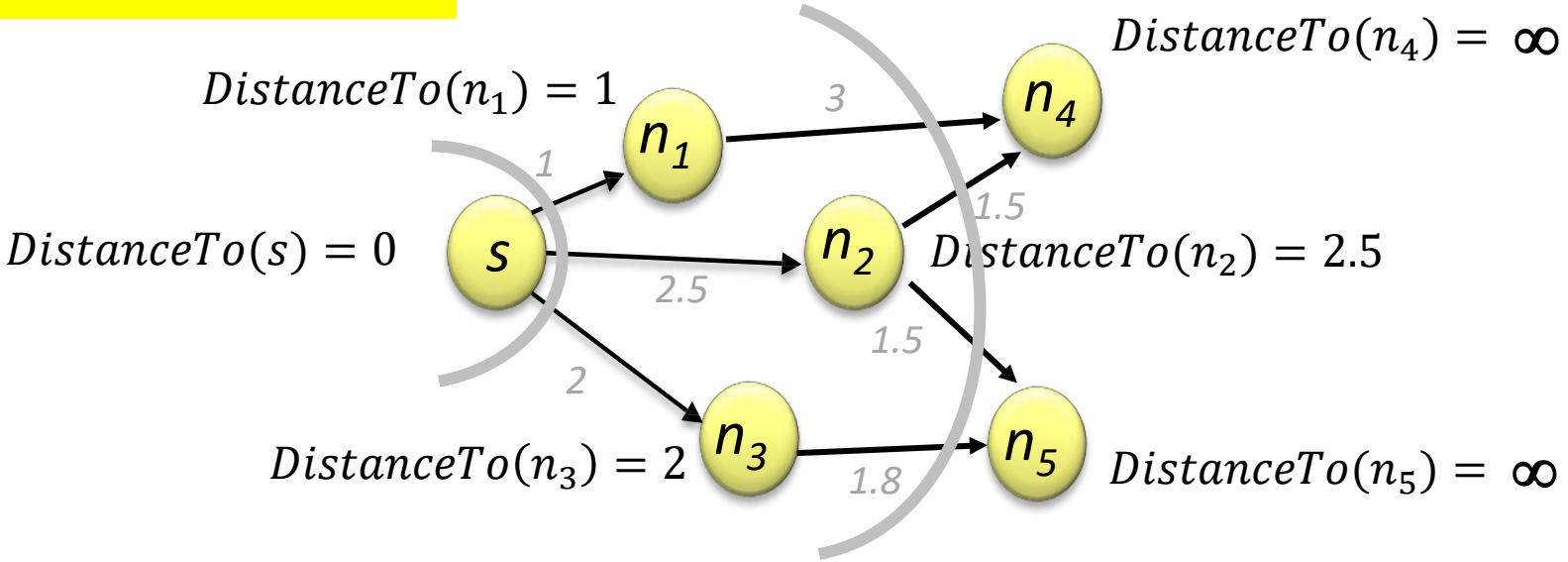


Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

$$DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$$

Second iteration

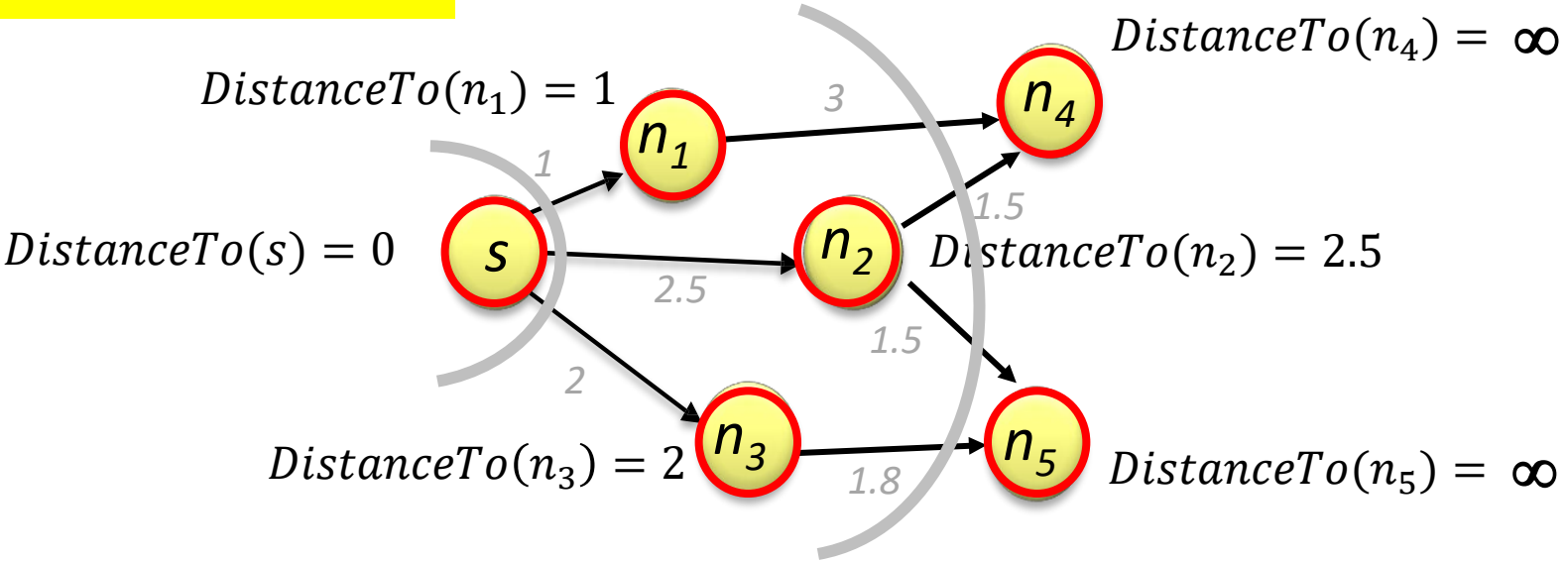


Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

$$DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$$

Second iteration



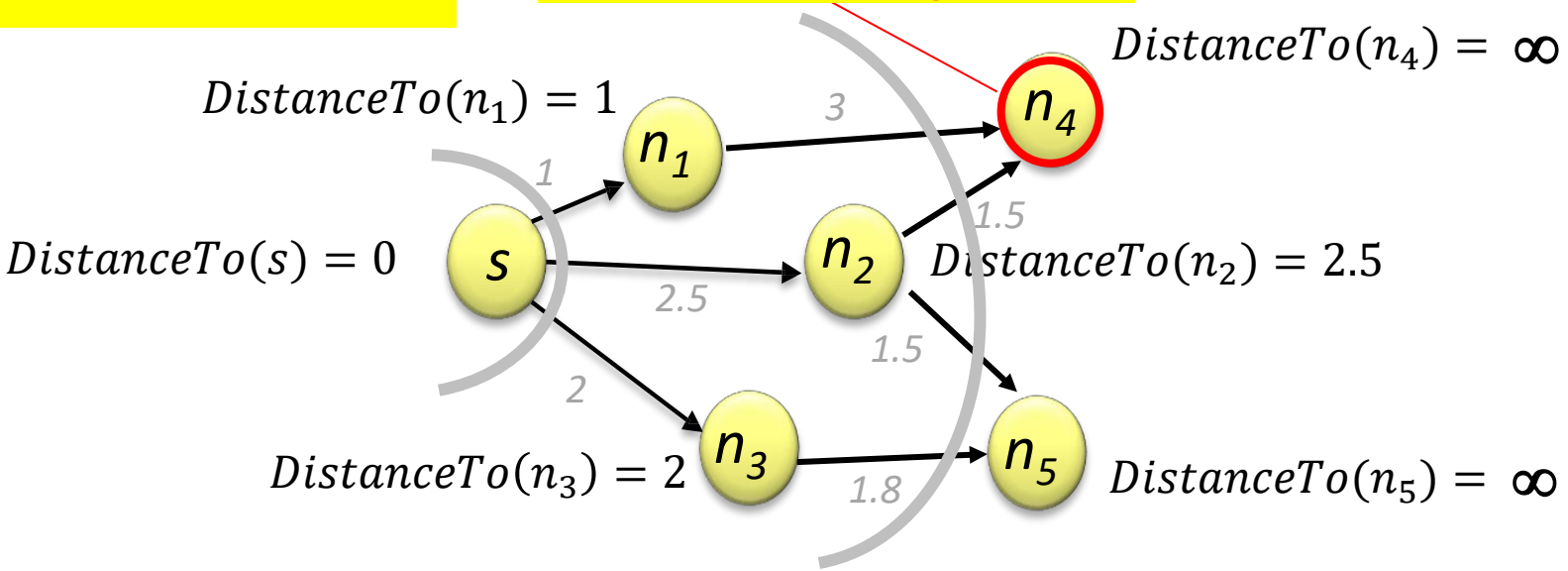
Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

Second iteration

use n4 as example

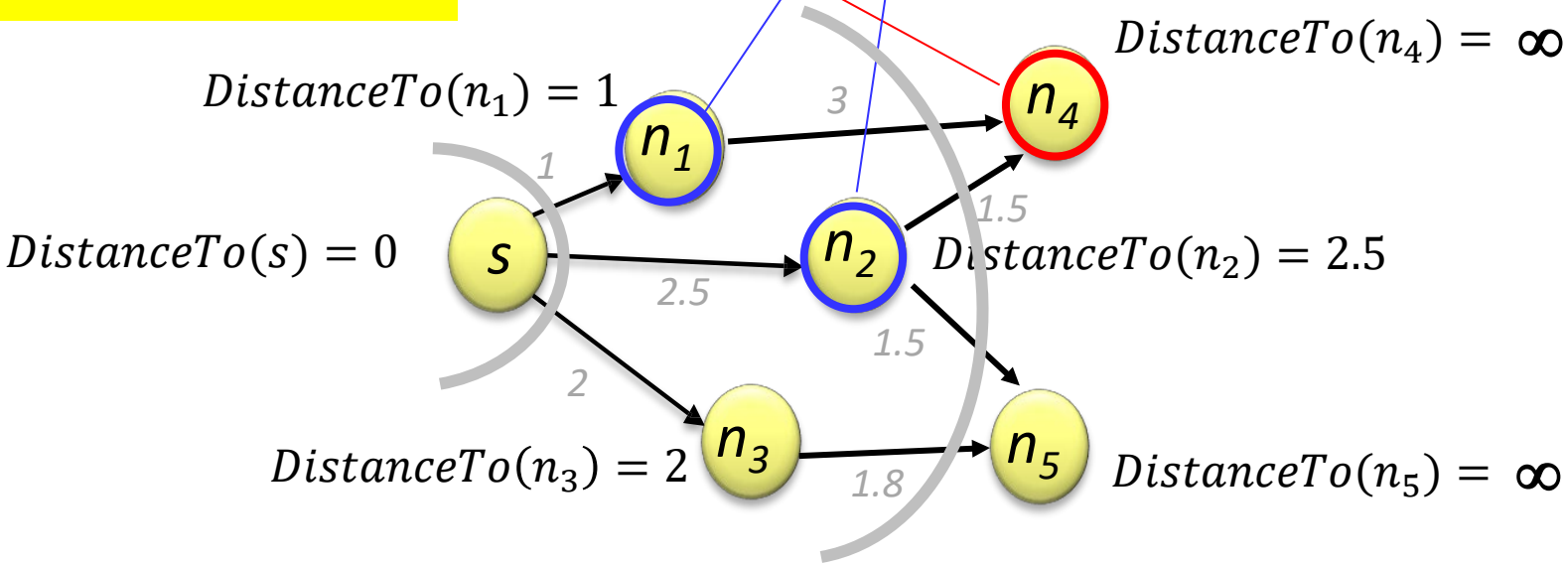


Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

Second iteration

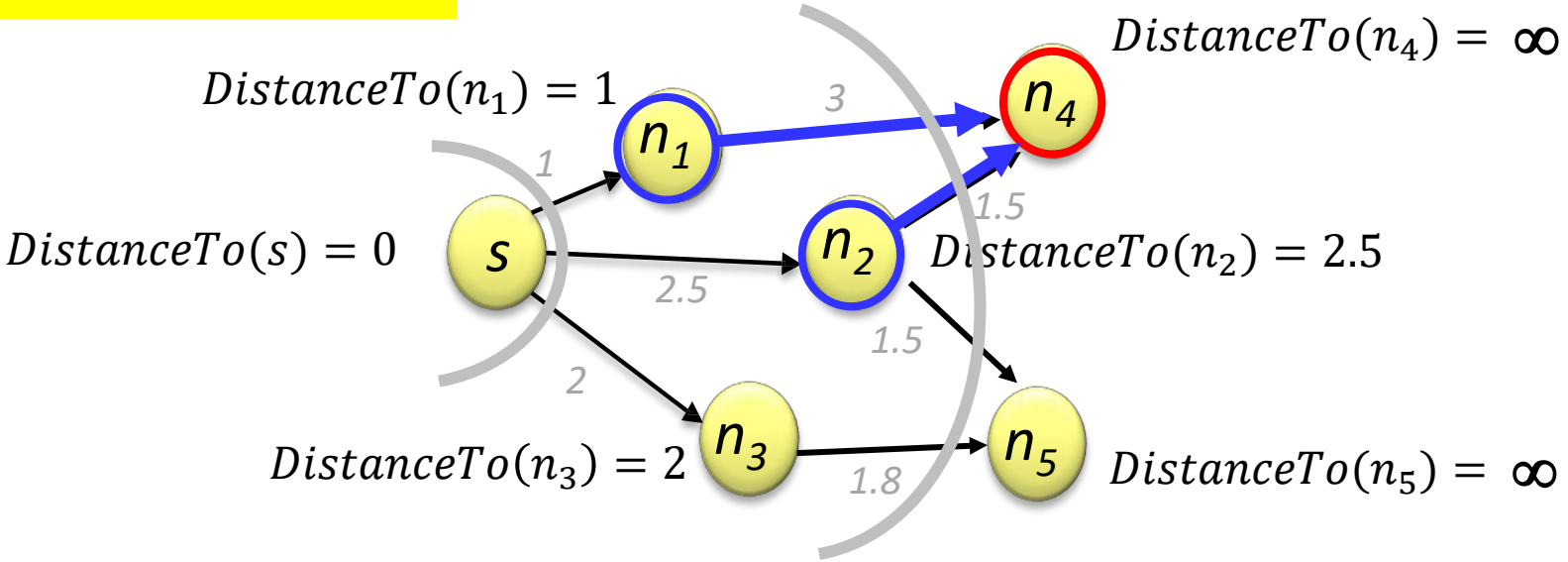


Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

Second iteration

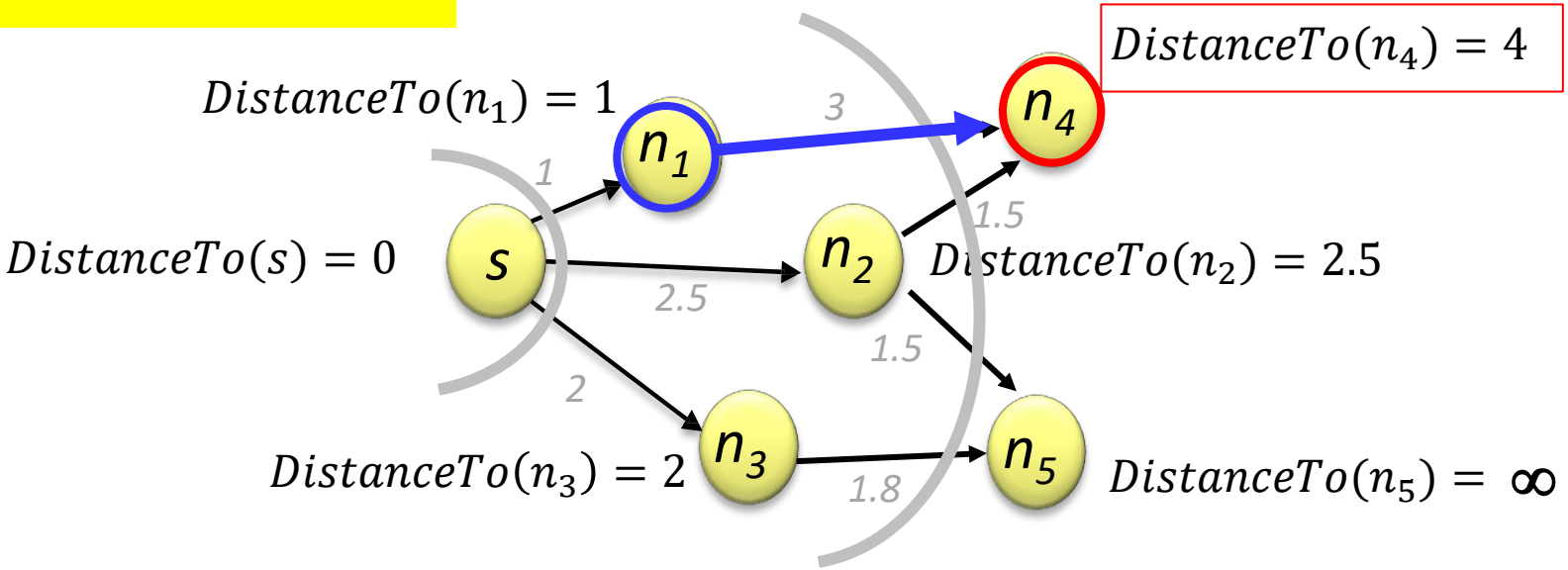


Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

$$DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$$

Second iteration



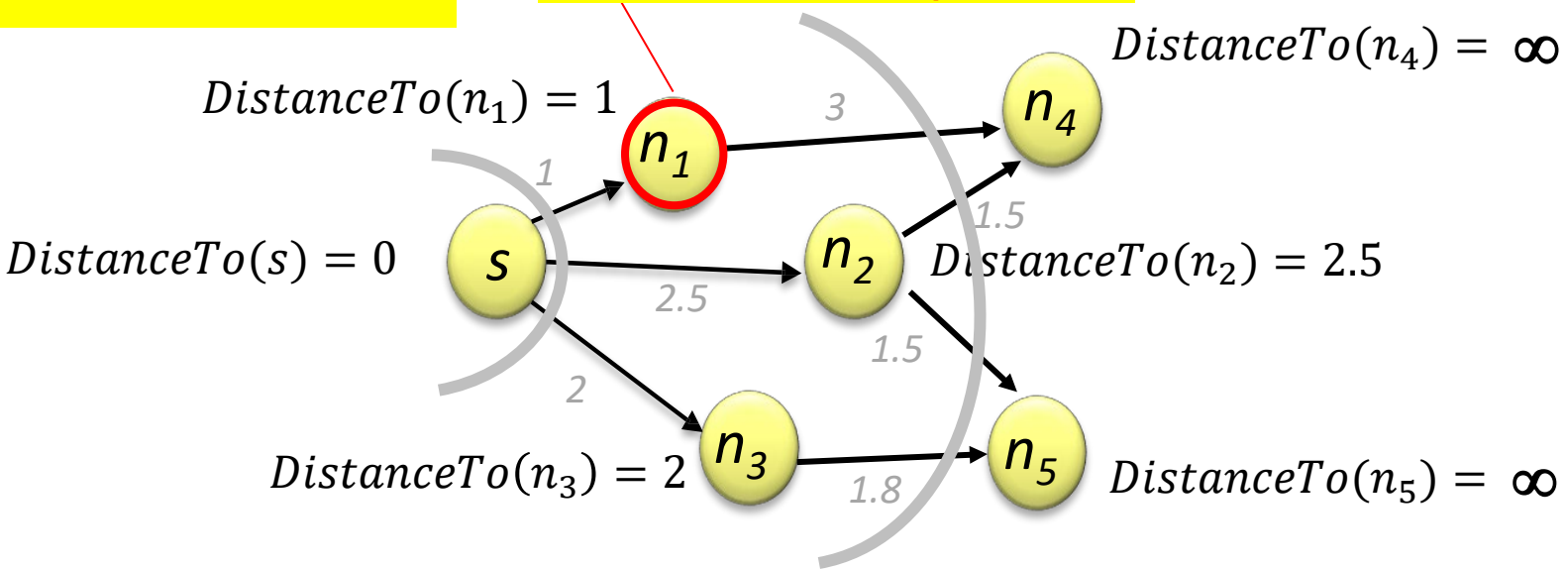
Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

    $DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

Second iteration

use n1 as example

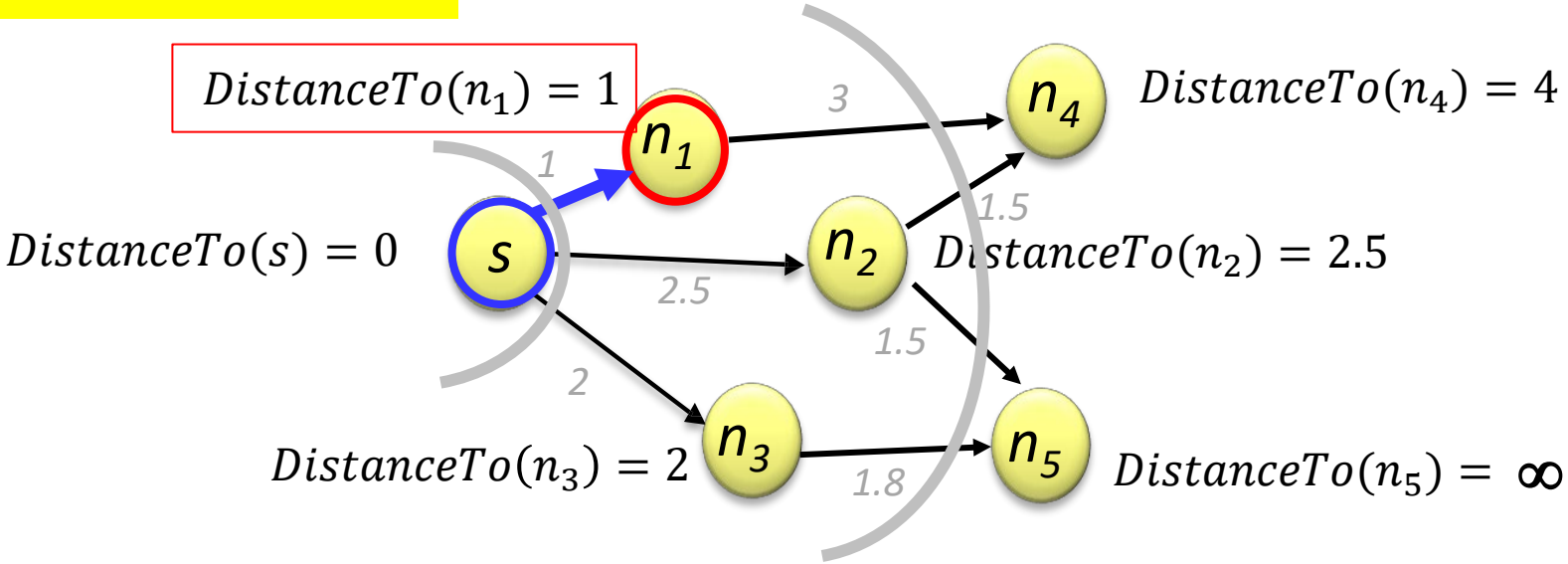


Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

$$DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$$

Second iteration

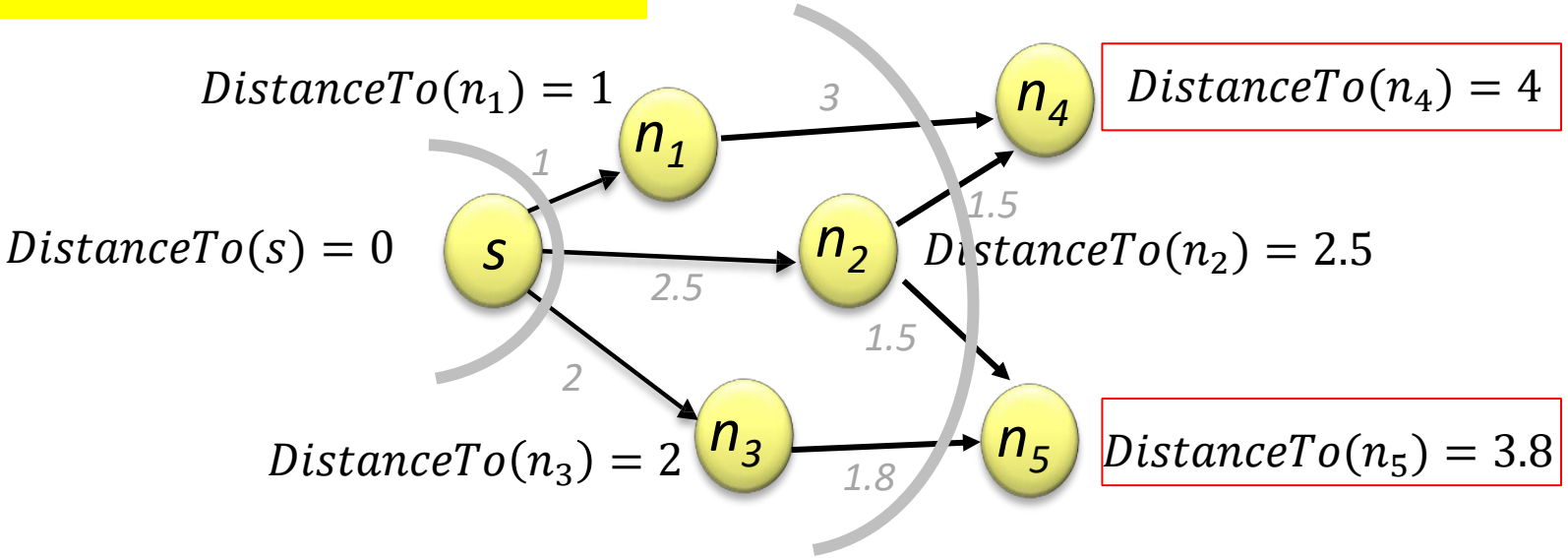


Finding the Shortest Path

- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

$$DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$$

After the second iteration

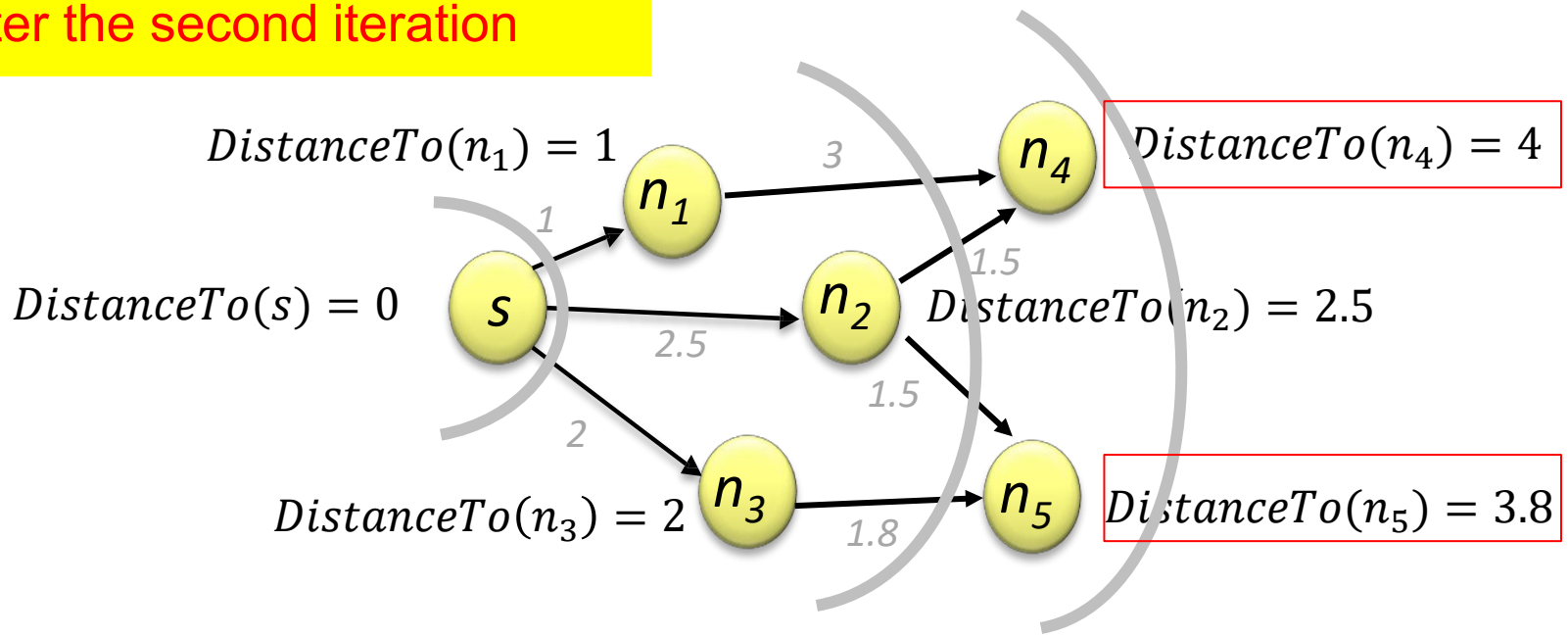


Finding the Shortest Path

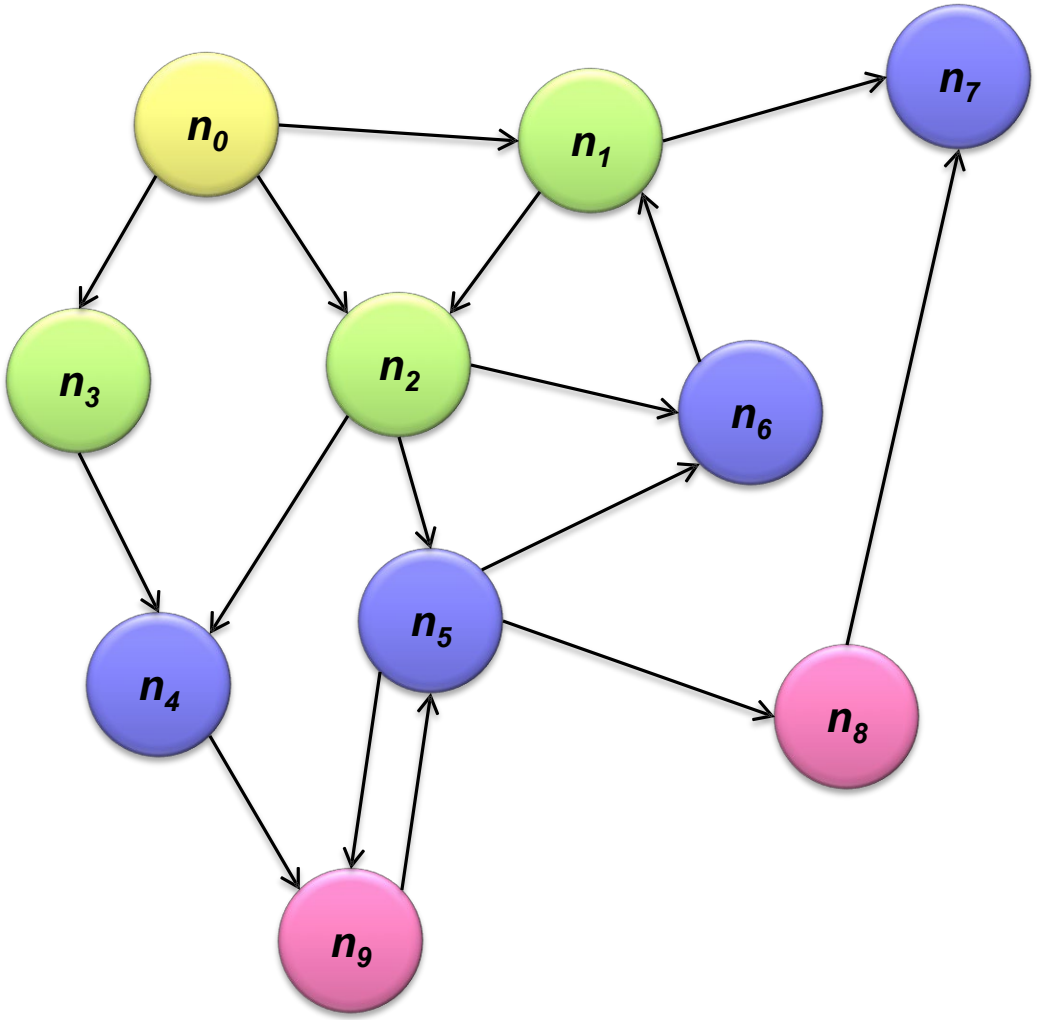
- Here's the intuition:
  - source s, initially
    - $DistanceTo(s) = 0$,
    - Other node n, $DistanceTo(n) = \infty$
  - For each node n reachable from a set of nodes M

$$DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$$

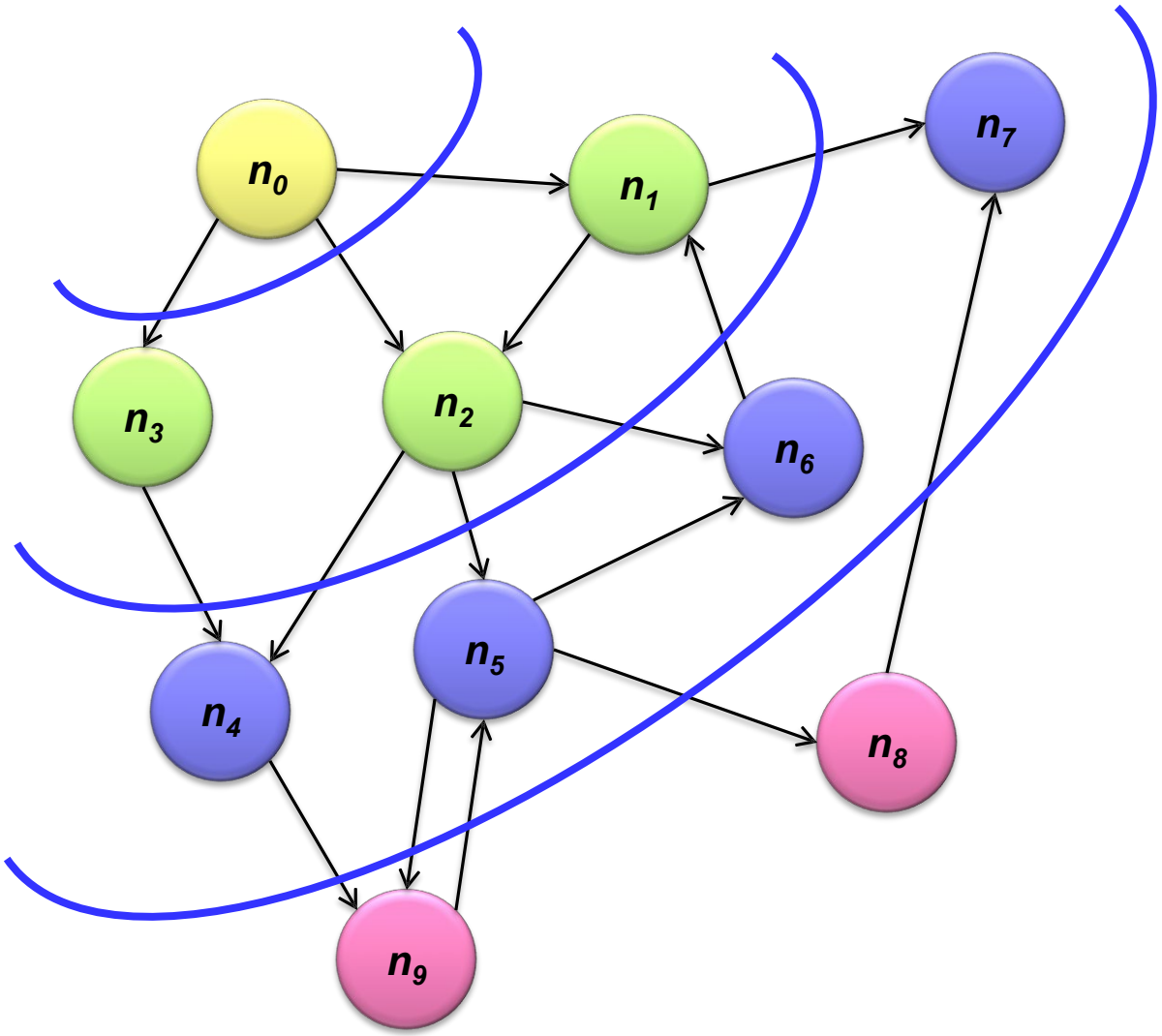
After the second iteration



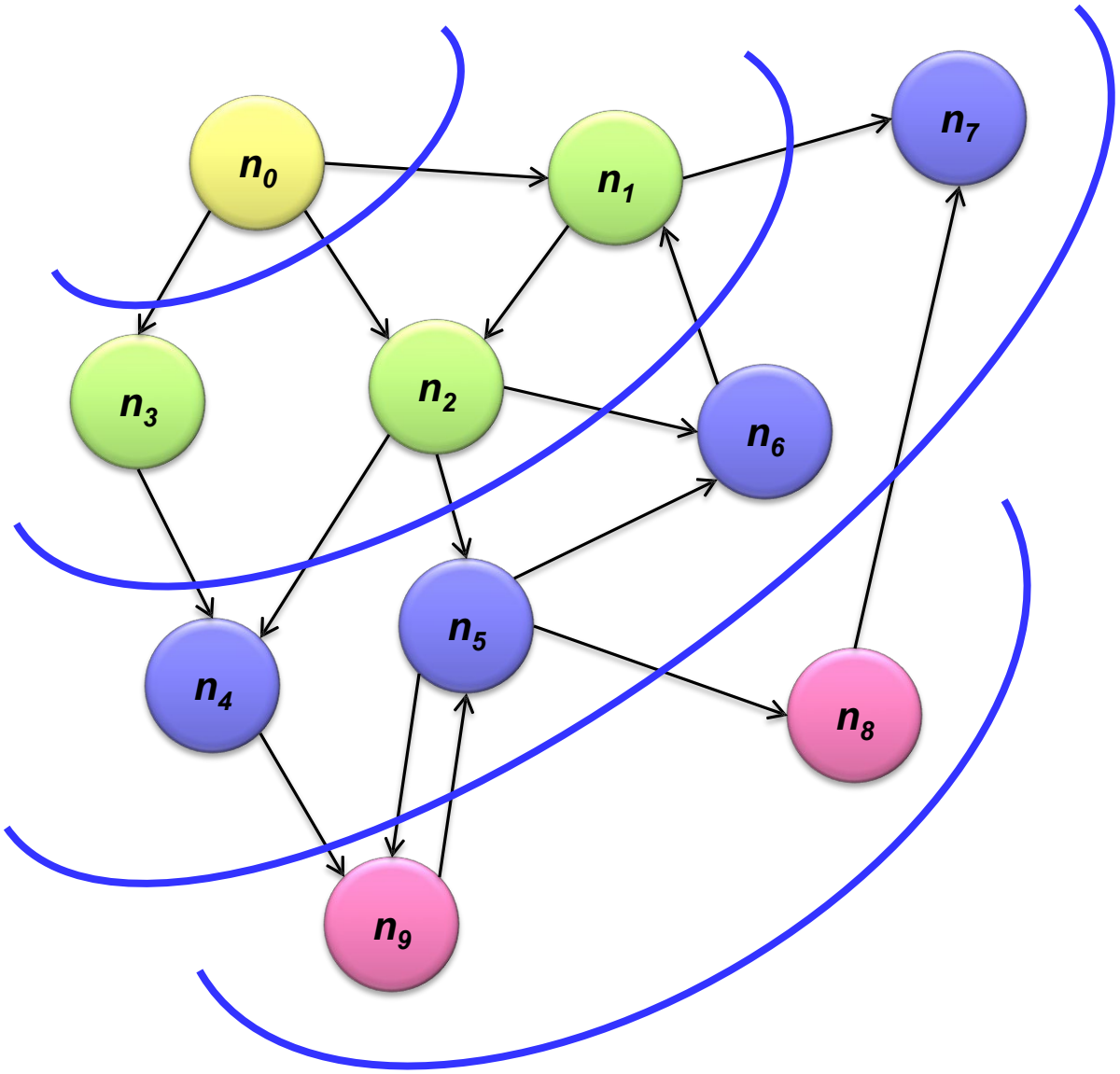
Propagate from source n_0 …



Propagate from source n_0 …



Propagate from source n_0 …



From Intuition to MapReduce Algorithm

Data representation <key, value>:

- Key: node n
- Value: d (distance from start)
  adjacency list (nodes reachable from n)

From Intuition to MapReduce Algorithm

Data representation <key, value>:

- Key: node n
- Value: d (distance from start)
  adjacency list (nodes reachable from n)

Initialization:

for all nodes except for start node, $d = \infty$

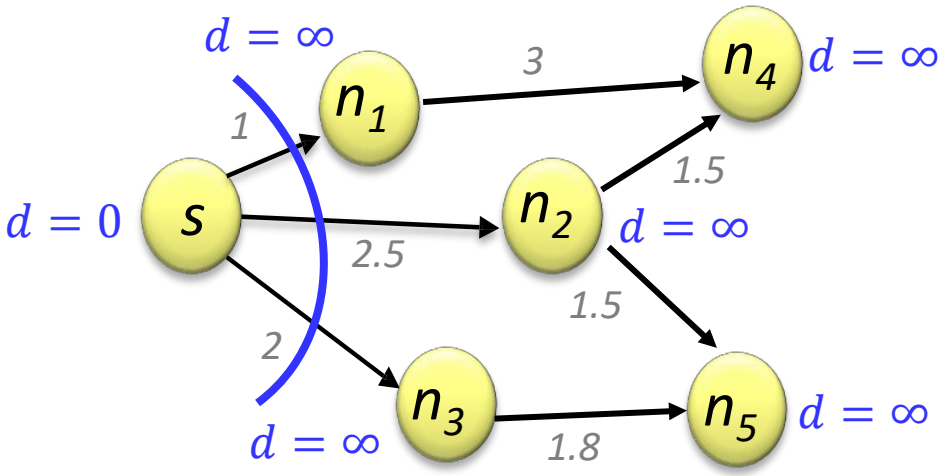
From Intuition to MapReduce Algorithm

Data representation <key, value>:

- Key: node n
- Value: d (distance from start)
  adjacency list (nodes reachable from n)

Initialization:

for all nodes except for start node, $d = \infty$

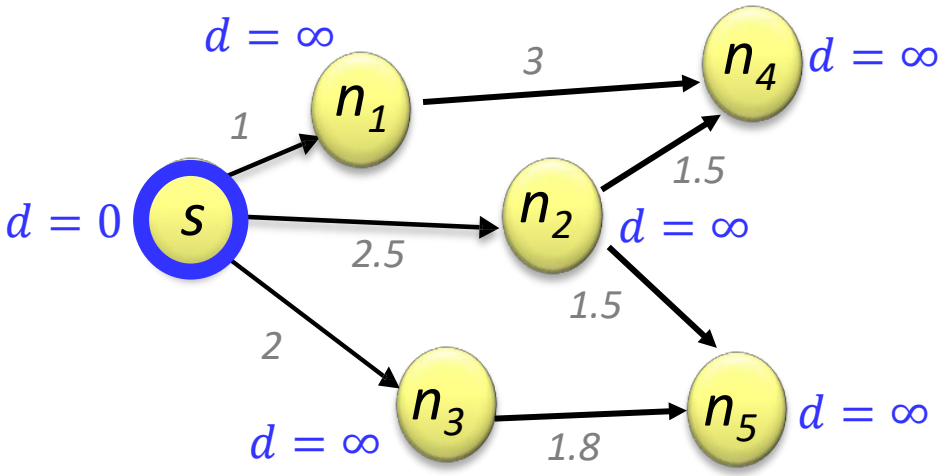


From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For all n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself

First iteration, for s, mapper emits



From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself

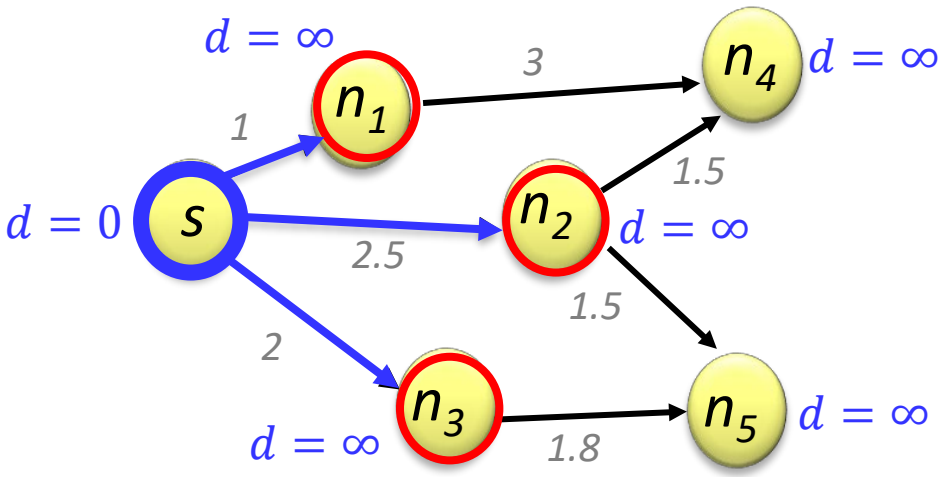
First iteration, for s, mapper emits

Adjacency list

s: n_1, n_2, n_3

n_2: n_5, n_4

...



emit (s, 0)

emit (n_1, 1)

emit (n_2, 2.5)

emit (n_3, 2)

From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself

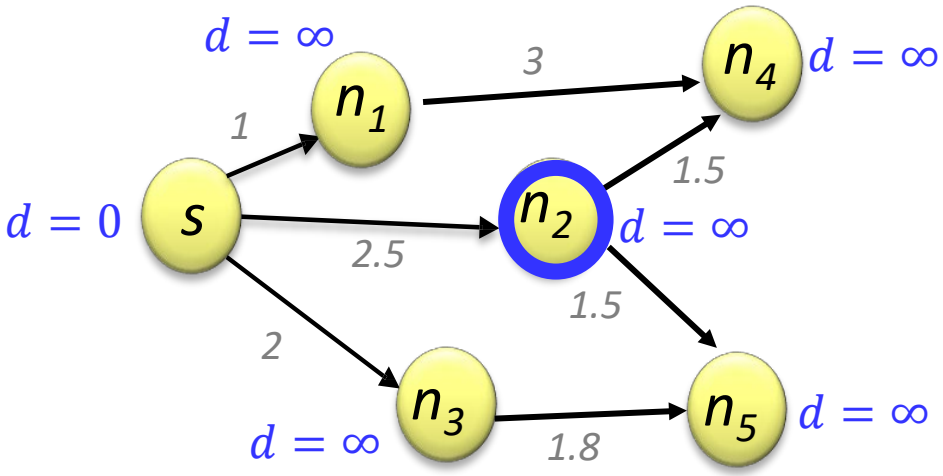
First iteration, for n2, mapper emits

Adjacency list

s: n_1, n_2, n_3

n_2: n_5, n_4

...



From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself

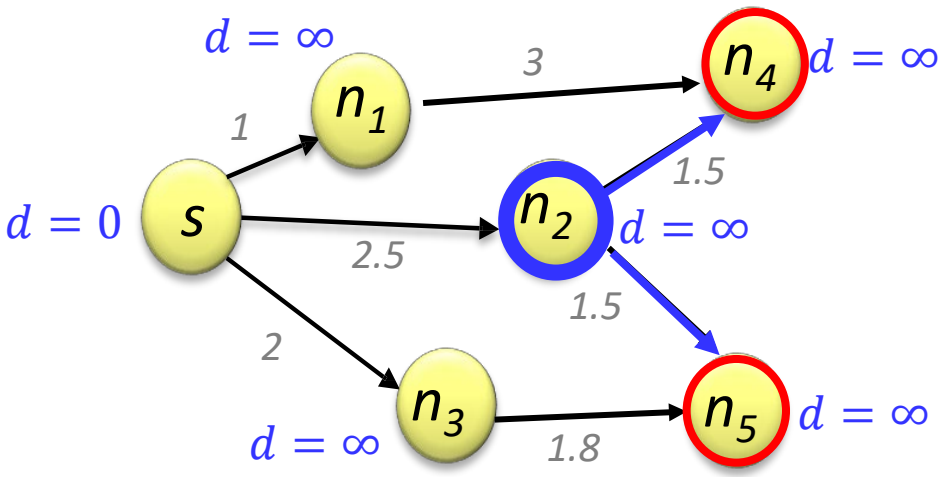
First iteration, for n2, mapper emits

Adjacency list

s: n_1, n_2, n_3

n_2: n_5, n_4

...



emit (n_2, ∞)

emit (n_4, ∞)

emit (n_5, ∞)

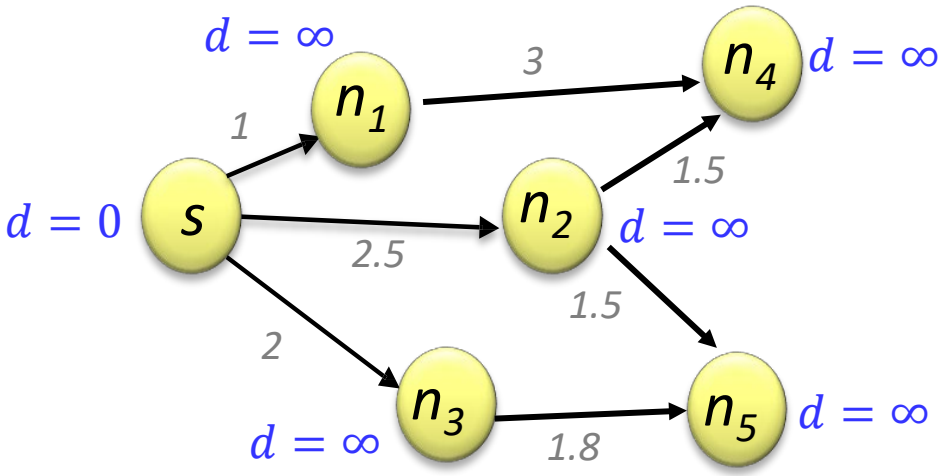
From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself

First iteration

In the same way, mapper process n1, n3, n4, n5

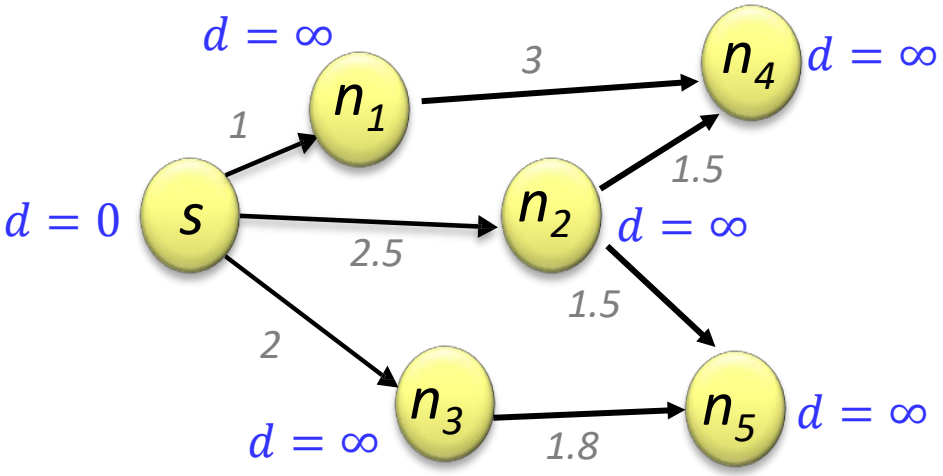


From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n,m))$
  - Remember to also emit distance to yourself

In the first iteration, mapper eventually emits



Adjacency list

server 2

s: n_1, n_2, n_3

n_1: n_4

n_4:

server 1

n_2: n_5, n_4

n_3: n_5

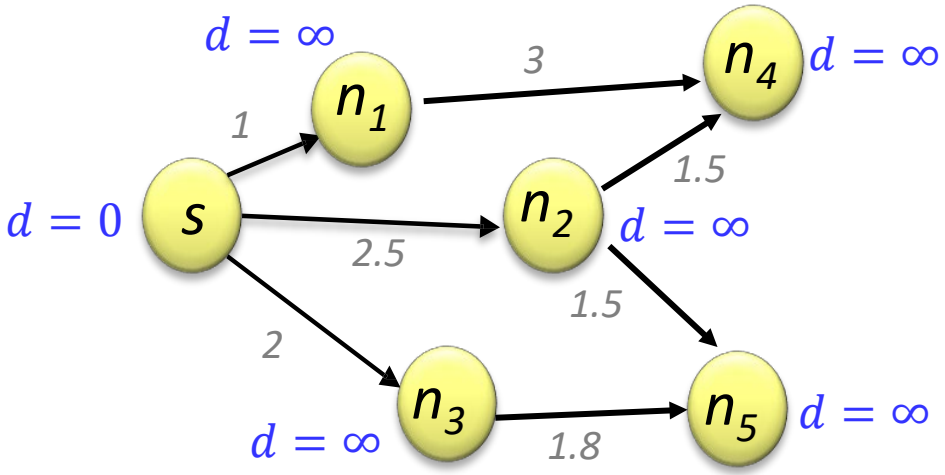
n_5:

From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself

In the first iteration, mapper eventually emits



emit (s, 0)
emit (n_1, 1)
emit (n_2, 2.5)
emit (n_3, 2)

emit (n_1, ∞)
emit (n_4, ∞)

emit (n_4, ∞)

emit (n_2, ∞)
emit (n_4, ∞)
emit (n_5, ∞)

emit (n_3, ∞)
emit (n_5, ∞)

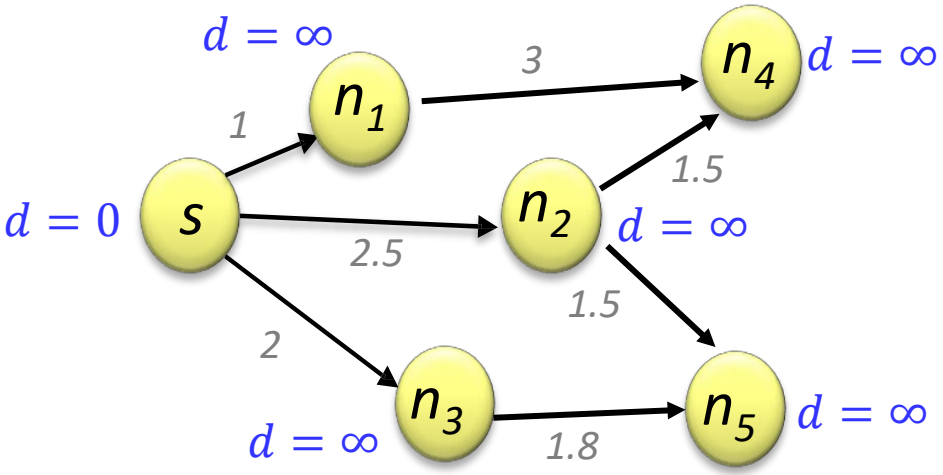
emit (n_5, ∞)

From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself

In the first iteration, mapper eventually emits



server 2

emit (s, 0)
emit (n_1, 1)
emit (n_2, 2.5)
emit (n_3, 2)

emit (n_1, ∞)
emit (n_4, ∞)

server 1

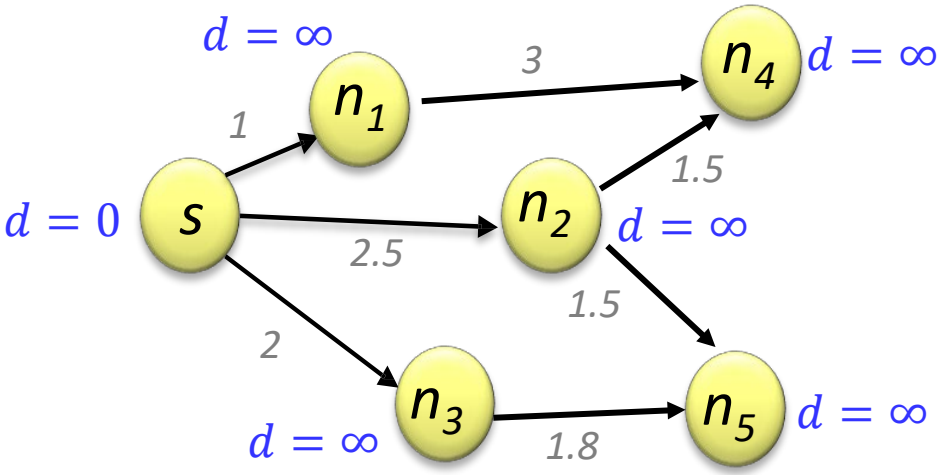
emit (n_2, ∞)
emit (n_4, ∞)
emit (n_5, ∞)

emit (n_3, ∞)
emit (n_5, ∞)

emit (n_5, ∞)

From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:
    emit $(m, DistanceTo(n) + d(n,m))$
  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes

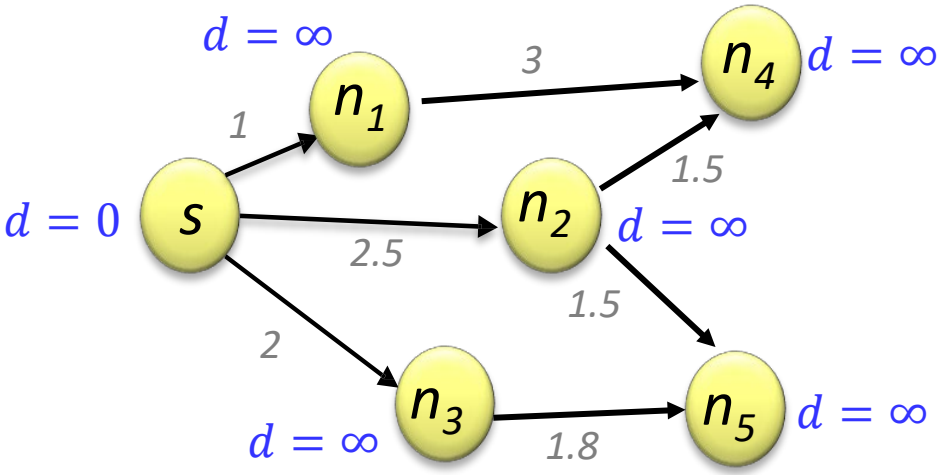


(s, 0)
(n_1, 1)
(n_2, 2.5)
(n_3, 2)
(n_1, ∞)
(n_4, ∞)
(n_4, ∞)

(n_2, ∞)
(n_4, ∞)
(n_5, ∞)
(n_3, ∞)
(n_5, ∞)
(n_5, ∞)

From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:
    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes



(s, 0)
(n_1, 1)
(n_2, 2.5)
(n_3, 2)
(n_1, ∞)
(n_4, ∞)
(n_4, ∞)

(n_2, ∞)
(n_4, ∞)
(n_5, ∞)
(n_3, ∞)
(n_5, ∞)
(n_5, ∞)

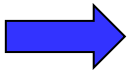
From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes

(s, 0)
(n_1, 1)
(n_2, 2.5)
(n_3, 2)
(n_1, ∞)
(n_4, ∞)
(n_4, ∞)

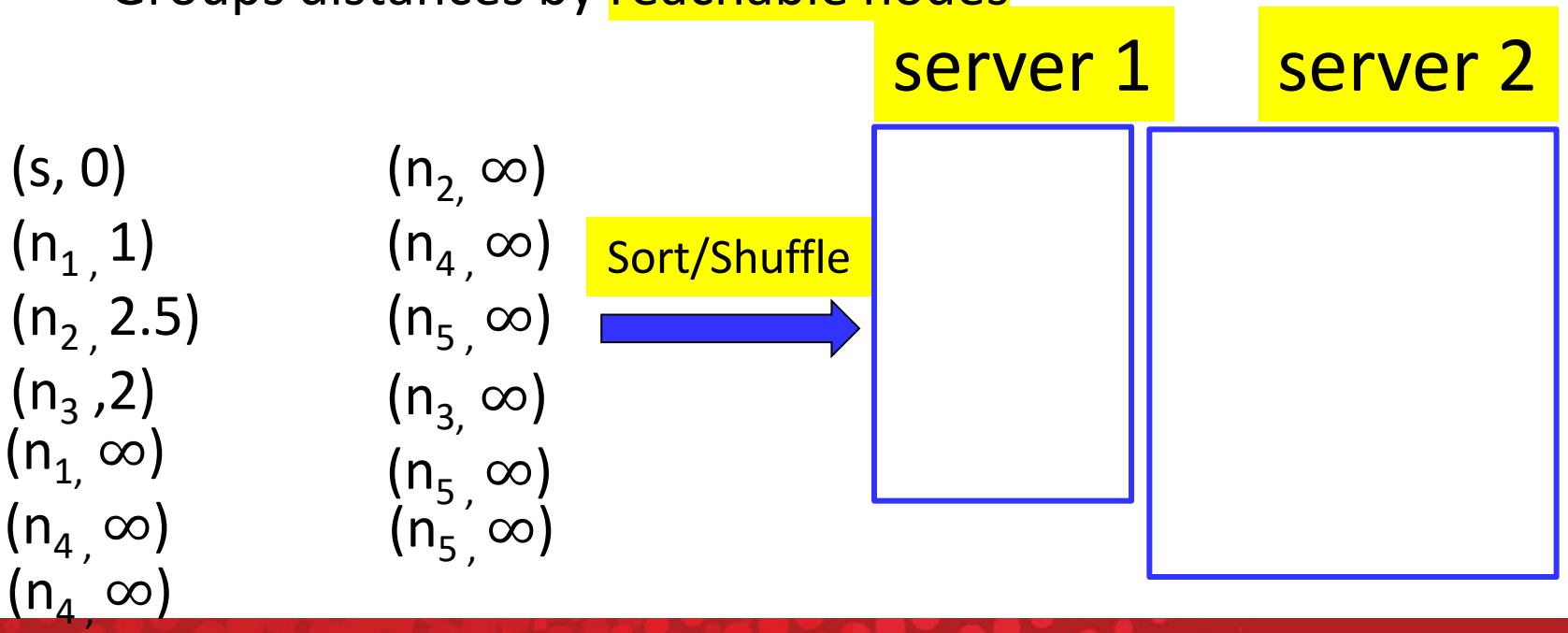
(n_2, ∞)
(n_4, ∞)
(n_5, ∞)
(n_3, ∞)
(n_5, ∞)
(n_5, ∞)



From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

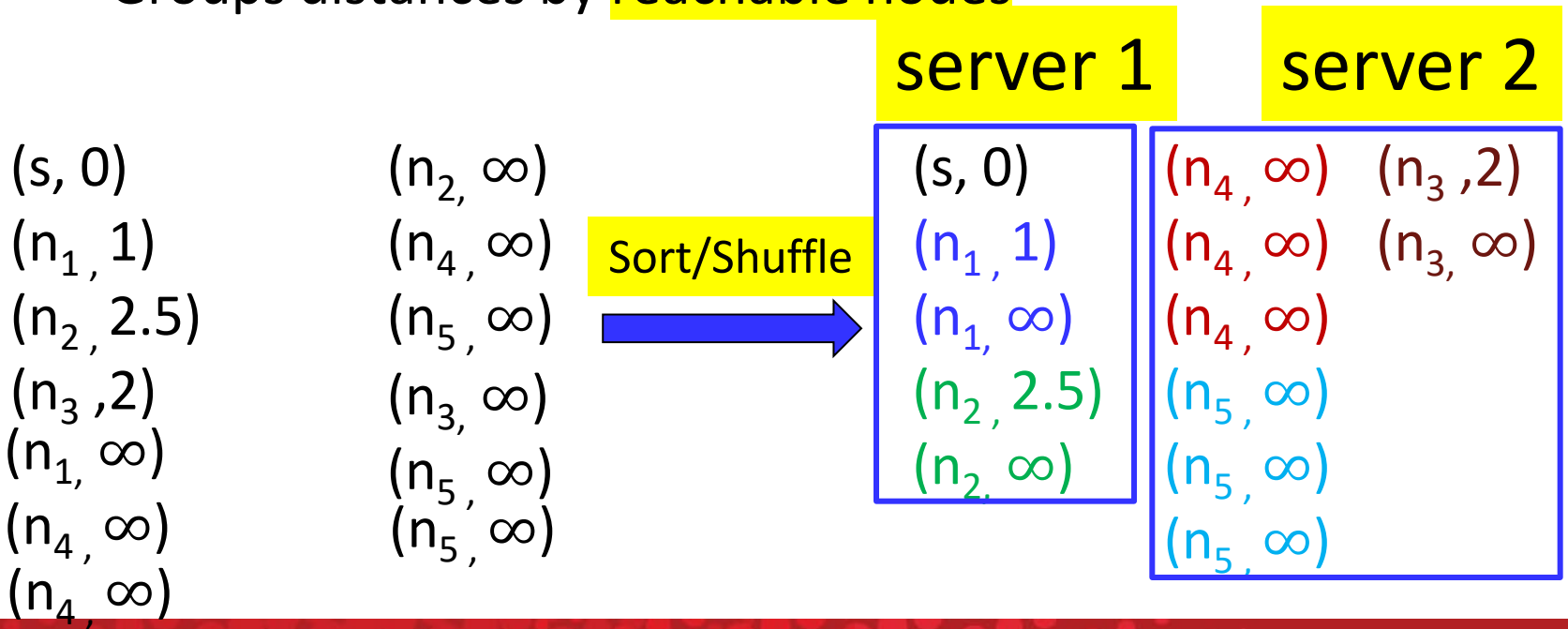
    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes



From Intuition to MapReduce Algorithm – first iteration

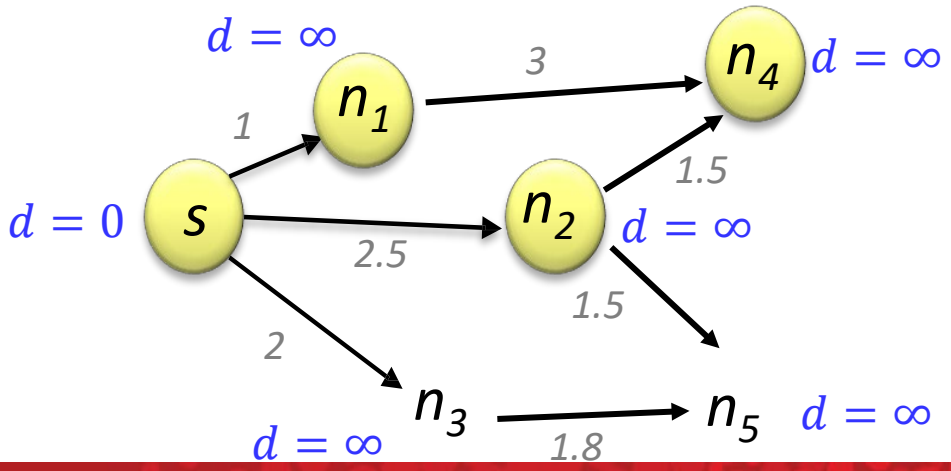
- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes



From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:
    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes
- Reducer:
  - Selects minimum distance path for each reachable node
  - Additional bookkeeping needed to keep track of actual path



server 1

(s, 0)
(n_1, 1)
(n_1, ∞)
(n_2, 2.5)
(n_2, ∞)

server 2

(n_4, ∞) (n_3, 2)
(n_4, ∞) (n_3, ∞)
(n_4, ∞)
(n_5, ∞)
(n_5, ∞)
(n_5, ∞)

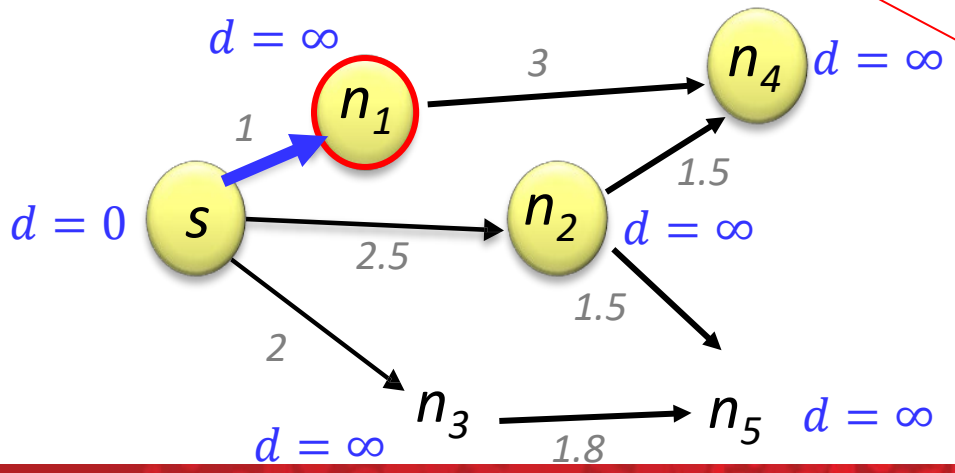
- source s, initially
  - $DistanceTo(s) = 0$,
  - Other node n, $DistanceTo(n) = 0$
- For each node n reachable from a set of nodes M

$$DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$$

– first iteration

emit $(m, DistanceTo(n) + d(n,m))$

  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes
- Reducer:
  - Selects minimum distance path for each reachable node
  - Additional bookkeeping needed to keep track of actual path



server 1

(s, 0)
(n_1, 1)
(n_1, ∞)
(n_2, 2.5)
(n_2, ∞)

server 2

(n_4, ∞) (n_3, 2)
(n_4, ∞) (n_3, ∞)
(n_4, ∞)
(n_5, ∞)
(n_5, ∞)
(n_5, ∞)

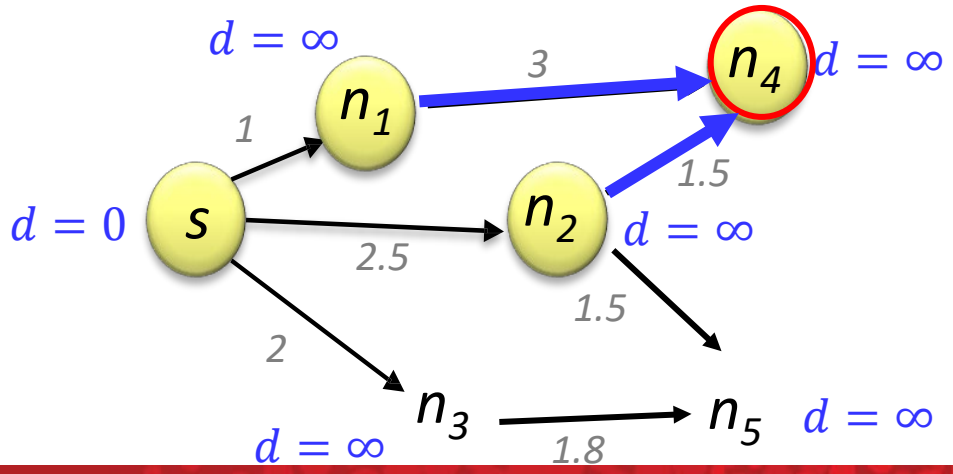
- source s, initially
  - $DistanceTo(s) = 0$,
  - Other node n, $DistanceTo(n) = 0$
- For each node n reachable from a set of nodes M

$DistanceTo(n) = \min(d(n,m) + DistanceTo(m), m \in M)$

– first iteration

emit $(m, DistanceTo(n) + d(n,m))$

  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes
- Reducer:
  - Selects minimum distance path for each reachable node
  - Additional bookkeeping needed to keep track of actual path



server 1

(s, 0)
(n_1, 1)
(n_1, ∞)
(n_2, 2.5)
(n_2, ∞)

server 2

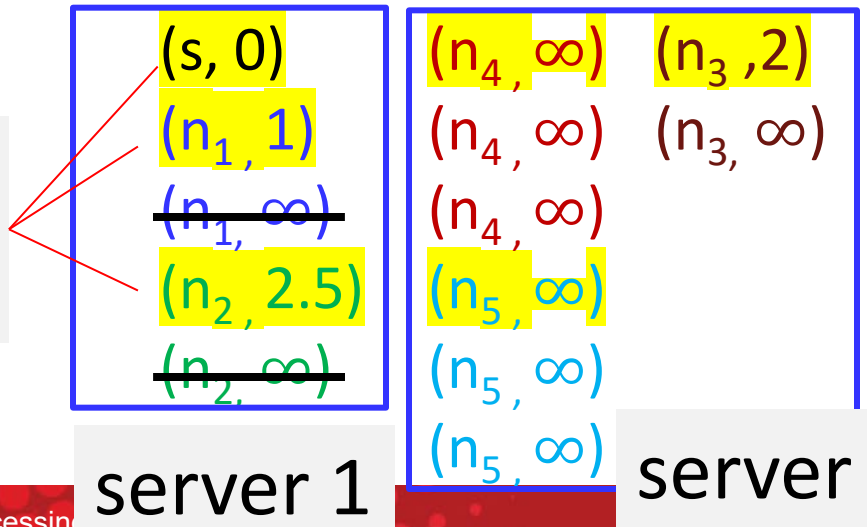
(n_4, ∞)
(n_4, ∞)
(n_4, ∞)
(n_5, ∞)
(n_5, ∞)
(n_5, ∞)
(n_3, 2)
(n_3, ∞)

From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes
- Reducer:
  - Selects minimum distance path for each reachable node
  - Additional bookkeeping needed to keep track of actual path

reducer output on server 1

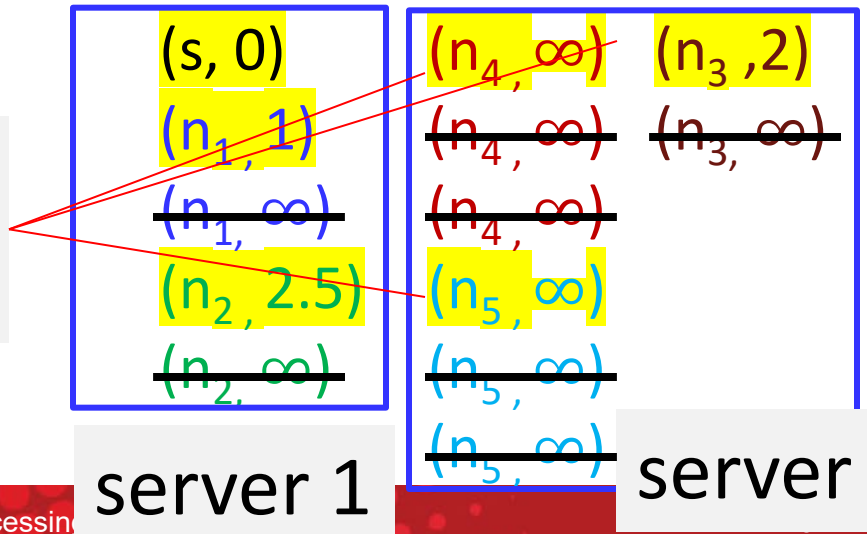


From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes
- Reducer:
  - Selects minimum distance path for each reachable node
  - Additional bookkeeping needed to keep track of actual path

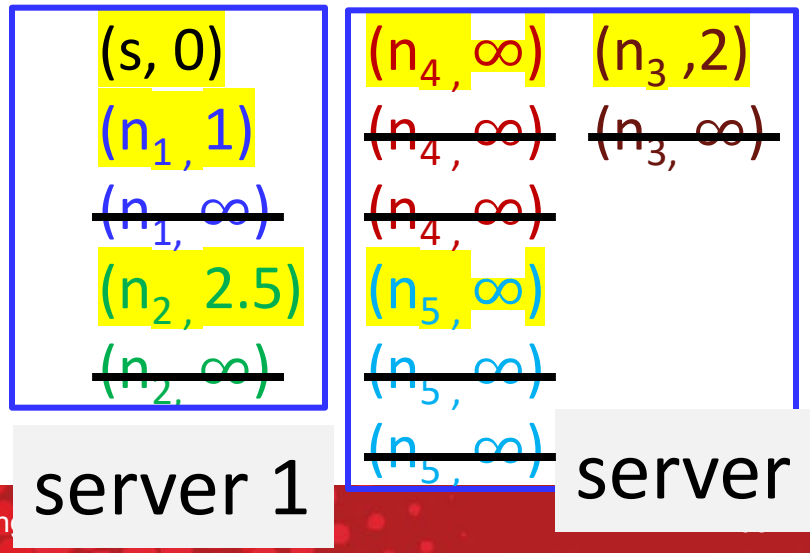
reducer output on server 2



From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:
    emit $(m, DistanceTo(n) + d(n,m))$
  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes
- Reducer:
  - Selects minimum distance path for each reachable node
  - Additional bookkeeping needed to keep track of actual path

First iteration complete!

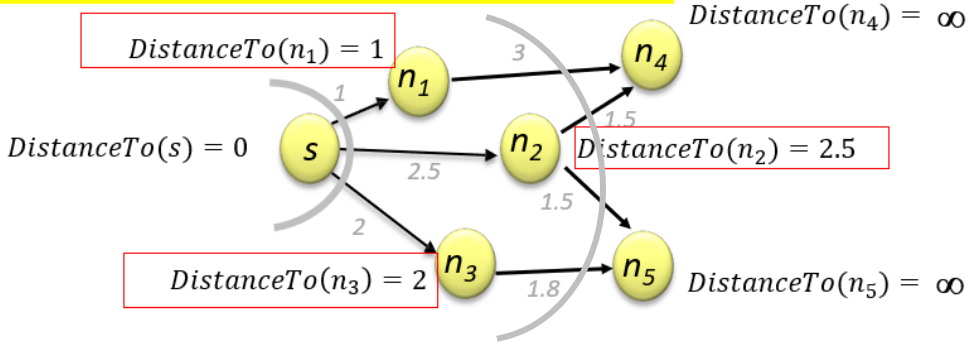


From Intuition to MapReduce Algorithm – first iteration

- Mapper:
  - For each n, $\forall m \in$ n's adjacency list:

    emit $(m, DistanceTo(n) + d(n, m))$
  - Remember to also emit distance to yourself
- Sort/Shuffle
  - Groups distances by reachable nodes
- Reducer:
  - Selects minimum distance path for each reachable node
  - Additional bookkeeping needed to keep track of actual path

At the end of first iteration, n1,n2,n3,n4,n5



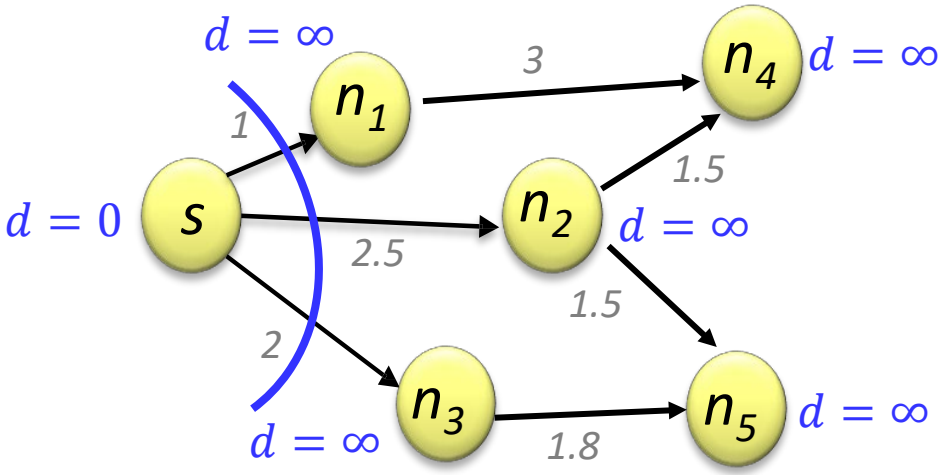
server 1

(s, 0)
(n1 , 1)
~~(n1, ∞)~~
(n2 , 2.5)
~~(n2, ∞)~~

server 2

(n4 , ∞)
(n3 ,2)
~~(n4 , ∞)~~
~~(n3, ∞)~~
~~(n4 , ∞)~~
(n5 , ∞)
~~(n5 , ∞)~~
~~(n5 , ∞)~~

From Intuition to MapReduce Algorithm – first iteration



Initialization

First iteration

BFS Pseudo-Code

mapper.py
Input: node file
For each node (nodeid n, node N) do
    $d \leftarrow N.Distance$
    emit(n, N)
    For each nodeid $m \in N.AdjacencyList$ do
        emit($m, d + l$)

reducer.py
Input: nodeid m, $[d_1, d_2, \ldots]$
    $d_{min} \leftarrow \infty$
    $M \leftarrow \emptyset$
    For each $d \in [d_1, d_2, \ldots]$ do
        if IsNode(d) then
            $M \leftarrow d$
        else if $d < d_{min}$ then
            $d_{min} \leftarrow d$
    $M.Distance \leftarrow d_{min}$
    emit(m, M)

First iteration

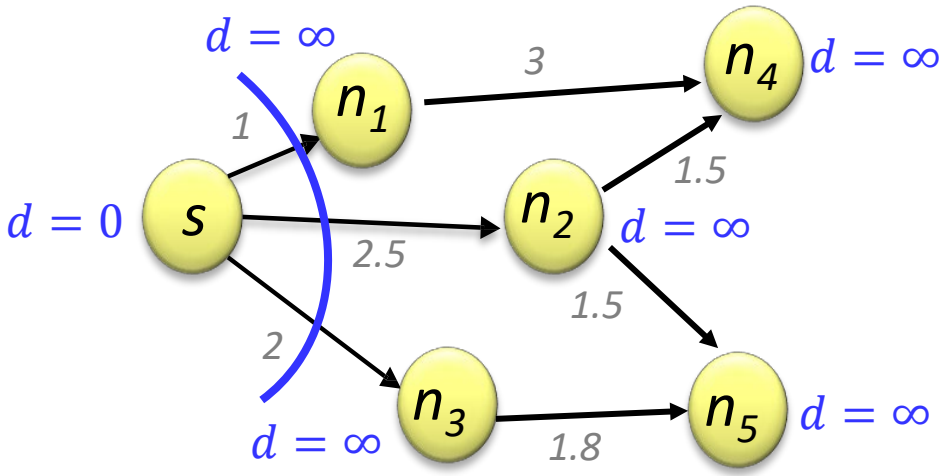
BFS Pseudo-Code – First Iteration

Adjacency list $n_1: n_4$

mapper.py
Input: node file
For each node (nodeid n, node N) do
    $d \leftarrow N.Distance$
    emit(n, N)
    For each nodeid $m \in N.AdjacencyList$ do
        emit$(m, d + l)$

node n_1 emits
(n_1, N_1)
(n_1, ∞)
(n_4, ∞)

reducer.py
Input: nodeid m, $[d_1, d_2, \ldots]$
    $d_{min} \leftarrow \infty$
    $M \leftarrow \emptyset$
    For each $d \in [d_1, d_2, \ldots]$ do
        if IsNode(d) then
            $M \leftarrow d$
        else if $d < d_{min}$ then
            $d_{min} \leftarrow d$
    $M.Distance \leftarrow d_{min}$
    emit(m, M)



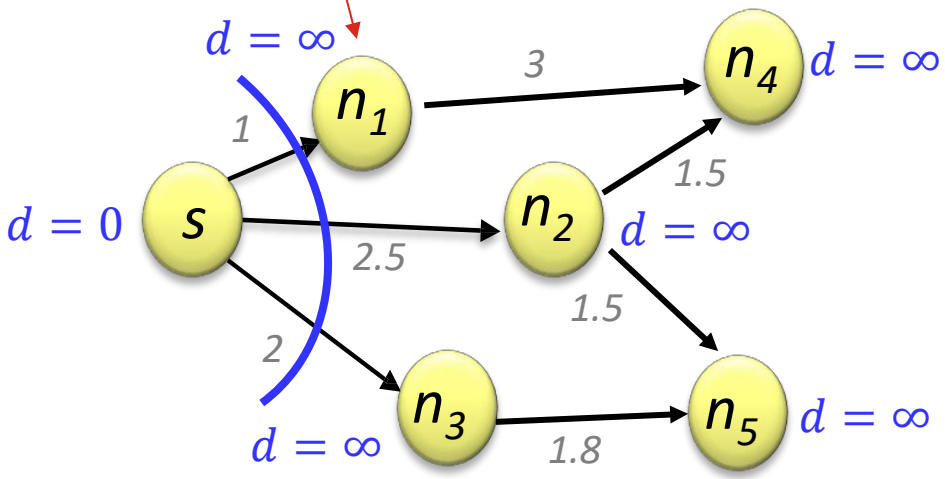
BFS Pseudo-Code – First Iteration

mapper.py
Input: node file
For each node (nodeid n, node N) do
    $d \leftarrow N.Distance$
    emit(n, N)
    For each nodeid $m \in N.AdjacencyList$ do
        emit($m, d + l$)

reducer.py
Input: nodeid m, $[d_1, d_2, \ldots]$
    $d_{min} \leftarrow \infty$
    $M \leftarrow \emptyset$
    For each $d \in [d_1, d_2, \ldots]$ do
        if IsNode(d) then
            $M \leftarrow d$
        else if $d < d_{min}$ then
            $d_{min} \leftarrow d$
    $M.Distance \leftarrow d_{min}$
    emit(m, M)

Adjacency list $n_1: n_4$

node n_1 emits
(n_1, N_1)
(n_1, ∞)
(n_4, ∞)



BFS Pseudo-Code – First Iteration

mapper.py
Input: node file
For each node (nodeid n, node N) do
    $d \leftarrow N.Distance$
    emit(n, N)
    For each nodeid $m \in N.AdjacencyList$ do
        emit($m, d + l$)

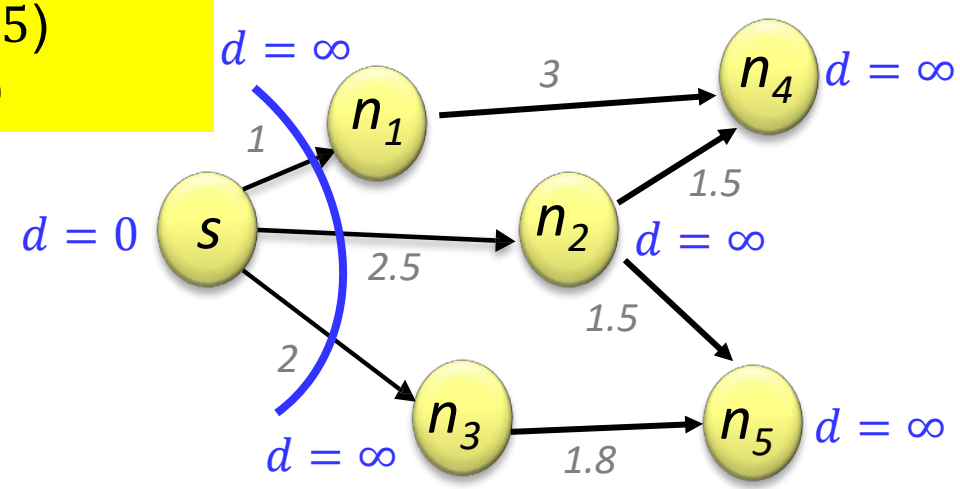
reducer.py
Input: nodeid m, $[d_1, d_2, \ldots]$
    $d_{min} \leftarrow \infty$
    $M \leftarrow \emptyset$
    For each $d \in [d_1, d_2, \ldots]$ do
        if IsNode(d) then
            $M \leftarrow d$
        else if $d < d_{min}$ then
            $d_{min} \leftarrow d$
    $M.Distance \leftarrow d_{min}$
    emit(m, M)

Adjacency list n_1: n_4

node n_1 emits
(n_1, N_1)
(n_1, ∞)
(n_4, ∞)

node S emits
(S, N_s)
$(S, 0)$
$(n_1, 1)$
$(n_2, 2.5)$
$(n_3, 2)$

Adjacency list s: n_1, n_2, n_3



BFS Pseudo-Code – First Iteration

mapper.py
Input: node file
For each node (nodeid n, node N) do
    $d \leftarrow N.Distance$
    emit(n, N)
    For each nodeid $m \in N.AdjacencyList$ do
        emit($m, d + l$)

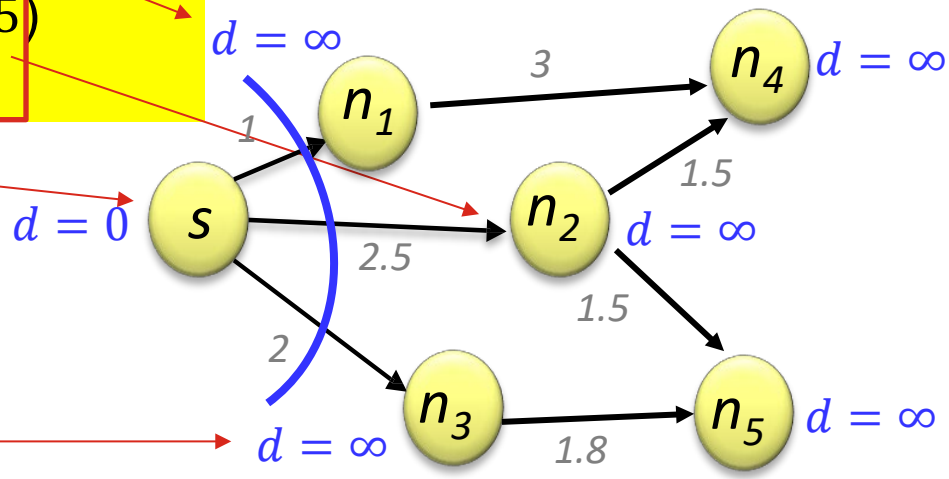
reducer.py
Input: nodeid m, $[d_1, d_2, \ldots]$
    $d_{min} \leftarrow \infty$
    $M \leftarrow \emptyset$
    For each $d \in [d_1, d_2, \ldots]$ do
        if IsNode(d) then
            $M \leftarrow d$
        else if $d < d_{min}$ then
            $d_{min} \leftarrow d$
    $M.Distance \leftarrow d_{min}$
    emit(m, M)

Adjacency list n_1: n_4

node n_1 emits
(n_1, N_1)
(n_1, ∞)
(n_4, ∞)

node S emits
(S, N_s)
$(S, 0)$
$(n_1, 1)$
$(n_2, 2.5)$
$(n_3, 2)$

Adjacency list s: n_1, n_2, n_3



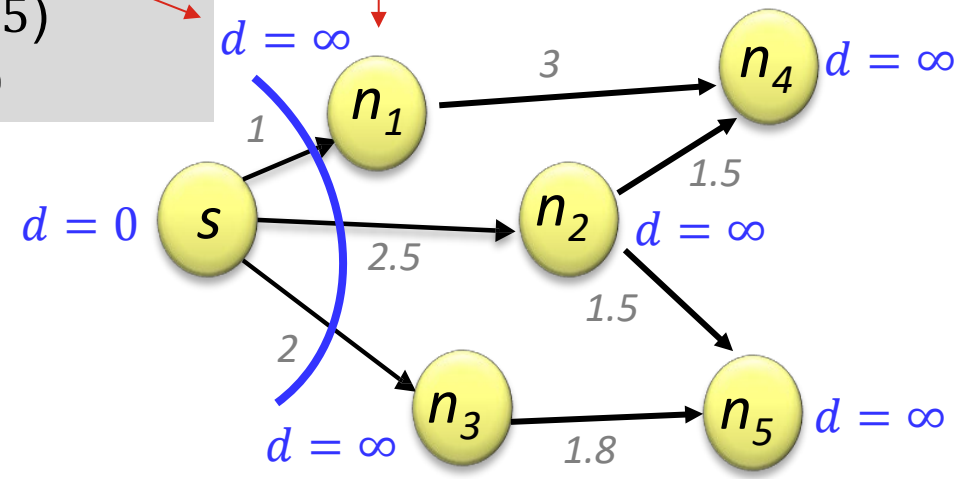
BFS Pseudo-Code

mapper.py
Input: node file
For each node (nodeid n, node N) do
    $d \leftarrow N.Distance$
    emit(n, N)
    For each nodeid $m \in N.AdjacencyList$ do
        emit($m, d + l$)

reducer.py
Input: nodeid m, $[d_1, d_2, \ldots]$
    $d_{min} \leftarrow \infty$
    $M \leftarrow \emptyset$
    For each $d \in [d_1, d_2, \ldots]$ do
        if IsNode(d) then
            $M \leftarrow d$
        else if $d < d_{min}$ then
            $d_{min} \leftarrow d$
    $M.Distance \leftarrow d_{min}$
    emit(m, M)

node n_1 emits
(n_1, N_1)
(n_1, ∞)
(n_4, ∞)

node S emits
(S, N_s)
$(S, 0)$
$(n_1, 1)$
$(n_2, 2.5)$
$(n_3, 2)$



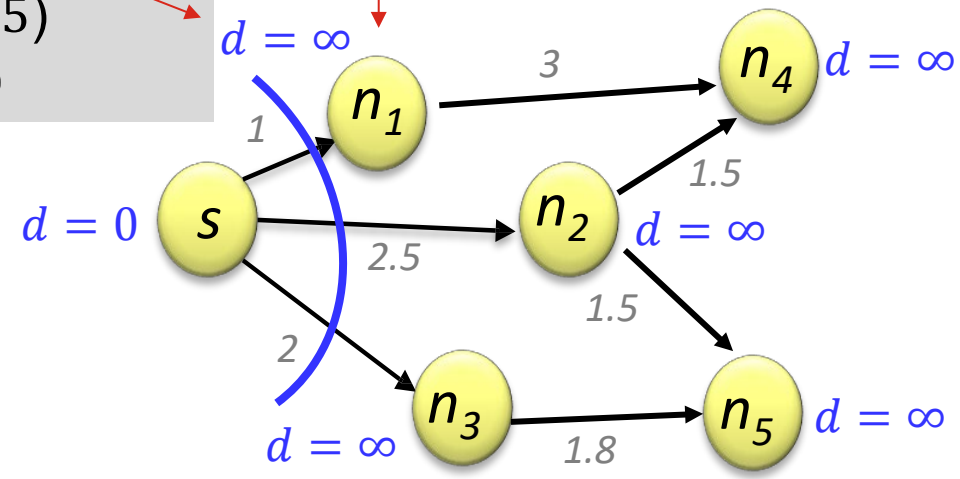
BFS Pseudo-Code

mapper.py
Input: node file
For each node (nodeid n, node N) do
    $d \leftarrow N.Distance$
    emit(n, N)
    For each nodeid $m \in N.AdjacencyList$ do
        emit(m, $d + l$)

reducer.py
Input: nodeid m, $[d_1, d_2, \ldots]$
    $d_{min} \leftarrow \infty$
    $M \leftarrow \emptyset$
    For each $d \in [d_1, d_2, \ldots]$ do
        if IsNode(d) then
            $M \leftarrow d$
        else if $d < d_{min}$ then
            $d_{min} \leftarrow d$
    $M.Distance \leftarrow d_{min}$
    emit(m, M)

node n_1 emits
(n_1, N_1)
(n_1, ∞)
(n_4, ∞)

node S emits
(S, N_s)
$(S, 0)$
$(n_1, 1)$
$(n_2, 2.5)$
$(n_3, 2)$



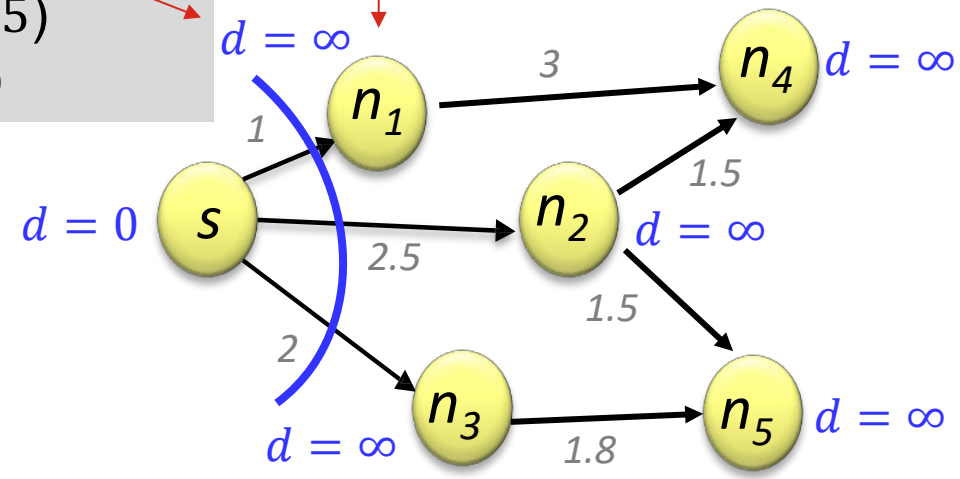
BFS Pseudo-Code

mapper.py
Input: node file
For each node (nodeid n, node N) do
    $d \leftarrow N.Distance$
    emit(n, N)
    For each nodeid $m \in N.AdjacencyList$ do
        emit(m, $d + l$)

reducer.py
Input: nodeid m, [$d_1, d_2, \ldots$]
    $d_{min} \leftarrow \infty$
    $M \leftarrow \emptyset$
    For each $d \in [d_1, d_2, \ldots]$ do
        if IsNode(d) then
            $M \leftarrow d$
        else if $d < d_{min}$ then
            $d_{min} \leftarrow d$
    $M.Distance \leftarrow d_{min}$
    emit(m, M)

node n_1 emits
(n_1, N_1)
(n_1, ∞)
(n_4, ∞)

node S emits
(S, N_s)
$(S, 0)$
$(n_1, 1)$
$(n_2, 2.5)$
$(n_3, 2)$



BFS Pseudo-Code

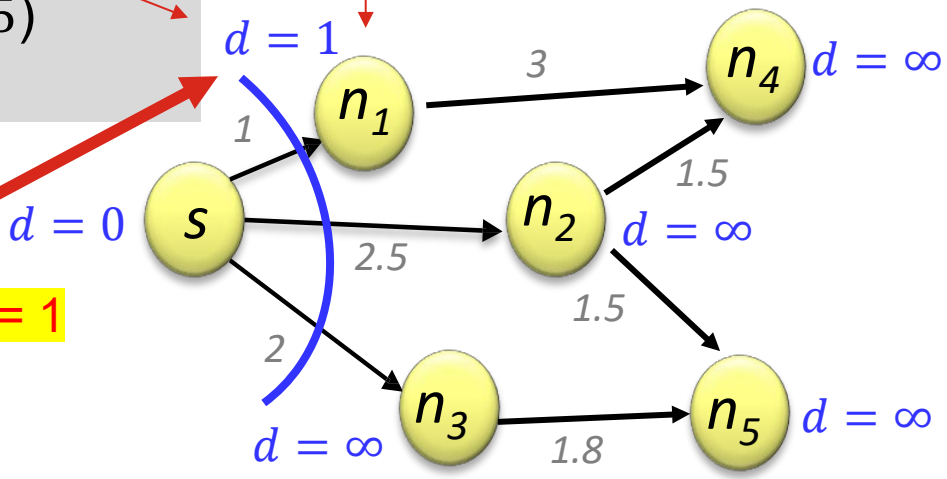
mapper.py
Input: node file
For each node (nodeid n, node N) do
    $d \leftarrow N.Distance$
    emit(n, N)
    For each nodeid $m \in N.AdjacencyList$ do
        emit($m, d + l$)

reducer.py
Input: nodeid m, $[d_1, d_2, \ldots]$
    $d_{min} \leftarrow \infty$
    $M \leftarrow \emptyset$
    For each $d \in [d_1, d_2, \ldots]$ do
        if IsNode(d) then
            $M \leftarrow d$
        else if $d < d_{min}$ then
            $d_{min} \leftarrow d$
    $M.Distance \leftarrow d_{min}$
    emit(m, M)

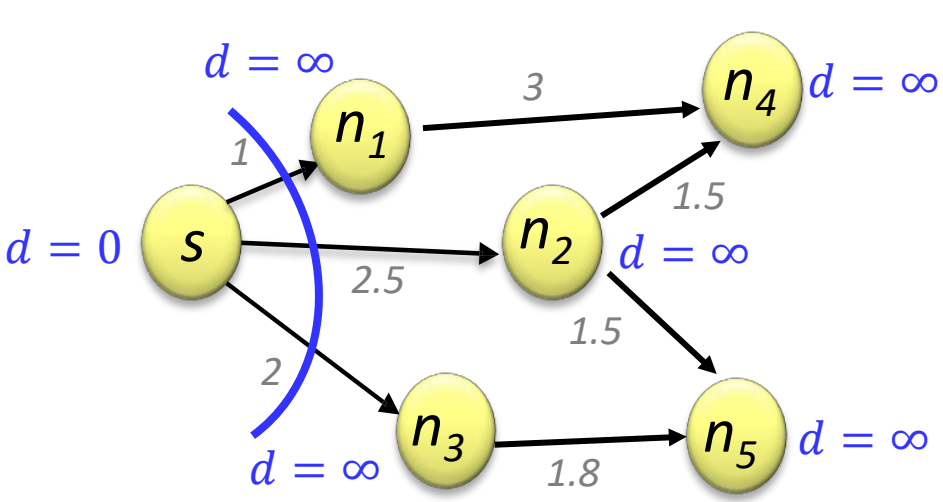
node n_1 emits
(n_1, N_1)
(n_1, ∞)
(n_4, ∞)

n_0 emits
(S, N_s)
$(S, 0)$
$(n_1, 1)$
$(n_2, 2.5)$
$(n_3, 2)$

N1.distance = 1

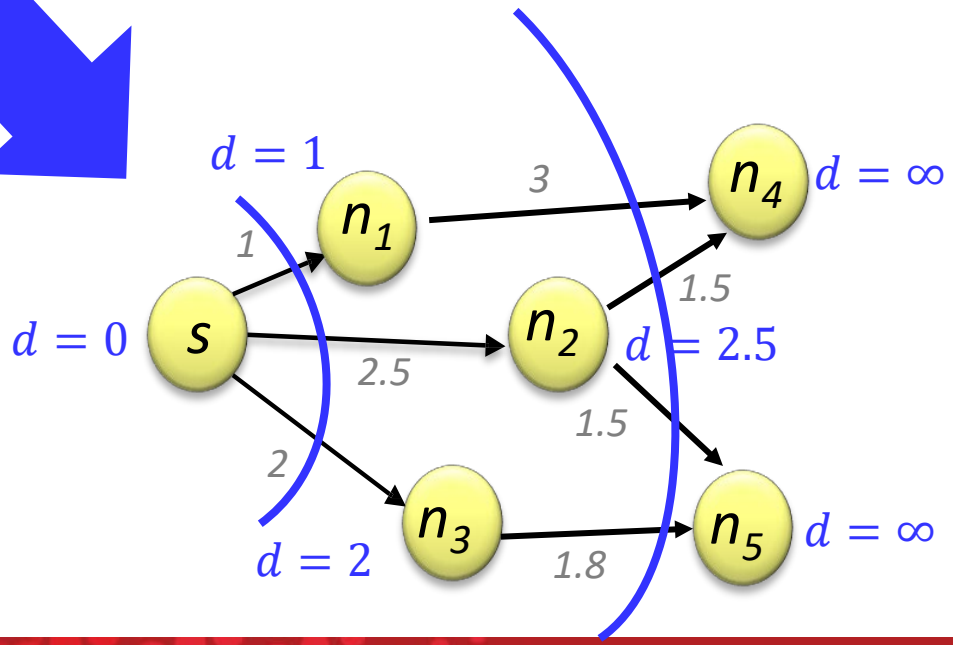


From Intuition to MapReduce Algorithm – first iteration



Initialization

First iteration



Stopping Criterion

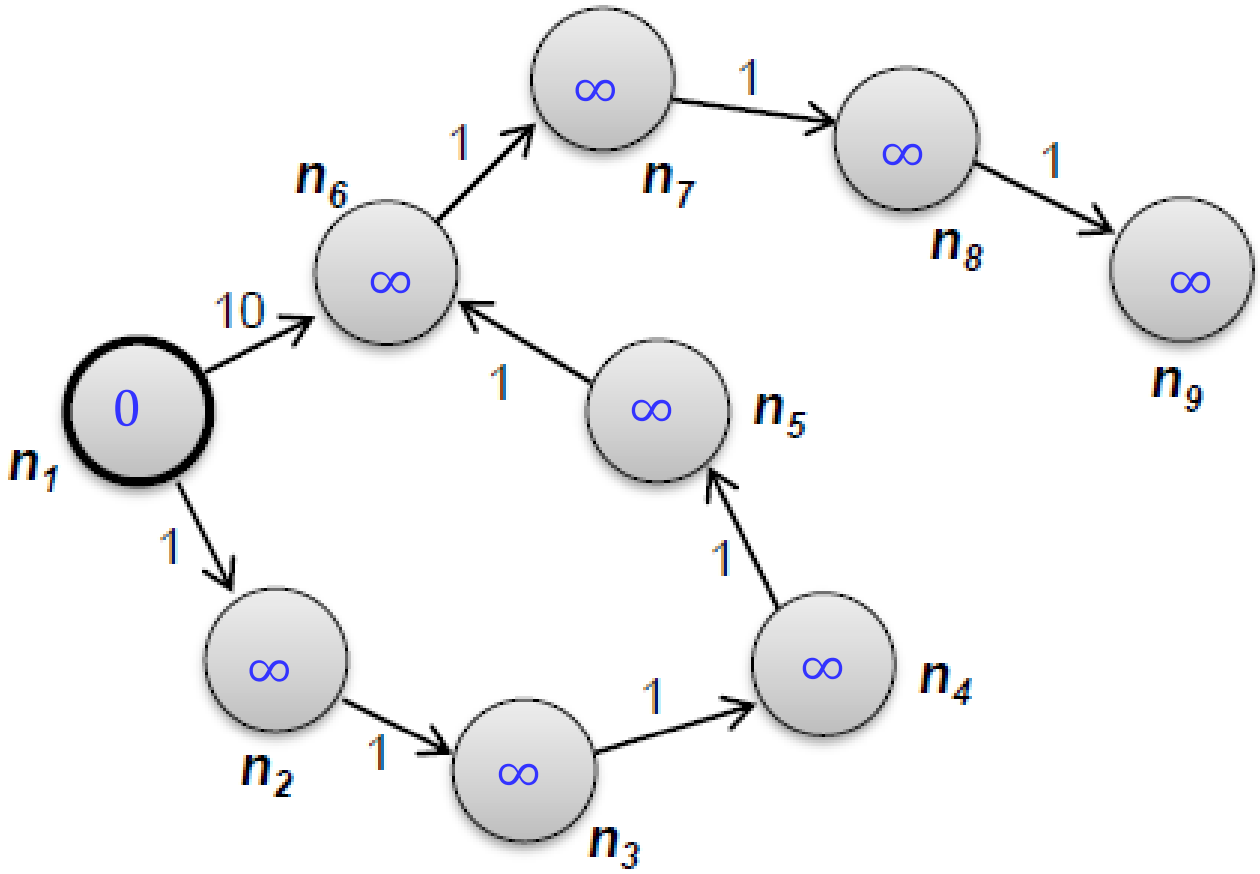
- The algorithm can terminate when distances at every node no longer change.
  - That is, the current distance at every node from the source after this iteration is same as that after last iteration.
  - Then, the current distance at every node is reported as the shortest path distance from source to this node.

Stopping Criterion

- The algorithm can terminate when distances at every node no longer change.
  - That is, the current distance at every node from the source after this iteration is same as that after last iteration.
  - Then, the current shortest distance at every node is the reported as the shortest path distance from source to this node.

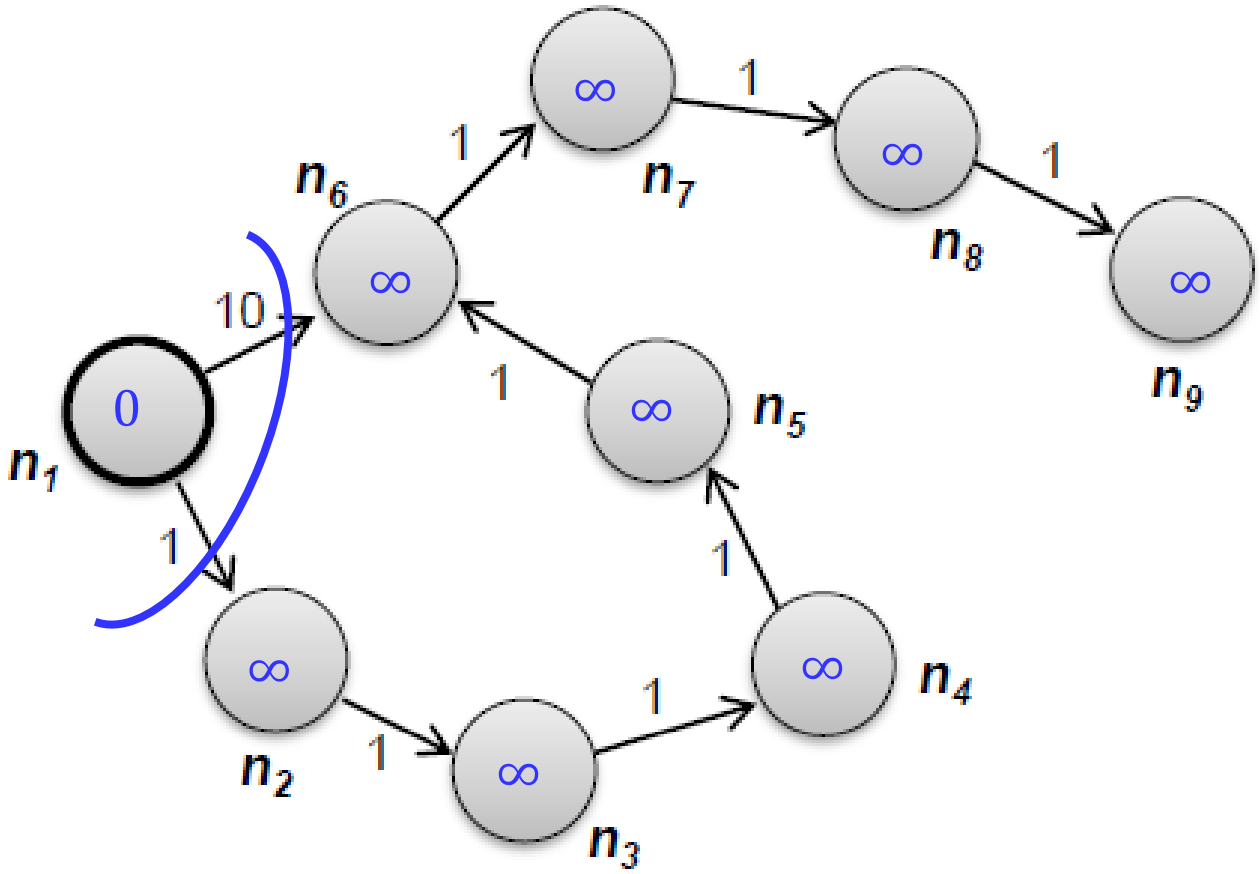
Stopping Criterion

From n1
of iterations?



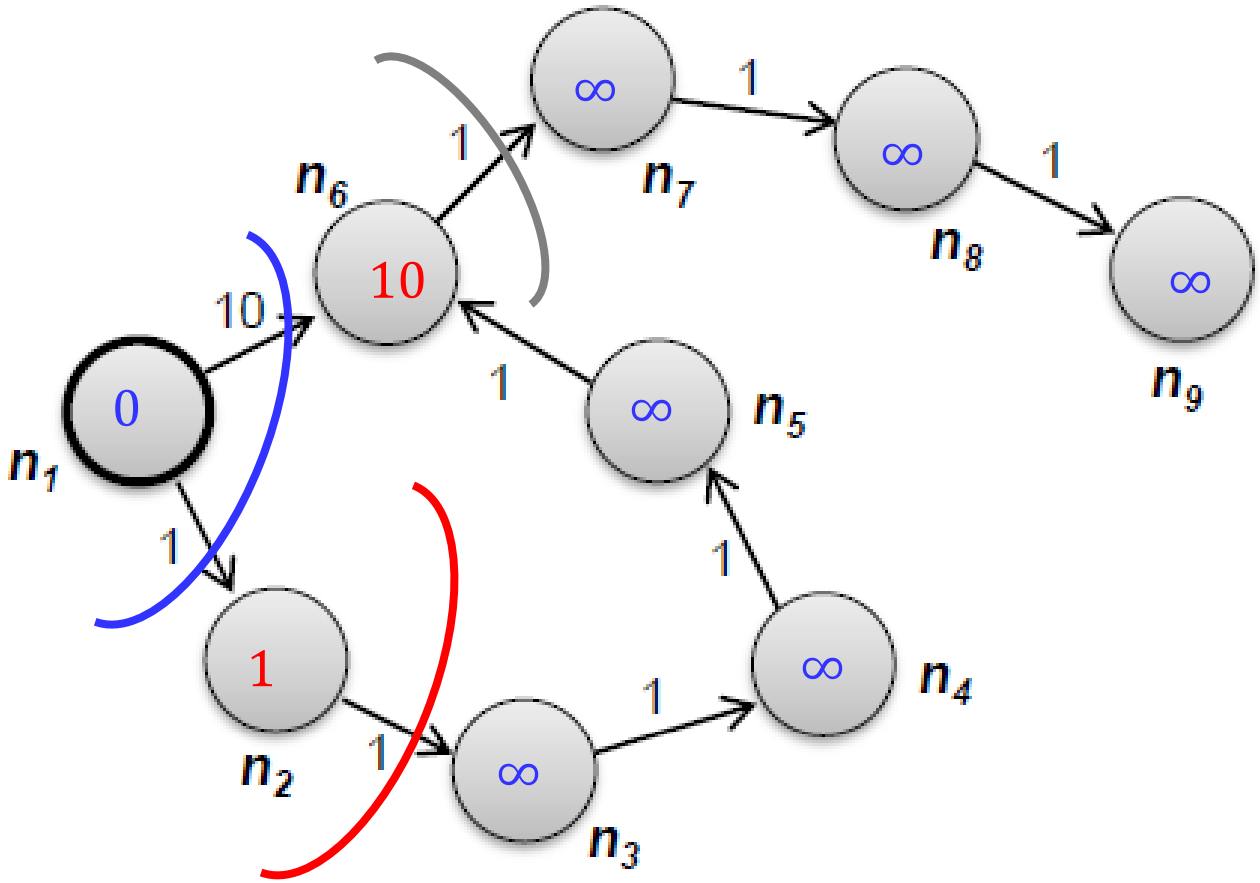
Stopping Criterion

From n1
of iterations?



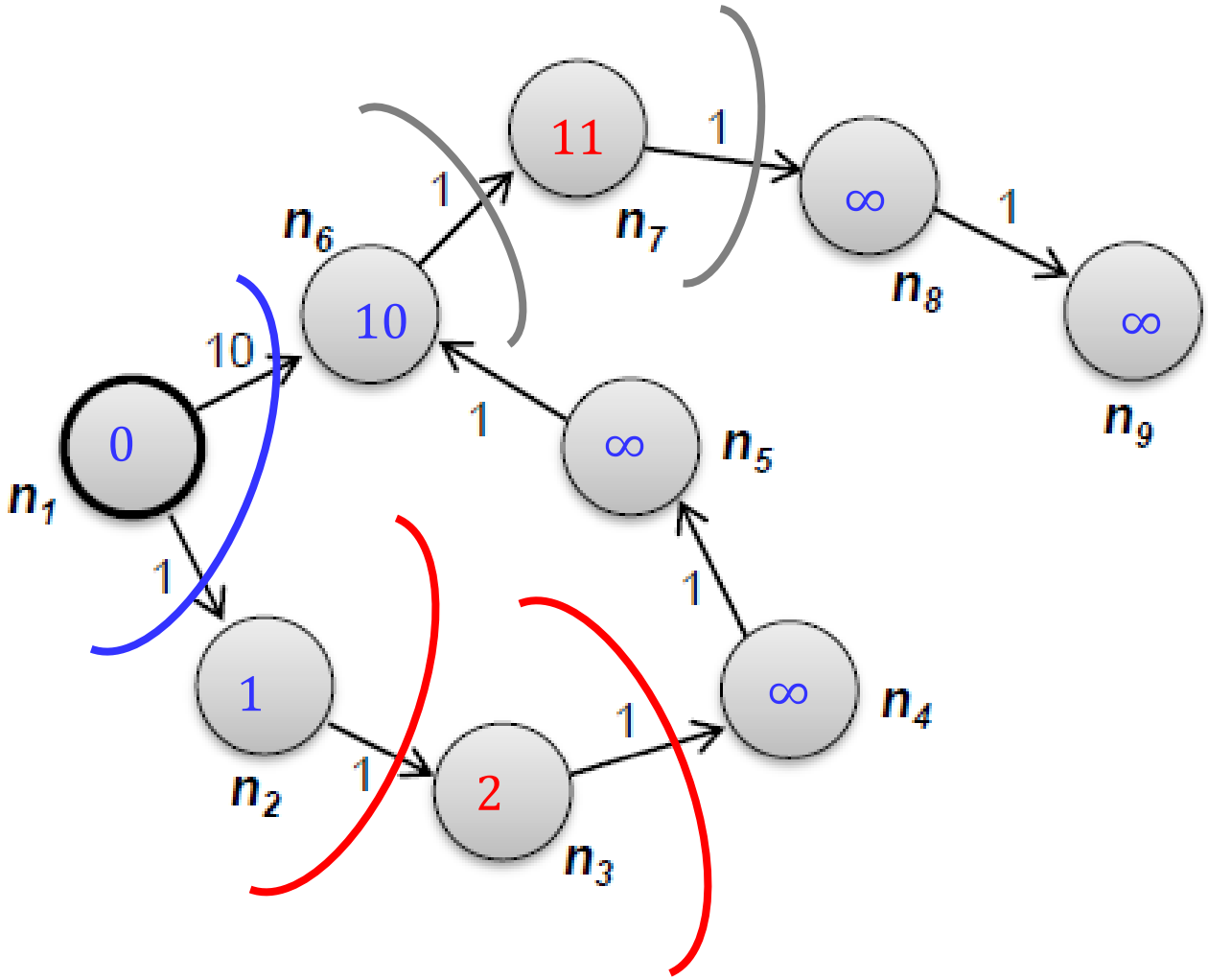
Stopping Criterion

From n1
of iterations?



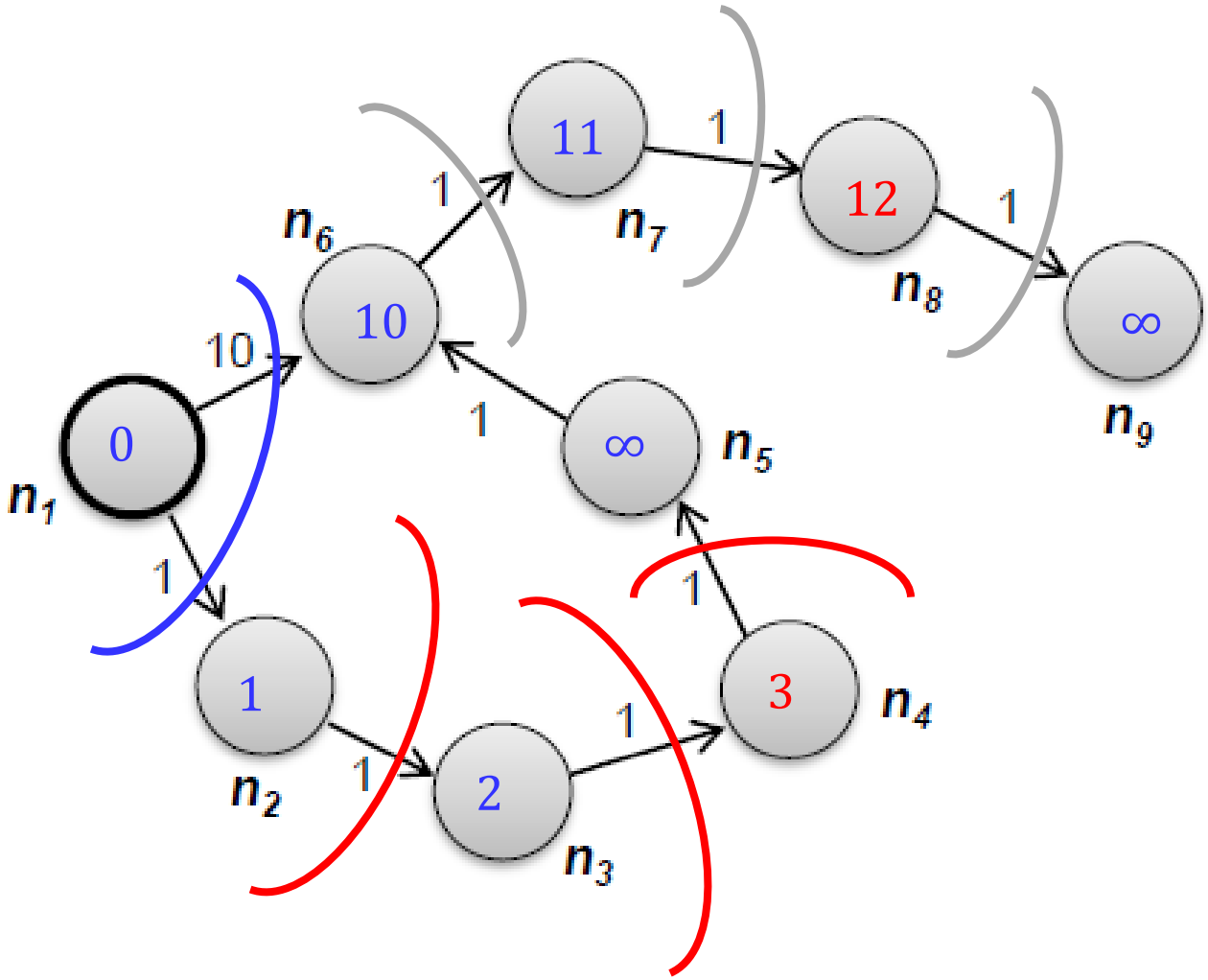
Stopping Criterion

From n1
of iterations?



Stopping Criterion

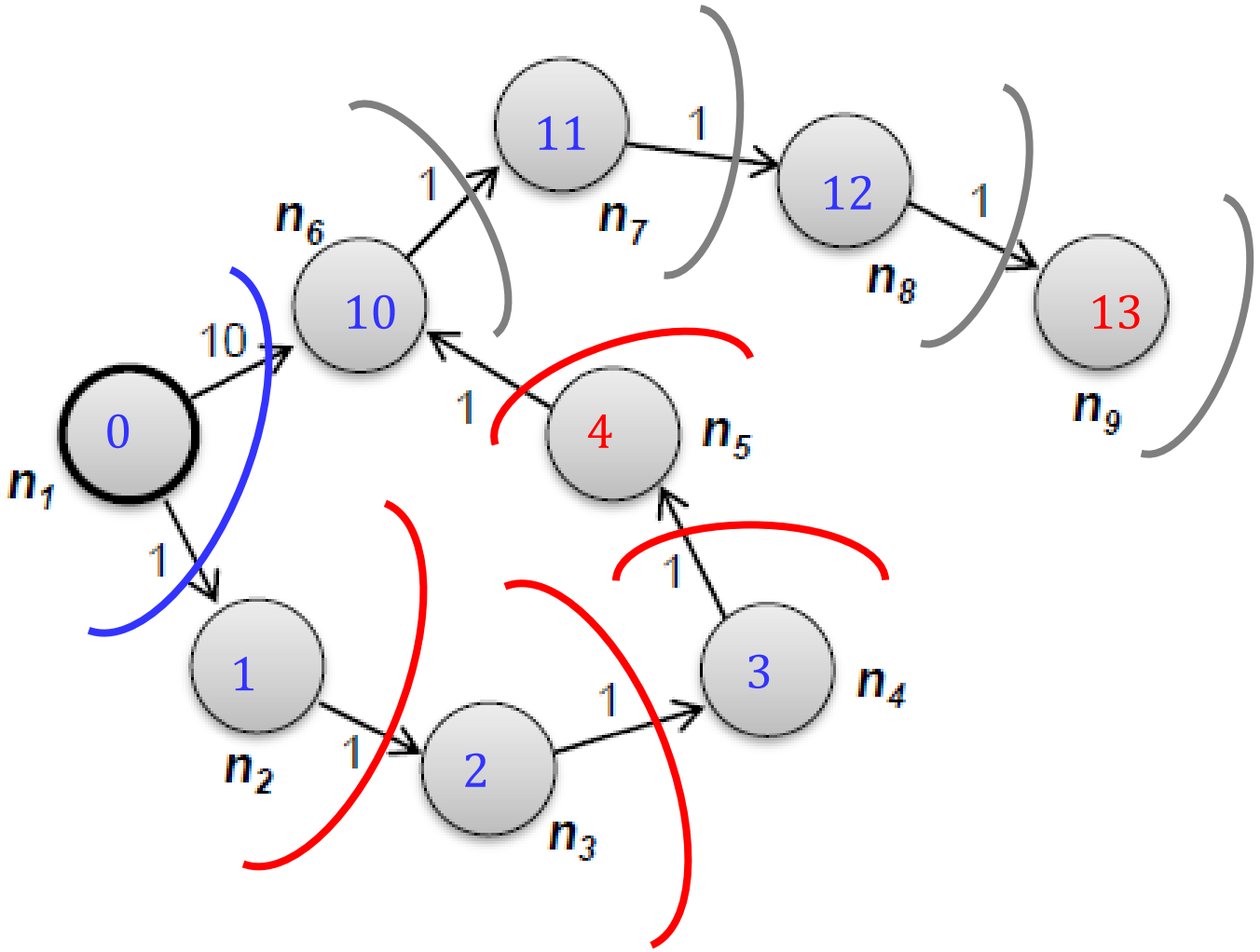
From n1
of iterations?



RMIT Classification:

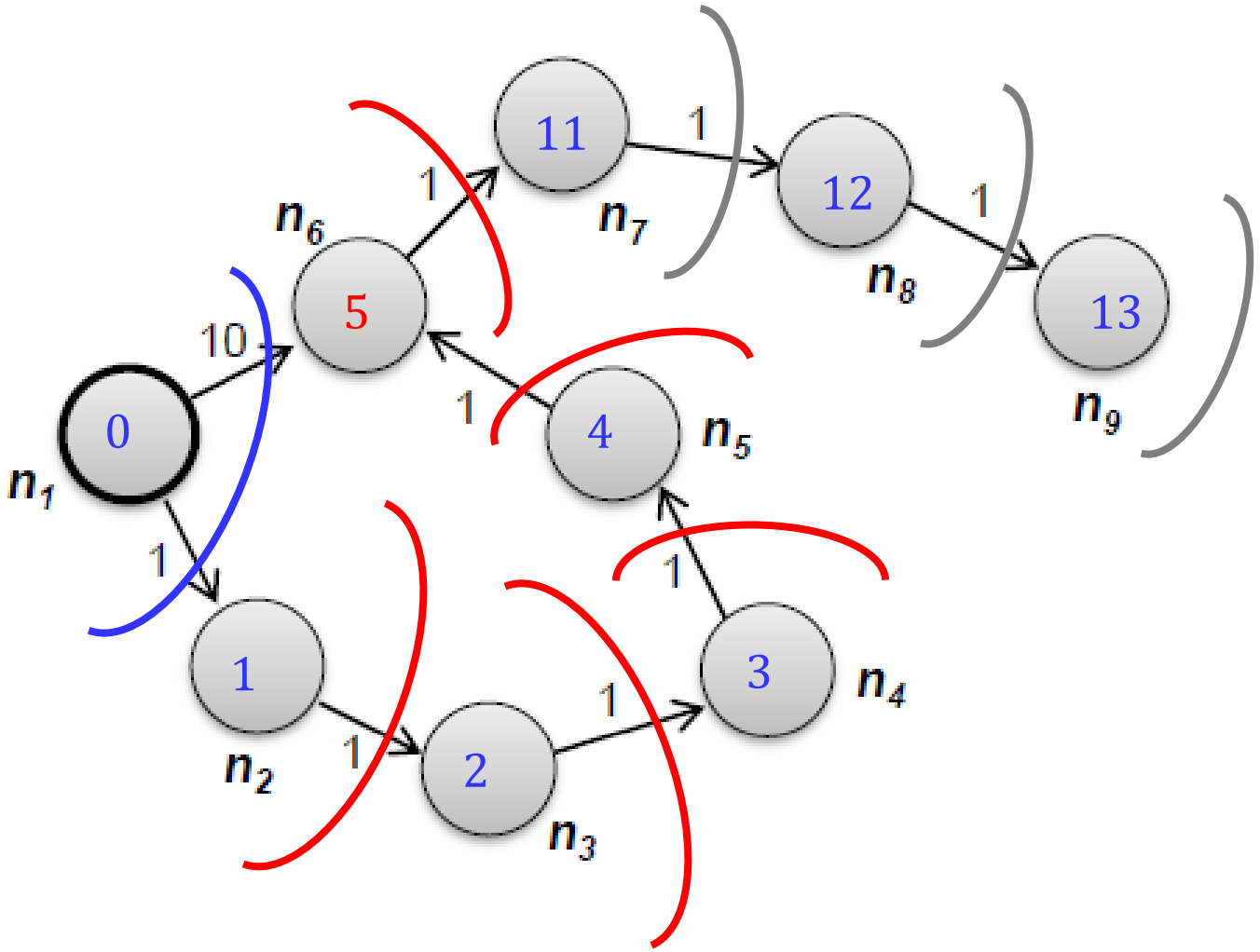
Stopping Criterion

From n1
of iterations?



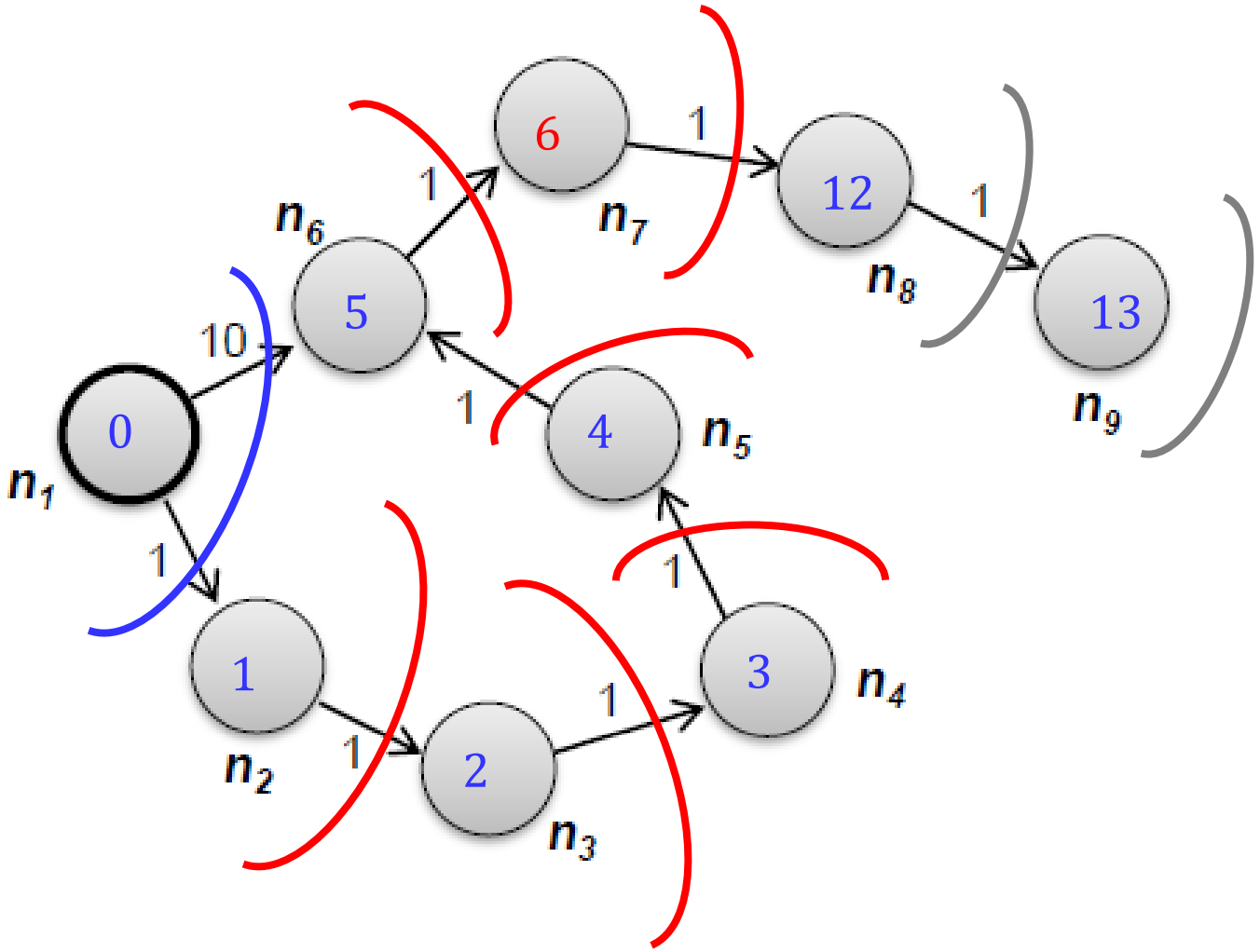
Stopping Criterion

From n1
of iterations?



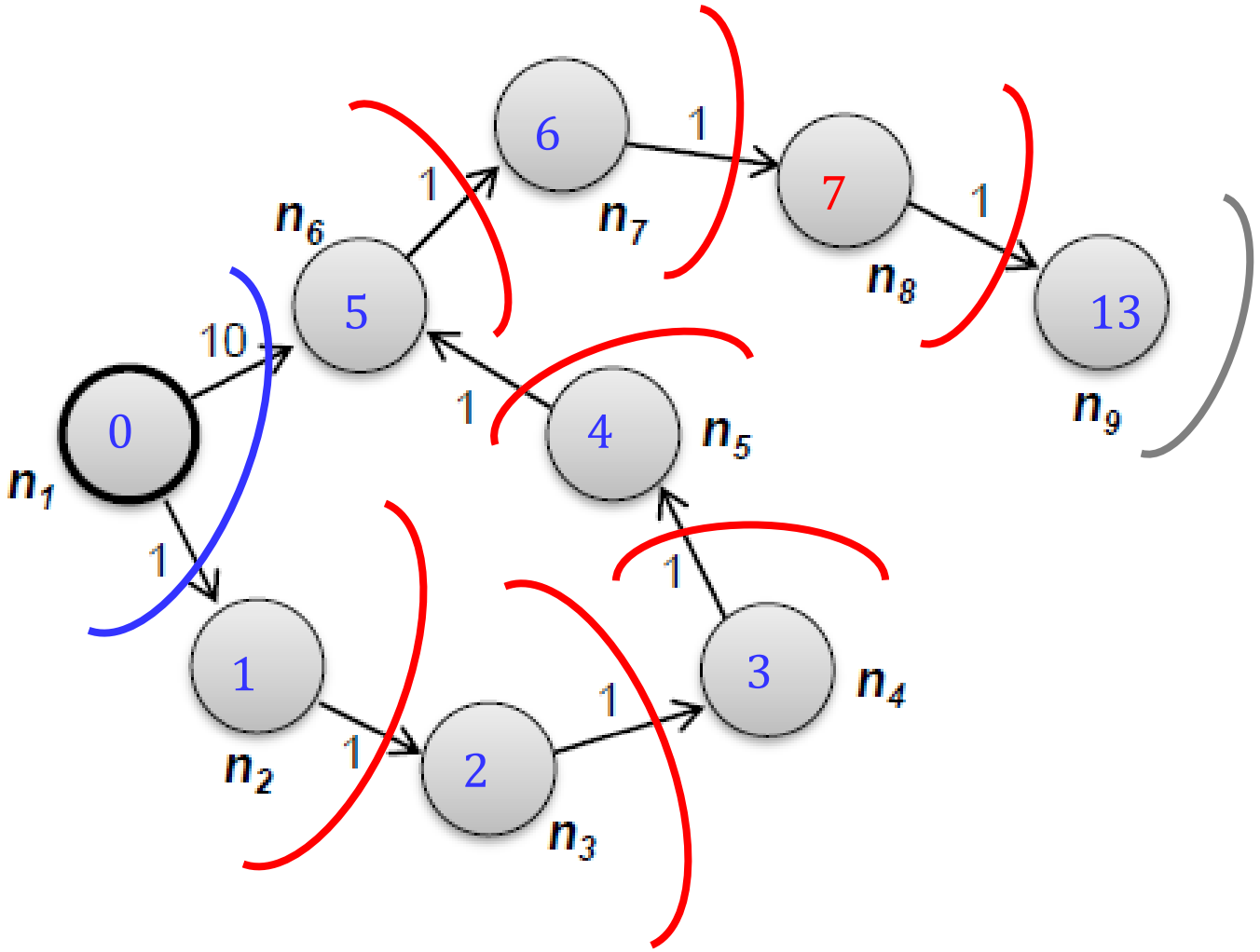
Stopping Criterion

From n1
of iterations?



Stopping Criterion

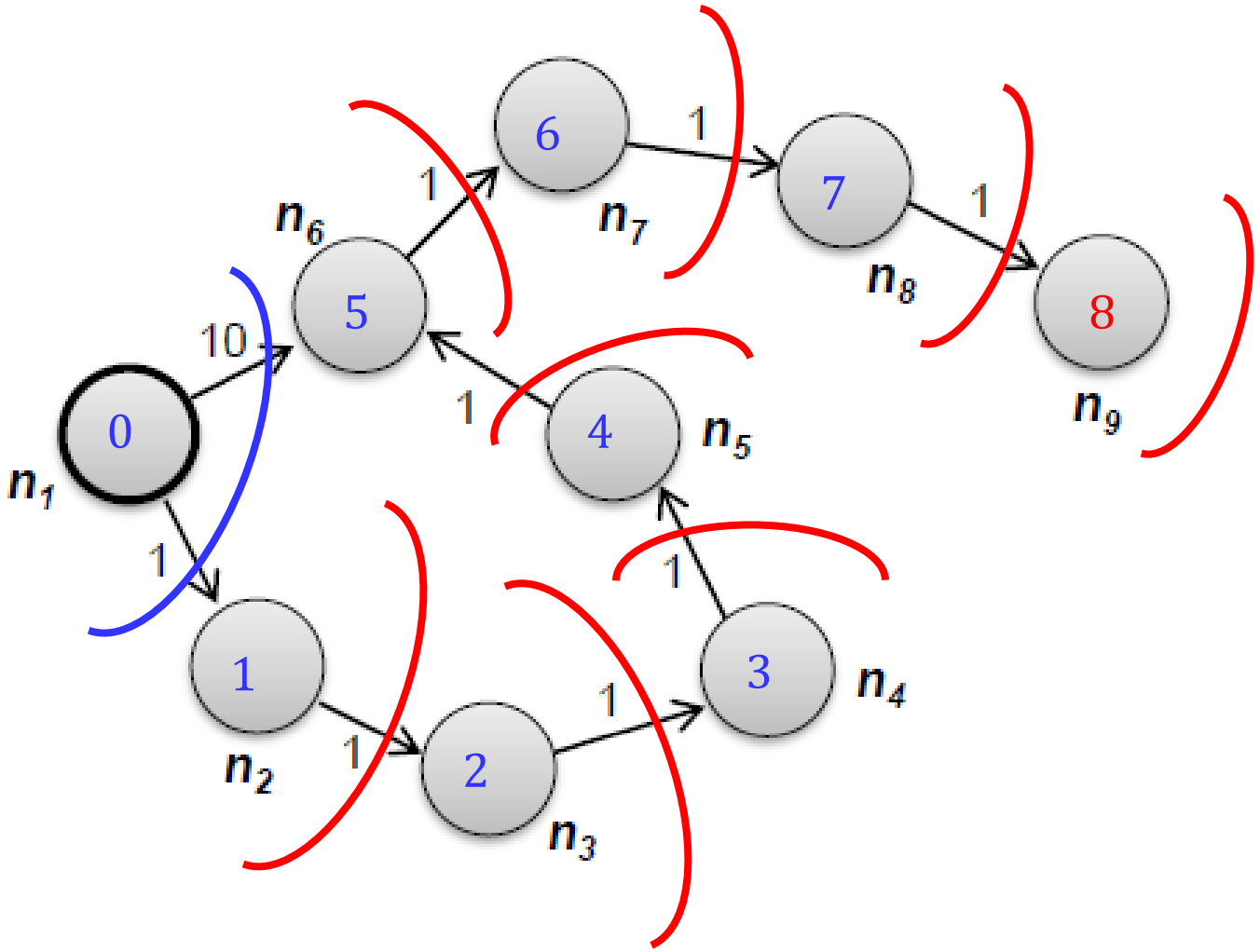
From n1
of iterations?



RMIT Classification:

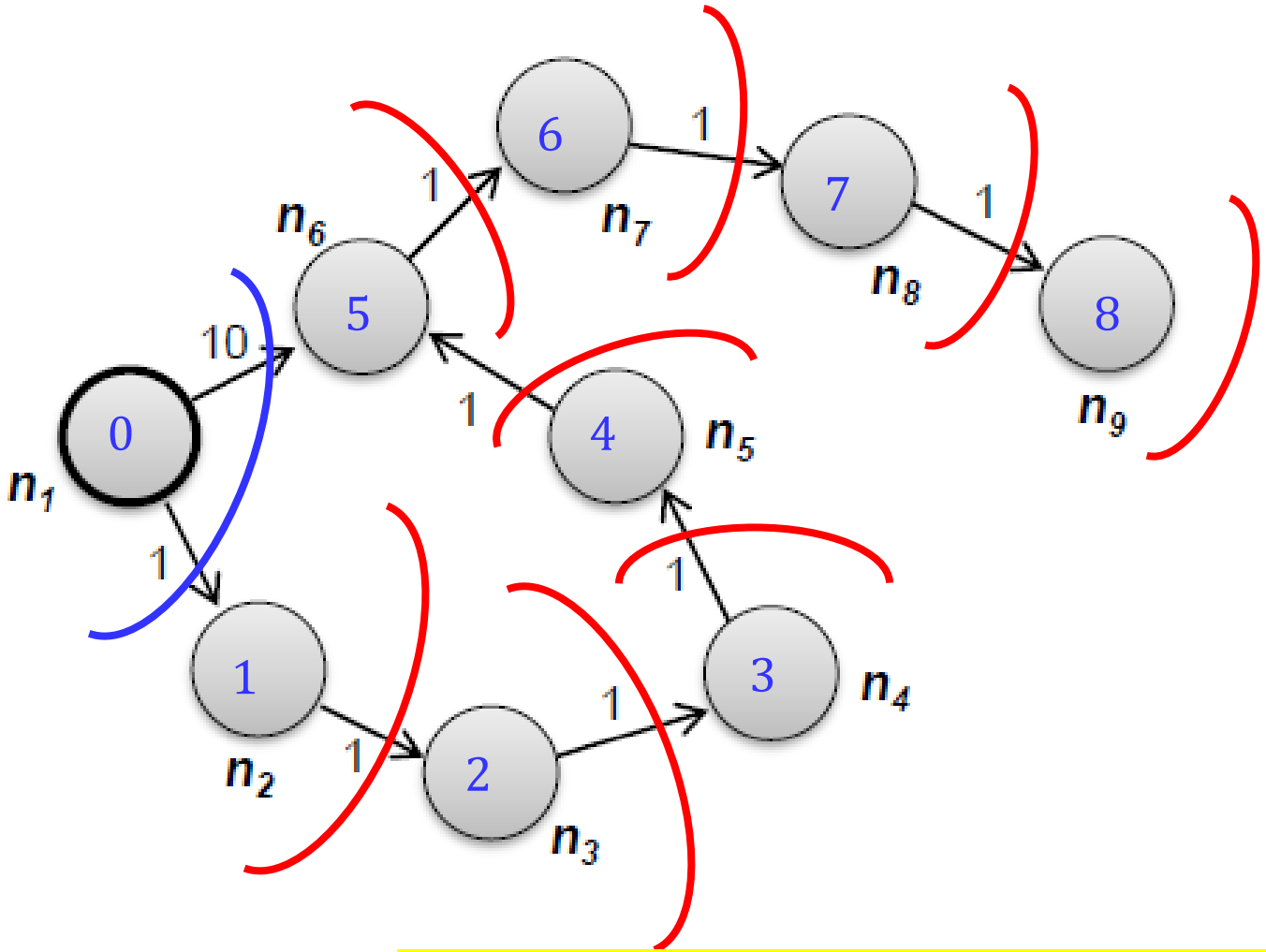
Stopping Criterion

From n1
of iterations?



Stopping Criterion

From n1
of iterations?



Stopping Criterion is satisfied

Comparison to Dijkstra

- Dijkstra's algorithm is more efficient
  - At each step, only pursues edges from minimum-cost path inside frontier
- MapReduce explores all paths in parallel
  - Lots of "waste"
  - Useful work is only done at the "frontier"

End of PART II

Next Lecture
Spark Basics